Neuromodulated Low-Rank Recurrent Neural Networks

A Mechanistic Account of Neuromodulation in Shaping Flexible Neural Dynamics

Shaunak Bhandarkar

A thesis presented for the degree of Bachelor of Science with Honors



Department of Mathematics

Stanford University

June 2024

Acknowledgements

Writing this thesis was an immensely rewarding process, and I am deeply grateful to all the people who helped me along the way. First and foremost, I would like to thank my collaborators – Julia Costacurta, David Zoltowski, and Scott Linderman – for conceiving the project idea and for helping me learn so much about neuromodulation, recurrent neural networks, dynamical systems, and everything in between. The work that we have done has been incredibly exciting and and I look forward to exploring future directions together. This thesis has grown out of several valuable discussions with each of them.

I would like to thank Jay McClelland as well as members of the PDP lab at Stanford for providing me with feedback as I was embarking on this project. I am also very grateful to my mathematics major advisor Brian Conrad for all of his thesis-related advice and, more broadly, for helping me navigate the math department throughout my time at Stanford.

Lastly, I would like to provide acknowledgement to Jake Hofgard, Kai Fronsdal, and Nicole Ticea for listening to my ideas and for teaching me about topics in math, computer science, and neuroscience along the way.

Neuromodulated Low-Rank Recurrent Neural Networks

A Mechanistic Account of Neuromodulation in Shaping Flexible Neural Dynamics

Shaunak Bhandarkar

Advisor: Dr. Scott Linderman

Humans and animals demonstrate an ability to adapt and generalize their behavior across various environments and contexts. A broad goal in computational and theoretical neuroscience is to understand how to model the neural dynamics underlying such flexible control. One widely-adopted approach for modeling neural computation is to train recurrent neural networks (RNNs) on relevant behavioral tasks, which captures the notion of behavior emerging through the evolution of a suitable dynamical system. However, a shortcoming of RNNs is that they make use of static weights, whereas synaptic weights in the brain are constantly being adjusted, in part by chemical neuromodulators (e.g., dopamine). Additionally, while RNNs often struggle to model long-sequence dependencies, neuromodulators – known to act across various timescales – may be able to selectively preserve relevant information over long input sequences. To that end, we develop a model – the neuromodulated low-rank RNN (NM-LR-RNN) – consisting of a pair of coupled RNNs in which one RNN generates a neuromodulatory signal that dynamically scales the recurrent weights of a low-rank RNN used to generate task outputs. Through empirical simulations, we find that the NM-LR-RNN reliably generalizes better than ordinary (low-rank and full-rank) RNNs and also possesses an advantage in modeling long-sequence dependencies. We also illustrate how neuromodulation in the NM-LR-RNN constitutes a "gate-switching" mechanism akin to those used by gated RNN models in machine learning as well as by latent variable models for inferring dynamical systems. Finally, we carry out theoretical analyses of the NM-LR-RNN's neuromodulatory signal, exploring how it is constrained by the task structure as well as how it enhances the fixed point structure of the output-generating low-rank RNN's dynamics.

Contents

1 Introduction **1**

1.1 Towards Flexible Neural Computation 1

1.2 Mechanistic Models in Neuroscience 2

1.2.1 Connectionism 2

1.2.2 Dynamical Systems in Neuroscience 3

1.3 The Mysteries of Neuromodulation 5

1.4 Problem Statement 7

1.5 A Roadmap of this Thesis 7

2 Low-Rank Recurrent Neural Networks **9**

2.1 What is a Low-Rank RNN? 9

2.2 Low-Rank RNNs with Tonic Contextual Inputs can Generalize 10

2.3 The Case for Neuromodulation 13

3 Neuromodulated Recurrent Neural Networks **16**

3.1 Model Design 16

3.2 How Does the Model Map onto the Biology? 19

3.3 Neuromodulation Adjusts the Spatial and

Temporal Modes of Computation 19

3.3.1 Gating the Spatial Structure of Neural Dynamics 20

3.3.2 Controlling Timescales of Dynamics 21

3.4 A Curious Connection to LSTMs 23

4 Empirical Results **34**

4.1 Time Interval Reproduction 34

4.1.1 Measure-Wait-Go Revisited 35

4.1.2 Low-Rank Neuromodulated RNNs can Generalize 36
4.1.3 Non-Neuromodulated RNNs Struggle to Generalize 39
4.2 Element Finding Task 41
4.2.1 Models of Comparison 42
4.2.2 NM-LR-RNNs can be LSTM-like Learners 43
4.2.3 How do Neuromodulated RNNs Solve the Element Finder Task? . 45
4.2.4 Analysis of the Neuromodulatory Sub-network 47
4.2.5 Analysis of the Output-generating Network 52

5 Building a Theory of Neuromodulation 56
5.1 How is Neuromodulation Constrained by the Output Signal? 56
5.2 Timing Task Revisited 58
5.2.1 Linearized Rank-1 Networks 59
5.2.2 Linearized Symmetric Rank-1 Networks 62
5.3 Fixed Point Analysis 65
5.3.1 Linear Symmetric NM-LR-RNNs 65
5.3.2 Rank-1 Neuromodulated RNNs 67

6 A Foray into Dynamical Mean Field Theory 70
6.1 Low-Rank Dynamics in Low-Rank RNNs 71
6.2 Fixed Point Analysis for Low-Rank RNNs 73
6.2.1 The Case of All Real Eigenvalues 76
6.3 Incorporating Neuromodulation 79
6.3.1 The Case of Real Eigenvalues (Revisited) 79

7 Neuromodulated RNNs as Recurrent Switching Linear Dynamical Systems 83
7.1 rSLDS Model Design 84
7.2 Mapping the Fully Linearized Coupled NM-LR-RNN onto the rSLDS Framework 86

8 Conclusion 90
8.1 Summary 90
8.2 Future Work 91

References **93**

Chapter 1

Introduction

1.1 Towards Flexible Neural Computation

Owing to their brains, humans and many kinds of animals show an ability to adapt and generalize their behavior across various environments and contexts. The human brain, in particular, consists of massive, distributed neural circuitry, and the activities of various populations of neurons across the brain give rise to sophisticated, context-dependent behaviors, like timing, navigation, and cognitive control. Even in social cognition, interpreting communicated information is highly context-specific; such context must be flexibly inferred in generating a response. Experiments in non-human primates has provided evidence of flexible control mechanisms in frontal cortex [1].

A central goal in theoretical neuroscience is to uncover the foundational computational principles of the brain that enable flexibility and generalization in behavior. In solving this problem, we encounter multiple important questions regarding neural computation:

1. What kinds architectures and connectivities (or, broadly speaking, *inductive biases*) should neural populations have to flexibly adapt their behavior within and/or across tasks?

2. What are the learning rules by which neurons learn task structure, and over what timescales can learning happen?

3. From a biological standpoint, does computation solely arise from neuronal activity, or are there other biological signals that can produce flexible, context-dependent behavior?

In this thesis, we will visit each of these questions while exploring the features and abilities of a neural network model of neural population dynamics.

1.2 Mechanistic Models in Neuroscience

A core goal in computational neuroscience is to understand the abstract computations underlying behavior and overlying specific neural instantiations (of such behavior) in the brain. One way to work towards this goal is by designing *mechanistic models* of neural computation, which aim to reproduce certain aspects of behavior by implementing suitable abstractions of biological mechanisms. Two influential frameworks have given rise to a large body of mechanistic models used in neuroscience: connectionism and the dynamical systems framework.

1.2.1 Connectionism

One way to explain how neural populations organize to accomplish various tasks is to create a simplified, yet biologically-faithful, model that simulates neural population activity. Beginning with simple "perceptron" models that simulated neuron-like units for integrating incoming input information [2, 3], *connectionism* was an approach to computationally understanding cognition that was developed and popularized by Rumelhart and McClelland [4]. In contrast to symbolic approaches to studying cognition, connectionism was built on the notion that real neural computation occurs in parallel across distributed systems in the brain; thus, to model cognitive phenomena, one could (in principle) design an entire network of neuron-like units, adjoined via synaptic weights, that (nonlinearly) propagated information across the different layers of such a system. Various early artificial neural network models were used to study aspects of language processing [5, 6, 7, 8], memory [9, 10, 11, 12], semantic cognition [13] cognitive control [14, 15], and neural coding mechanisms in the brain [16, 17, 18, 19]. Among the core tenets of connectionism [20] are that, in an artificial neural network model (modeling some aspect of cognition):

1. Cognitive processes arise through the propagation of neural activity via connection weights.

2. Neural activity is distributed over neuron-like units.

3. Learned knowledge is encoded within the connection weights (i.e., synapses).

4. Learning and representation are graded (continuous) phenomena, rather than discretely updating over time.

5. Learning depends on the statistics of the inputs (from the environment).

Connectionism has also largely given rise to *deep learning*, in which artificial neural networks consist of several layers stacked in succession. In recent years, deep neural networks have emerged as promising models for studying the brain [21]. They have been used to understand cognitive processes such as semantic cognition [22], abstraction [23], and reinforcement learning [24], as well as neural systems like the ventral visual stream [25] and the hippocampus [26].

1.2.2 Dynamical Systems in Neuroscience

Another broad class of computational models in neuroscience arises from studying neural computations as time-varying dynamical processes, i.e., systems whose evolution are governed by suitable differential equations. For instance, the early Hodgkin-Huxley provided a circuit model that quantitatively described the dynamics underlying action potentials [27] (i.e., the electrical signals sent by neurons to adjoining neurons). Additionally, many models of neural systems have modeled neural firing rates as well as synaptic plasticity via continuous-time dynamics [28, 29, 30]. In recent years, dynamical systems have featured in statistical methods for inferring the dynamics present in real neural populations [31, 32, 33], fueled by interest in understanding how neurons encode information and interact with one another.

A recurrent theme in theoretical neuroscience is that of understanding how populations of neurons interacting with one another may be viewed as a dynamical system [34]. In broad terms, we may view neural dynamics as being governed by equations of the form

$$\tau \frac{dx}{dt} = F(x(t), u(t)) \tag{1.1}$$

where $x(t) \in \mathbb{R}^N$ is a representation of neural firing rates in the population of interest, $u(t) \in \mathbb{R}^{N_{in}}$ is the time-varying input signal (e.g., coming from other neurons), and τ

is a constant governing the timescale of dynamics. As it turns out, many instantiations of Equation 1.1 come in the form of connectionist models – especially *recurrent neural networks* (RNNs). RNN dynamics are typically given by equations of the form

$$\tau \frac{dx}{dt} = -x(t) + J\varphi(x(t)) + Bu(t) \tag{1.2}$$

where $J \in \mathbb{R}^{N \times N}, B \in \mathbb{R}^{N \times N_{in}}$ are (learned or random) connection matrices and $\varphi(\cdot)$ is some nonlinear activation function (e.g., tanh). (While we may build RNNs with multiple layers of recurrently connected neurons, we primarily study the above one-layer RNN, which we will refer to as a "vanilla RNN.") Owing to their mathematically precise description, RNNs have been the source of many theoretical analyses [35, 36, 37, 38], which draw from concepts in dynamical systems theory and statistical mechanics. RNNs have also found applications in studying the computations present in real high-dimensional neural systems [39].

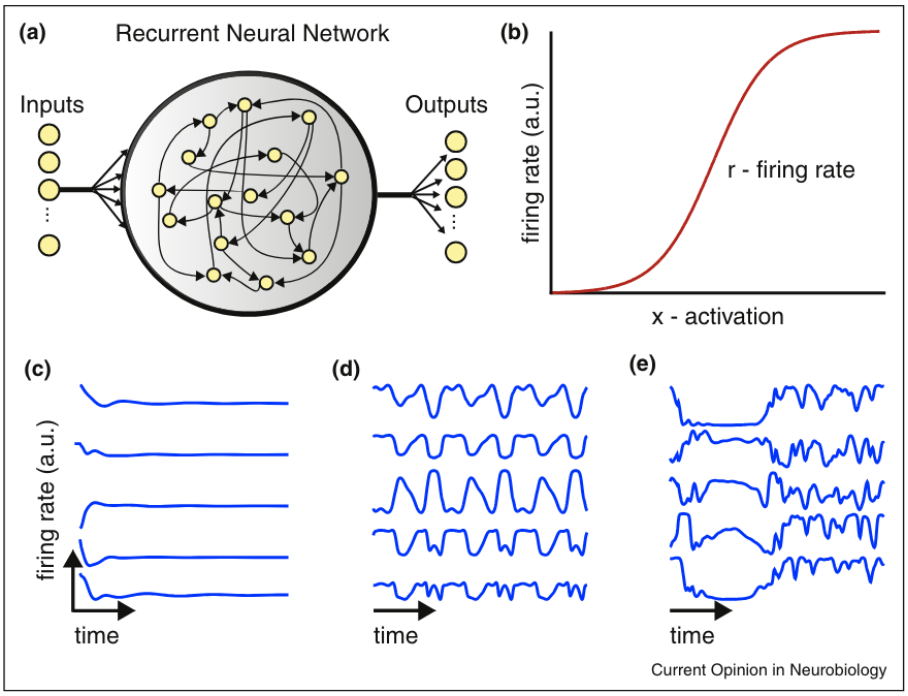


Figure 1.1 Schematic of a vanilla RNN along with possible dynamics. Figure taken from Susillo (2014) [39]. **(a)** Vanilla RNN with inputs, along with a readout to output states. **(b)** Possible graph of a nonlinear activation function φ so that the firing rate r and net activation x are related via $r = \varphi(x)$. **(c)-(e)** Possible dynamics for 5 selected components of the recurrent state variable x. (c) shows settling to an attractor state, (d) shows oscillatory behavior, and (e) shows chaotic behavior.

1.3 The Mysteries of Neuromodulation

In this thesis, we aim to study the computational properties of *neuromodulatory signals*, which are known to have wide-reaching control over the neural activity underlying behavior. Neuromodulators – often chemical agents like acetylcholine, peptides, dopamine, and glutamate – are signaling molecules that can induce long-lasting changes within neural populations; they may alter strengths of individual synapses (adjoining a pair of neurons), as well as those of many neurons simultaneously [40]. According to Dayan, three big questions in understanding neural processing is that of "who talks to whom, what they are allowed to say, and how the answers to these questions can change"; neuromodulators may present the answers to these questions [41]. In particular, neuromodulators are widely distributed across the brain, and can exert control over different neural population sizes, whereas synaptic transmission between neurons is local and restricted. Additionally, neuromodulators determine the kind of information that is propagated across neurons (e.g., excitatory versus inhibitory signals). Moreover, by influencing the excitability of neurons and synapses over various timescales, neuromodulation offers the potential to facilitate flexible context-specific behaviors. According to Marder, neural circuits provide a minimal architecture on top of which neuromodulators specify the functional properties (of such circuits) that ultimately give rise to behavior [42].

Developing a computational model involving neuromodulation thus requires a specification of the requisite neural architecture along with the specific means by which neuromodulatory signals influence neural computation. (Certain simple architectures incorporating neuromodulation are shown in Figure 1.2.) For example, incorporating a neuromodulatory parameter into a first-order ODE describing the dynamics of (slowly-changing) neuronal membrane current, Abbott (in 1990) constructed a recurrent network that was capable of switching between being a selective, long-term memory storage and a nonselective, short-term memory buffer [43]. This highlighted a notion of neuromodulation serving as a gating mechanism between short- and long-term memory. More recent work has involved using an adjustable neuromodulatory parameter to globally modulate the dynamics of a spiking recurrent neural network [44] as well as using neuromodulation to selectively scale synaptic weights (corresponding to particular neurons) in the recurrence matrix of a vanilla RNN [45].

Still, much is unknown about the computational underpinnings of neuromodulation

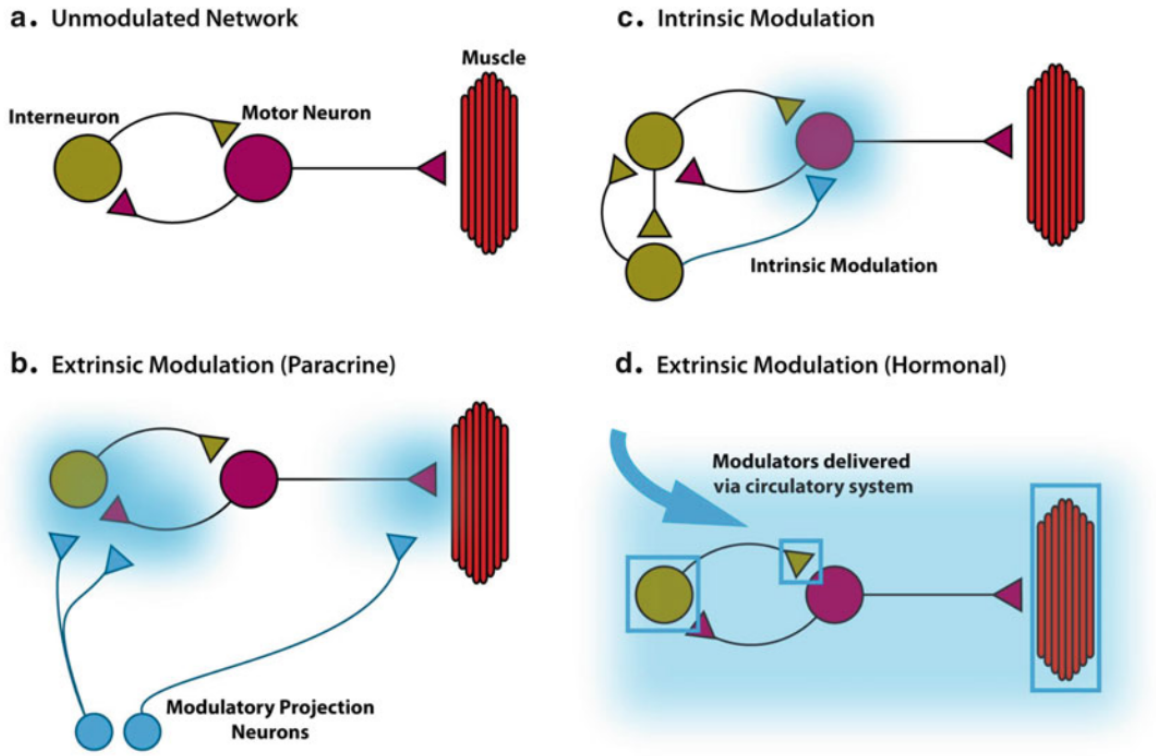


Figure 1.2 Different ways in which neuromodulators can target a neural system. Figure taken from Williams et al (2013) [40]. **a.** A simple motor circuit with no neuromodulation. **b.** A motor network that is modulated by modulatory neurons that are extrinsic to the network. **c.** A network in which the source of neuormodulation comes from within. **d.** A network in which neuromodulators are hormonally delivered.

in the brain. While neuroscientists are familiar with specific neuromodulatory agents and instances in which they influence neural activity, much remains to be known about how neuromodulation can exert control over (large) neural populations, thus globally altering neural dynamics. Even for specific neuromodulators, like dopamine, it remains to be seen whether the role of such signals across the brain can be understood within a unified computational framework. Answering these questions would enable us to build a fuller picture of neural computation (i.e., that includes more than just an understanding of neurons and synapses) and understand how neural systems might malfunction when neuromodulation fails to be delivered. It also offers the potential to enhance existing artificial neural network architectures, adding an additional element of flexibility [45].

1.4 Problem Statement

Given the potential for neuromodulation to flexibly control neural dynamics – whether by chemical signaling or by modulatory neurons – the goal of this thesis is to explore how neuromodulation might suitably be incorporated into existing mechanistic models in neuroscience. Given that RNNs have been widely used in modeling systems-level neural dynamics, we seek to incorporate a computational analog of neuromodulation into the RNN framework (somewhat similarly to Tsuda et al (2022) [45]). In particular, given that RNNs are traditionally modeled with *static* weights, incorporating an additional element of neuromodulatory *dynamics* into these weights could more biologically faithfully model the ever-changing strengths of synapses connecting neurons. Thus, we arrive at our central research question:

Question 1. Can a suitable notion of neuromodulation be incorporated into an RNN model of neural population dynamics, and can this account for flexible control and generalization across changing contexts?

In this thesis, we design an RNN-variant that incorporates neuromodulation to achieve flexible and structured dynamics. Our proposed model consists of a small "neuromodulatory" RNN that supplies a neuromodulatory signal to a larger (but low-rank) RNN that generates the model's output. In particular, the neuromodulatory RNN modulates each rank-1 component of the output-generating RNN's recurrence matrix, affording flexible yet intelligent switches in the dynamics followed by the output-generating RNN.

Building atop existing mathematical analyses of RNNs (e.g., fixed point stability analyses, mean-field theory), we crucially aim to carry out similar analyses of RNNs with neuromodulation. Throughout this paper, we will keep revisiting the notion of neuromodulation as a gating mechanism, analyze how neuromodulatory signals are constrained by target output signals, as well as explore how neuromodulation alters the fixed point structure in RNNs.

1.5 A Roadmap of this Thesis

In order to work towards answering our central question, we begin by exploring the low-rank RNN framework and its applications to a time interval reproduction task (Measure-

Wait-Go) in Chapter 2. In Chapter 3, we introduce our central model of study – the neuromodulated low-rank recurrent neural network (NM-LR-RNN) – and explore the concept of neuromodulatory gating. We also derive a mathematical connection between our model and a well-known gated recurrent neural network model: the LSTM.

Having introduced the neuromodulated low-rank RNN, in Chapter 4 we explore the performance of the NM-LR-RNN on two tasks relating to flexible control (of relevance in neuroscience), assessing how well our model is able to generalize to unseen contexts. We also carefully analyze the roles taken on by the different components of our model – especially the neuromodulatory component. In Chapter 5, we build on these results by quantitatively characterizing how neuromodulation is constrained by the target outputs (for any given task) for (linearized) rank-1 networks and by mathematically analyzing the fixed point structure afforded by our model.

We continue our fixed point analyses in Chapter 6 by foraying into the dynamical mean-field theory (DMFT) [35, 46]; this method, originating from tools in statistical mechanics, enables us to understand the fixed point structure in randomly initialized networks in the limit of infinitely large network size. Then, in Chapter 7, we visit a curious connection between our (mechanistic) neuromodulated RNN model and a (descriptive) Bayesian latent variable model that has been used to infer latent dynamics in real neural data: Recurrent Switching Linear Dynamical Systems (rSLDS) [47, 48]. This will highlight the notion of the NM-LR-RNN as a model whose global dynamics consist of simpler dynamical systems that are stitched together by the neuromodulatory component; moreover, continuously varying the neuromodulatory component induces a continuous transition between these simpler dynamical systems.

We finally present our conclusions and look to future work in Chapter 8.

Chapter 2

Low-Rank Recurrent Neural Networks

In Chapter 1, we introduced recurrent neural networks (RNNs) as useful models for studying neural dynamics. However, using large RNNs to solve a given task often comes at the cost of interpretability. Additionally, many tasks on which RNNs are trained are inherently low-dimensional, potentially making many dimensions of dynamics superfluous in large RNNs. Finally, training large RNNs on low-dimensional datasets may pose an issue with overfitting to the training data, potentially hindering generalization ability. To address these potential issues, we introduce a particular kind of recurrent neural network model that constrains the effective dimensionality of dynamics: the *low-rank RNN*, first introduced by Mastrogiuseppe and Ostojic in 2018 [46].

We will find that low-rank RNNs are effective at solving *and* exhibiting generalization within a time interval reproduction task when aided by contextual input signals; in contrast, full-rank RNNs fail to successfully generalize to unseen data. However, we highlight limitations with this approach, which ultimately motivates the design of our central model of study in Chapter 3.

2.1 What is a Low-Rank RNN?

Recall the dynamics equation governing the update of the state variable $x(t) \in \mathbb{R}^N$ in a vanilla RNN:

$$\tau \frac{dx}{dt} = -x(t) + W_{xx}\varphi(x(t)) + B_{xu}u(t)$$

where $u(t) \in \mathbb{R}^{N_{in}}$ is the input signal, $W_{xx} \in \mathbb{R}^{N \times N}$ and $B_{xu} \in \mathbb{R}^{N \times N_{in}}$ are the recurrent and input weights, respectively, and $\varphi(\cdot)$ denotes a nonlinear activation function, which we set to be tanh. By nature of this design, the recurrence matrix W_{xx} almost surely has full rank N. To address the issues described at the start of this chapter, we may enforce rank regularization of the recurrence matrix. That is, if we wish for the RNN's dynamics to effectively "live" in R-dimensional space (where $R << N$), then we may define $W_{xx} := \frac{1}{N} UV^T = \frac{1}{N} \sum_{r=1}^{R} u^{(r)} v^{(r)^T}$, where $U, V \in \mathbb{R}^{N \times R}$ are learnable parameters, and where $u^{(1)}, \ldots, u^{(R)}$ and $v^{(1)}, \ldots, v^{(R)}$ denote the columns of U and V, respectively. Since W_{xx} is now made to be the sum of R rank-1 matrices, it follows that W_{xx} has rank R. This gives us the dynamics equations for the *low-rank RNN*:

$$\tau \frac{dx}{dt} = -x(t) + \frac{1}{N} UV^T \varphi(x(t)) + B_{xu} u(t) \tag{2.1}$$

$$y(t) = Cx(t) + r \tag{2.2}$$

Here, the output $y \in \mathbb{R}^{N_{out}}$ is obtained by passing x through a linear readout, and $C \in \mathbb{R}^{N_{out} \times N}$ and $r \in \mathbb{R}^{N_{out}}$ are learnable parameters. Because $R << N$ (usually), such a model is governed by a minimal connectivity structure [46]. Furthermore, given that many high-dimensional representations of neural activity tend to effectively operate within lower-dimensional spaces when performing low-dimensional tasks [49, 50, 51], the low-rank RNN is an appealing framework for modeling low-dimensional (and more interpretable) computation while retaining a high-dimensional neural representation (given by $x(t)$). Inspired by the fact that the low-dimensionality of the low-rank RNN may help in facilitating generalization, we turn to analyzing this model on a timing task, highlighting its advantages over full-rank (vanilla) RNNs.

2.2 Low-Rank RNNs with Tonic Contextual Inputs can Generalize

In this section, we compare the generalization abilities of low-rank and full-rank RNNs, recapitulating empirical results from Beiran et al (2023) [52]. (For completeness, it should be noted that the readout equation used for the RNNs tested in this work is given as $y(t) = C\varphi(x)$, rather than by Equation 2.2. In all networks we test, from Chapter 3 and

onwards, the output readout will be given by Equation 2.2.)

Beiran et al hypothesize that flexible control in timing emerges from a combination of low-dimensional dynamics and modulation by contextual inputs. To test this hypothesis, they first introduce a task called Cue-Set-Go (CSG), in which there are four time intervals during training, and which is designed as follows. For any time interval T (measured in timesteps) among the four time intervals, a tonic (i.e., constant) signal that is linearly related to T is provided as input (to the given network). Then, at a certain timestep, a transient input pulse (corresponding to 'set') is provided, instructing the given network to start producing a linearly ramping output signal such that the ramp crosses some predetermined threshold exactly T timesteps after the 'set' pulse. In this manner, the target output signal corresponding to the time interval T can be deduced from the tonic contextual input signal provided to the network (see Figure 2.1A).

Beiran et al test unconstrained full-rank RNNs as well as rank-2 RNNs with the same number of neurons (i.e., $x(t) \in \mathbb{R}^N$ in both cases) as the former networks. As demonstrated by Figure 2.1B, the low-rank counterparts demonstrate much better extrapolation ability for unseen time intervals compared to full-rank RNNs. Thus, tonic contextual inputs appear to be effective in facilitating generalization.

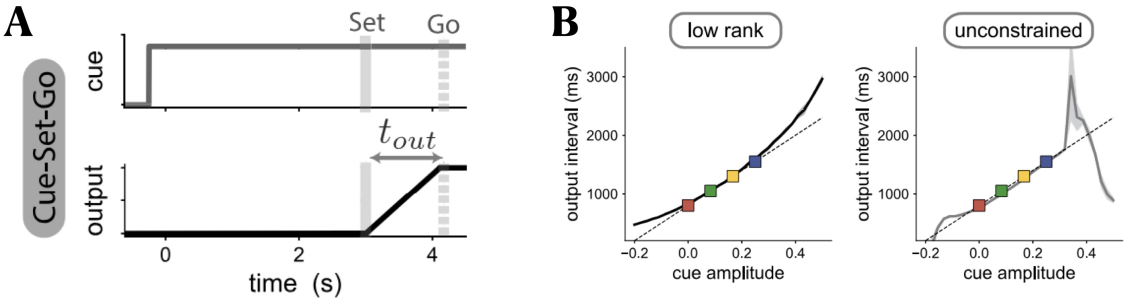


Figure 2.1 Schematic of Cue-Set-Go input-output pair and results, taken from Beiran et al (2023) [52]. **A** For any given time interval, a tonic input linearly related to the interval is provided as input, and once the 'set' (pulse) cue is provided, the network must linearly ramp and reach a predetermined threshold after precisely the duration of the time interval. **B** Rank-2 RNNs trained on the CSG tasks extrapolate fairly well to higher and lower (unseen) time intervals, whereas full-rank RNNs chiefly fail to extrapolate for larger time intervals.

Then, Beiran et al move on to a more complex task called Measure-Wait-Go (MWG). In this task, tonic contextual input signals are no longer provided. Rather, any given input signal consists solely of three transient pulses: a 'measure', a 'wait', and a 'go' cue. The number of timesteps T between the 'measure' and 'wait' cues demarcate the desired

time interval a network must reproduce. Then, when the 'go' pulse arrives (after a short delay from the 'wait' pulse), the given network must start linearly ramping such that it reaches a predetermined threshold exactly T timesteps after the 'go' pulse was delivered. A schematic of the task description is shown in Figure 2.2.[1]

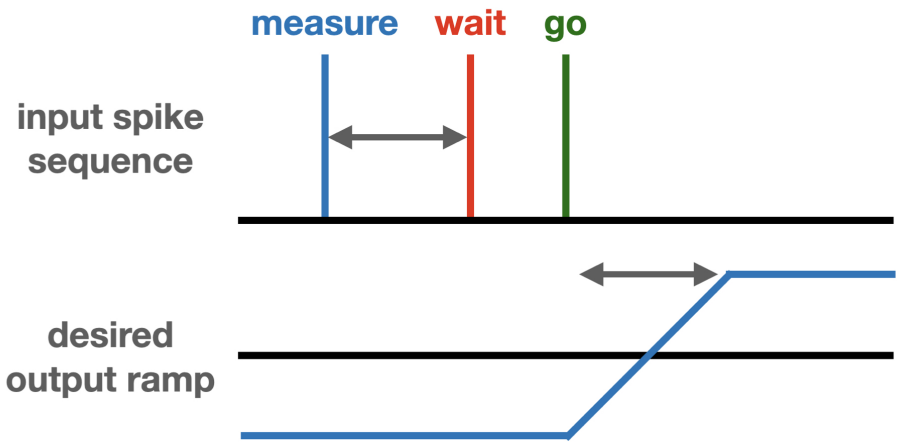


Figure 2.2 Schematic of Measure-Wait-Go input and target output signals, taken from Beiran et al (2023) [52]. Shown above is a visualization of a sample input signal, with 'measure', 'wait', and 'go' cues. The target output signal is a ramping signal in which linear ramping must start at the 'go' cue and continue for precisely the duration between the 'measure' and 'wait' cues. (Figure credits to Julia Costacurta.)

Finding that it takes low-rank RNNs of rank at least 3 to match the performance of full-rank RNNs on the MWG training set, Beiran et al evaluate the ability of full-rank RNNs and rank-3 RNNs (with the same number of neurons) to extrapolate to unseen intervals. As demonstrated by Figure 2.3, both networks fail to exhibit generalization. Based on this finding, they hypothesize that the addition of a tonic contextual input signal (such as during the CSG task) would salvage the ability of low-rank RNNs to generalize on the MWG task.

Consequently, they design a new task called Measure-Wait-Go with Context (MWG + Ctxt) which operates in the same manner as the original MWG task, except that each input signal is supplemented with an additional tonic input signal providing a "hint" as to the reduced range of intervals that contains the target time interval (see Figure 2.4A).

The results of training and testing both rank-3 RNNs and full-rank RNNs (with the same number of neurons as their low-rank counterparts) are shown in Figure 2.4. First,

[1] We will revisit the Measure-Wait-Go task in Chapter 4.

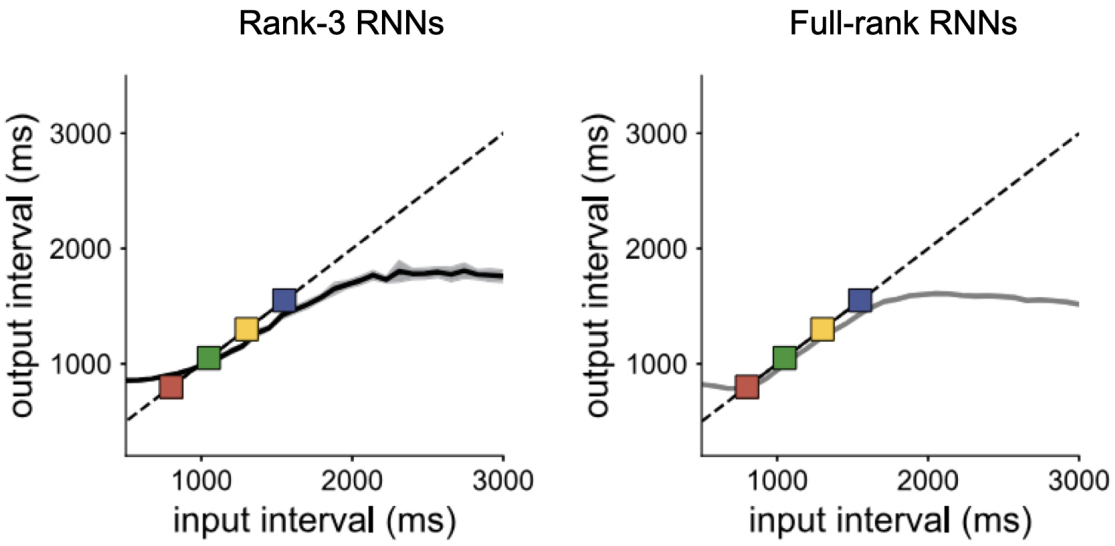


Figure 2.3 Extrapolation abilities of rank-3 RNNs versus (unconstrained) full-rank RNNs (with the same number of neurons) on the MWG task. While both networks are reasonably able to replicate intervals from within the range of trained intervals, they fail to extrapolate to unseen intervals above and below this range. Figure taken from Beiran et al (2023) [52].

Figure 2.4B illustrates the ability of rank-3 networks to extrapolate reasonably in line with the provided contextual signal, whereas full-rank networks fail to extrapolate regardless of context. Moreover, Figure 2.4C shows that, when novel (previously unseen) context distributions are used for testing, rank-3 RNNs demonstrate the ability to extrapolate to higher intervals, intervals in between those that were trained on, and lower intervals, reasonably well in line with the provided novel contextual signal. Thus, Beiran et al demonstrate that the contextual input signal is a form of input modulation that causes low-rank RNNs to flexibly adjust the output ramp they produce, enabling such networks to generalize to previously unseen intervals.

2.3 The Case for Neuromodulation

In the Measure-Wait-Go with Context task, every input signal consists of the three MWG input pulses along with the tonic contextual input signal. This task specification brings to light an important question: where does this contextual input signal come from? That is, when providing this tonic input signal, we effectively augment the original MWG input

signal. However, solving the original MWG task, in part, requires a network to discern the right context without it being provided a priori. Consequently, we seek to determine whether a suitable variant of low-rank RNNs can perform the original MWG task (i.e., without any tonic contextual input signal) and generalize to unseen intervals.

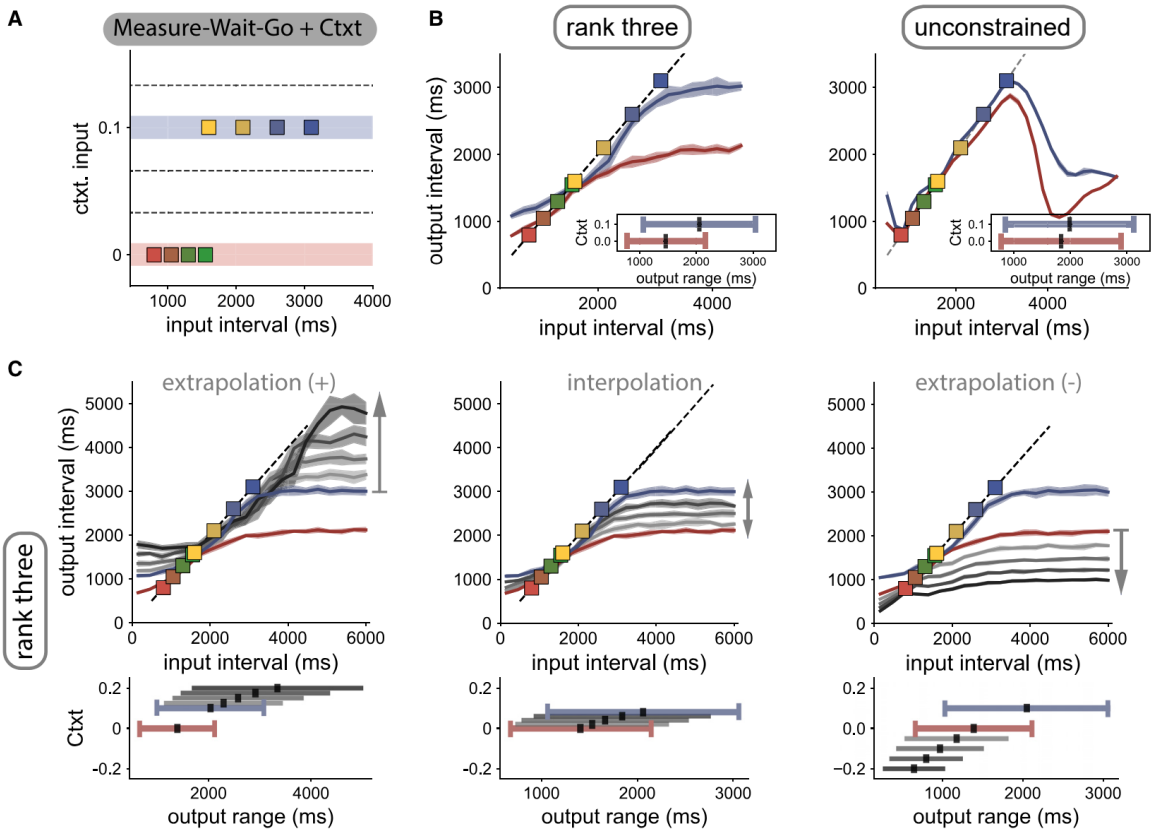


Figure 2.4 Description and results for the MWG + Ctxt task, taken from Beiran et al (2023) [52]. **A** Each target time interval during training is sampled from one of two context distributions; all intervals in the same context correspond to the same tonic input signal supplementing the MWG input pulses. **B** Rank-3 RNNs exhibit significantly better extrapolation ability than their full-rank counterparts. **C** After training is complete, new context distributions are used for testing. Rank-3 RNNs are capable of extrapolating to larger unseen intervals when provided with the corresponding larger tonic contextual input signal (left); similar results hold for interpolating to time intervals in contexts in between the two contexts that were trained on (middle), as well as for extrapolating to contexts below those that were trained on (right). The range of the novel context distributions is shown at the bottom of the figure.

This is where we revisit the idea of *neuromodulation*, first introduced in Section 1.3. While ordinary low-rank (and full-rank) RNNs fail to generalize on the MWG task, we hypothesize that suitably modulating the recurrent dynamics of a low-rank RNN (based on the MWG input signal, with no additional contextual input added) would facilitate

such a network to flexibly reproduce both known and previously unseen time intervals. In particular, one could imagine a model in which a group of "modulatory neurons" sends control signals to an output-generating low-rank RNN, injecting into the low-rank RNN's dynamics a "self-generated contextual clue" as to what target time interval must be reproduced. This model, called the neuromodulated low-rank RNN (NM-LR-RNN), will be introduced in Chapter 3; it will constitute our proposed solution to Question 1 and be the subject of this thesis.

Chapter 3

Neuromodulated Recurrent Neural Networks

In this chapter, we return to our guiding question: computationally, how can we study the role that neuromodulation plays in shaping flexible neural dynamics? Having introduced low-rank recurrent neural networks (LR-RNNs) in the previous chapter, we will introduce a model that incorporates the additional element of neuromodulation into this existing framework, motivated by the potential for neuromodulation to systematically add an element of flexibility in adjusting neural dynamics. This will enable us to study a "minimal model" that could shed insight into how neuromodulation adjusts the dynamics of (recurrently connected) neural circuits in the brain.

3.1 Model Design

We propose a neural network model called the *neuromodulated low-rank recurrent neural network* (NM-LR-RNN), consisting of two linked RNNs that correspond to neuromodulation and output generation, respectively. To introduce a notion of neuromodulatory drive in a recurrent neural network model, define a variable $z(t) \in \mathbb{R}^M$ to denote the state of a *neuromodulatory network* (a "vanilla" RNN). This network will evolve according to dynamics that depend on its internal state $z(t)$ and a time-varying input signal $u(t) \in \mathbb{R}^{N_{in}}$. We then enable the neuromodulatory sub-network to scale the low-rank factors of the recurrence matrix of the *output-generating network*, a low-rank RNN of rank R with internal state $x(t) \in \mathbb{R}^N$ which is subsequently used to produce an output

state $y(t) \in \mathbb{R}^{N_{out}}$. This effectively constitutes a form of "gating," in which the vector subspace (of $\mathbb{R}^N$) over which dynamics occur can be flexibly adjusted by the neuromodulatory signal. Mathematically, we may model the NM-LR-RNN as the following coupled ODE:

$$\tau_z \frac{dz(t)}{dt} = -z(t) + W_{zz}\varphi(z(t)) + c(x(t)) + B_{zu}u(t) \tag{3.1}$$

$$\tau_x \frac{dx(t)}{dt} = -x(t) + W_{xx}(z(t))\varphi(x(t)) + B_{xu}u(t) \tag{3.2}$$

$$y(t) = Cx(t) + r \tag{3.3}$$

Here, the neuromodulatory network is modeled as a "vanilla" RNN (with hidden size M) with its own time-constant τ_z, learnable recurrent weights $W_{zz} \in \mathbb{R}^{M\times M}$, and learnable inputs weights $B_{zu} \in \mathbb{R}^{M\times N_{in}}$ (where $u(t) \in \mathbb{R}^{N_{in}}$ is the input signal). φ denotes a suitable transfer function, which we usually set to be tanh. Additionally, $c(x(t))$ is defined as a *feedback coupling* term (from the low-rank RNN) given by $c(x(t)) := B_{zx}\varphi(x(t)) + b_z$, where $B_{zx} \in \mathbb{R}^{M\times N}, b_z \in \mathbb{R}^M$ are learnable parameters. For simplicity, unless otherwise stated, we will assume that $c(x(t)) = 0$, i.e., there is no feedback coupling from x to z.

At each point in time, the neuromodulatory state $z(t)$ influences the dynamics of a low-rank RNN – the output-generating network – by continuously altering the *de-facto singular values* ("de-facto" explained below) of the recurrence weights $W_{xx}(z(t)) \in \mathbb{R}^{N\times N}$. (The input weights $B_{xu} \in \mathbb{R}^{N\times N_{in}}$ are also learned.) More precisely, we set up the matrix $W_{xx} \in \mathbb{R}^{N\times N}$ as

$$W_{xx}(z(t)) = \frac{1}{N} U \operatorname{diag}[s(t)]\, V^\top \tag{3.4}$$

$$s(t) = \sigma(W_s z(t) + b_s) \tag{3.5}$$

where $W_s \in \mathbb{R}^{R\times M}$, $b_s \in \mathbb{R}^R$, $U \in \mathbb{R}^{N\times R}$, and $V \in \mathbb{R}^{N\times R}$ are learned parameters, and $\sigma : \mathbb{R} \to [0, 1]$ denotes the sigmoid activation function

$$\sigma(w) = \frac{1}{1 + e^{-w}}.$$

Here, we can view the time-varying quantity $s(t) \in \mathbb{R}^R$ as the output of the neuromodu-

latory sub-network and as the neuromodulatory drive supplied to the output-generating network; the entries of this vector are the de-facto singular values[1] of W_{xx}, and we collectively refer to $s(t)$ as the *neuromodulatory signal* supplied to the output-generating network. If we let $U_1, \ldots, U_R$ and $V_1, \ldots, V_R$ denote the columns of U and V, respectively, then we may explicitly express W_{xx} as $\frac{1}{N}\sum_{r=1}^{R} s_R U_R V_R^T$; that is, the neuromodulation can "determine" how much each rank-1 component of W_{xx} contributes to the overall recurrent dynamics. Moreover, in contrast to having a static recurrence matrix, the given W_{xx} can be viewed as dynamically changing over the timesteps of computation (because of the time-varying signal $s(t)$). Finally, the hidden state of the output-generating RNN is used to produce an output $y(t) \in \mathbb{R}^{N_{out}}$. Here, the readout parameters $C \in \mathbb{R}^{N_{out} \times N}$ and $r \in \mathbb{R}^{N_{out}}$ are learnable.

From a methodological standpoint, we typically create the neuromodulatory sub-network such that it is a small "vanilla" RNN (i.e., M is usually much less than N). We also initialize the time constants τ_x, τ_z with $\tau_z >> \tau_x$, to model neuromodulation as occurring across longer timescales than those along which neural circuits evolve [53, 54]. Moreover, when training an NM-LR-RNN in practice, we discretize time (i.e., "$dt = 1$", so that $\frac{dz}{dt} = z_t - z_{t-1}$); at each timestep of computation, we first update z and then use the updated value of z to update the value of x. Up to rearrangement of terms, we may express the discrete approximation to our model as follows:

$$z_t = \left(1 - \frac{1}{\tau_z}\right) z_{t-1} + \frac{1}{\tau_z}\left(W_{zz}\varphi(z_{t-1}) + c(x_{t-1}) + B_{zu}u_t\right) \tag{3.6}$$

$$x_t = \left(1 - \frac{1}{\tau_x}\right) x_{t-1} + \frac{1}{\tau_x}\left(W_{xx}(z_t)\varphi(x_{t-1}) + B_{xu}u_t\right) \tag{3.7}$$

$$y_t = Cx_t + r \tag{3.8}$$

(Again, the coupling term $c(x_t)$ is set to 0 unless explicitly stated otherwise.) All NM-LR-RNN models created for the experiments presented in Chapters 4 and 5 were trained via backpropagation in the Python JAX library.

[1]The entries of $s(t)$ are not *really* singular vectors in our optimization, since we do not constrain U and V to be orthogonal, unless explicitly mentioned otherwise. Consequently, we refer to $s(t)$ as the vector of de-facto singular values.

3.2 How Does the Model Map onto the Biology?

Given that the NM-LR-RNN is intended to constitute (at least, partially) a suitable abstraction of real biological computations, the goal of this section is to make the model's connection to the brain more explicit. Certainly, the neuromodulatory and output-generating networks constitute biologically plausible architectures, as they consist of populations of recurrently connected neurons. However, it remains to unpack how the neuromodulatory sub-network is able to modulate the *de-facto singular values* (i.e., $s(t)$) of the recurrence matrix W_{xx} of the output-generating sub-network (as in Equation 3.4) in a biologically plausible way.

One such way is presented in Figure 3.1. Rather than viewing $W_{xx}(z(t))$ as a single weight matrix in its own right, we may view it as a composition of multiple neural computations. Starting with the vector $\varphi(x(t))$ (shown in Equation 3.2), we may apply the weight matrix $V^T \in \mathbb{R}^{R\times N}$ to reach a layer of R neuromodulatory interneurons (e.g., dopaminergic neurons in the striatum). In particular, the rth interneuron in this layer has an excitability, or gain modulation, factor of $s_r(t)$; that is, the neuromodulatory vector $s(t)$ scales the activities of the interneurons elementwise. Then, the synaptic weight matrix $U \in \mathbb{R}^{N\times R}$ goes from this layer of interneurons back to the original pool of N neurons (in the low-rank RNN). The net effect of these computations is precisely to compute the quantity $[W_{xx}(z(t))]\,\varphi(x)$.

The computations presented in Figure 3.1 may indeed be considered as biologically plausible, since synaptic weight matrices (i.e., U and V^T) carry neural activity across layers of neurons, and neuromodulators are able to tune the excitations of neurons (e.g., by elementwise gain modulation of neural activities).

3.3 Neuromodulation Adjusts the Spatial and Temporal Modes of Computation

Now that we have defined an RNN model that incorporates neuromodulation, we seek to understand in what ways the latter affects the evolution of neural dynamics. Prior work has found that including gating in recurrent neural network models (akin to what we have presently modeled as neuromodulation) allows for flexible control over two key features

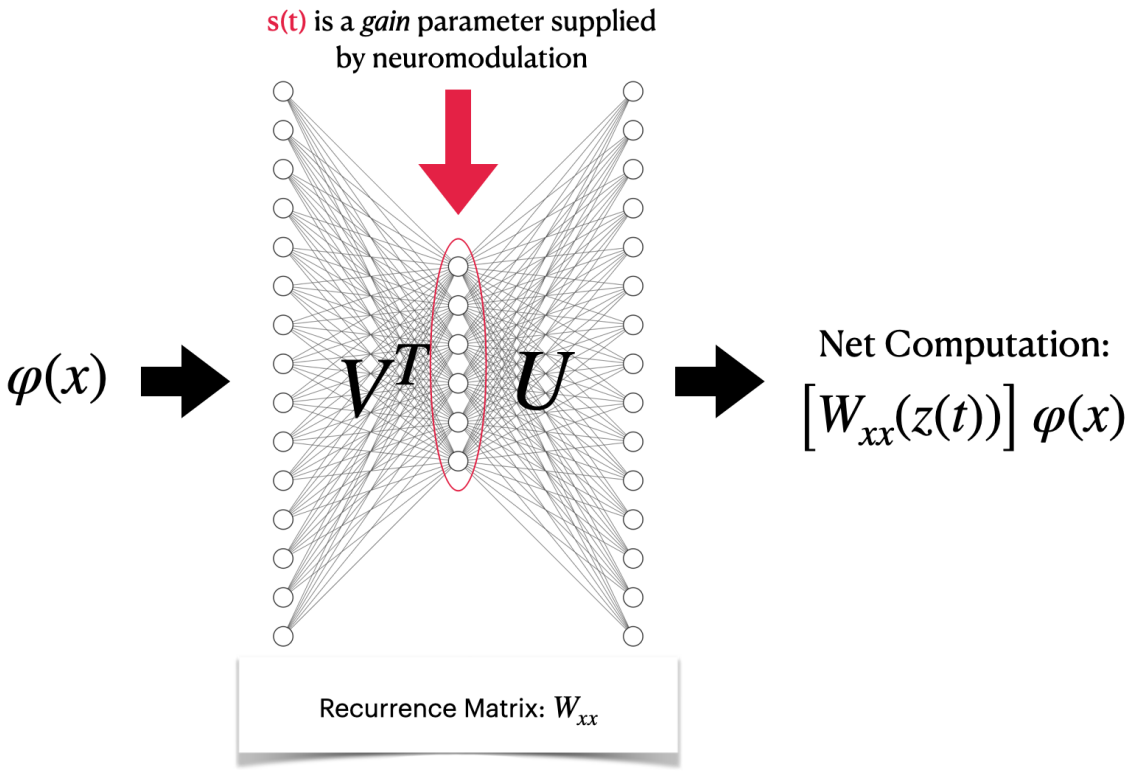


Figure 3.1 It is possible to "unpack" the matrix W_{xx}, viewing it as a composition of applying the matrices V^T and U successively, with a layer of gain-modulated interneurons in between.

of neural dynamics: timescales and dimensionality ([55]). In the following subsections, we explore how our model is able to use neuromodulation to leverage control over these features.

3.3.1 Gating the Spatial Structure of Neural Dynamics

For each $r \in \{1, 2, \ldots, R\}$, we say that the matrix $U_r V_r^T \in \mathbb{R}^{N \times N}$ is a *spatial mode* of the low-rank RNN's dynamics because it is one of the R rank-1 matrices contributing to the recurrence matrix W_{xx}, and because (geometrically) it represents a rank-1 linear transformation that maps a vector v onto the span of U_r scaled by its dot product similarity to V_r. By virtue of the fact that at each timestep t, $W_{xx}(z(t)) = \sum_{r=1}^{R} s_r(t) U_r V_r^T$ and that $s(t) \in [0, 1]$ (as s is the output of a sigmoid nonlinearity), intuitively we see that the recurrence matrix of the output-generating RNN is a weighted linear combination these R rank-1 components, meaning that the low-rank RNN has the flexibility to prioritize which components, or spatial modes, should contribute more to the dynamics. In turn, this affects the resulting spatial geometry along which the dynamics evolve. In some

sense, we may view this phenomenon as a form of *gating*, whereby some of the dynamical modes may be switched off (i.e., when the corresponding $s_r(t) = 0$) and others may be switched on (i.e., when the corresponding $s_r(t) > 0$). In Section 3.4, we will formally connect the NM-LR-RNN to a well-known gated recurrent model: the Long Short-Term Memory (LSTM). This will demonstrate that the NM-LR-RNN indeed belongs to the class of gated recurrent models, and as such, enjoys certain advantages over standard RNN models.

3.3.2 Controlling Timescales of Dynamics

In addition to affecting the spatial geometries over which dynamics occur, neuromodulation is also capable of flexibly adjusting the *timescales* over which the output-generating RNN's dynamics settle (to fixed points). The intuition for this idea is best revealed in a *linear relaxation* of the NM-LR-RNN in which the recurrence matrix W_{xx} is constrained to be symmetric, giving the following dynamics equation for the output-generating network:

$$\tau_x \frac{dx(t)}{dt} = -x(t) + USU^T x(t) + B_{xu}u(t) \tag{3.9}$$

In this setting, we can imagine that the low-rank RNN is operating in the linear regime of the tanh nonlinearity and that $U = V$ are constrained to be orthogonal.[1] Moreover, we have set $S := \text{diag}(s(t))$, where $s(t)$ is defined as before (in Equation 3.5). Additionally, $z(t)$ follows the same dynamics as before (as presented in Equation 3.1), and the readout equation for $y(t)$ remains the same. In this simplified model, we can derive a simple characterization of how neuromodulation exerts control over the time-evolution of $x(t)$. Crucially, this simplification also captures the essence of our original NM-LR-RNN model – namely, it is a tool for explaining how a nonlinear "controller" network (i.e., the neuromodulatory subnetwork) can alter the dynamics of an existing dynamical system (the output-generating RNN).

To proceed with our mathematical analysis in this linearized regime, we define the variable $h(t) \in \mathbb{R}^R$, given by $h(t) := U^T x(t)$, to denote the low-rank variable corresponding to $x(t)$. The variable h largely captures the underlying R-dimensional dynamics of

[1] In Equation 3.9, we have removed the $\frac{1}{N}$ prefactor because in the original low-rank RNN, where U and V need not be orthogonal, we may simply scale $U = V$ by $\frac{1}{\sqrt{N}}$ and correspondingly extract a prefactor of $\frac{1}{N}$.

the low-rank RNN; indeed, in the absence of inputs, $x(t)$ will always belong to the column space of the orthogonal matrix U, so the low-rank variable $h = U^T x \in \mathbb{R}^R$ captures effectively captures full information about x, i.e., x can be recovered as Uh. Moreover, we obtain the following equation for the dynamics of h:

$$\tau_x \frac{dh}{dt} = -h + Sh + U^T B_{xu} u = -(I - S)h + U^T B_{xu} u.$$

In particular, in the absence of inputs ($u = 0$), we simply have

$$\tau_x \frac{dh}{dt} = -(I - S)h.$$

This equation really consists of R decoupled first-order ODEs, which we can then solve to obtain

$$h_i(t) = h_i(0) \cdot \exp\left(-\frac{1}{\tau_x}\left(t - \int_0^t s_i(t')dt'\right)\right) \tag{3.10}$$

for each $i = 1, \ldots, R$. This directly illustrates how the neuromodulatory state s is able to adjust the timescale along which each low-rank mode h_i evolves.

We can then describe the dynamics for the "residual" mode $h^\perp := x - UU^T x = (I - UU^T)x$:

$$\tau_x \frac{dh^\perp}{dt} = (I - UU^T)(-x + USU^T x + B_{xu} u)$$

$$= -(I - UU^T)x + (I - UU^T)USU^T x + (I - UU^T)B_{xu} u$$

$$= -h^\perp + (I - UU^T)B_{xu} u.$$

In particular, if $u = 0$, then we simply have

$$\tau_x \frac{dh^\perp}{dt} = -h^\perp,$$

i.e., the residual mode simply follows an exponential decay. That is, $h^\perp(t) = h^\perp(0) \cdot \exp(-t/\tau_x)$. Consequently, we may write

$$x(t) = UU^T x(t) + h^\perp(t) = Uh(t) + h^\perp(0)e^{-t/\tau_x},$$

where the components of h are given by Equation 3.10.

Since $h^{\perp}$ tends to 0 in the absence of inputs, to understand how neuromodulation is affecting $x(t)$, it suffices to understand how neuromodulation is affecting the low-rank state h. Equation 3.10 sheds light on this issue: for any $i \in \{1, 2, \ldots, R\}$ fixed, if s_i gets driven down to 0, then h_i will more rapidly exponentially decay to 0, and if s_i tends to 1, then the decay rate of h_i will slow down. In particular, in the limiting case of $s_i = 1$, h_i will settle to some stationary value.

In summary, we find that for the linearized NM-LR-RNN with symmetric recurrence matrix W_{xx}, the neuromodulatory signal $s(t)$ has the ability to control the rate of convergence to fixed points. Understanding how neuromodulation affects the timescales of neural dynamics in the general (nonlinear) NM-LR-RNN is certainly more opaque, but we have at least built the intuition that control over timescales is one of the possible effects of neuromodulation.

3.4 A Curious Connection to LSTMs

In Section 3.3, we explored the concept of neuromodulation as a form of gating. In the present section, we explore a curious connection between the NM-LR-RNN and a well-known gated recurrent model that has enjoyed much success in the machine learning literature: the Long Short-Term Memory (LSTM), first introduced by Hochreiter and Schmidhuber [56]. The LSTM was invented as a gated version of the RNN, so that the gates could selectively filter the information being processed by the recurrent components of the network across each timestep; consequently, the LSTM has enjoyed greater success (than traditional RNNs) in performing tasks that involve keeping track of long-distance dependencies in the input signal [57]. Thus, highlighting the connection between the NM-RNN and LSTM will suggest the NM-RNN's ability to successfully model long-timescale dependencies, unlike regular RNNs. We will empirically explore this phenomenon in Section 4.2.

In contrast to ordinary RNNs, LSTMs make use of two states that recurrently update across each timestep t: a *hidden state* $h_t \in \mathbb{R}^{N_{LSTM}}$ and a *cell state* $c_t \in \mathbb{R}^{N_{LSTM}}$. Moreover, let u_t denote the input signal at timestep t. The update equations for an

LSTM's internal states are

$$f_t = \sigma(W_f[h_{t-1}, u_t] + b_f) \quad \text{(forget gate)} \tag{3.11}$$

$$i_t = \sigma(W_i[h_{t-1}, u_t] + b_i) \quad \text{(input gate)} \tag{3.12}$$

$$\tilde{c}_t = \tanh(W_c[h_{t-1}, u_t] + b_c) \quad \text{(activation gate)} \tag{3.13}$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad \text{(cell state)} \tag{3.14}$$

$$o_t = \sigma(W_o[h_{t-1}, u_t] + b_o) \quad \text{(output gate)} \tag{3.15}$$

$$h_t = o_t \odot \tanh(c_t) \quad \text{(hidden state)} \tag{3.16}$$

Here, $\odot$ denotes elementwise multiplication (for vectors). Interestingly, in the case of a linearized symmetric NM-LR-RNN (as described in Section 3.3.2) with $\tau_x = 1$, the discretized equation for the dynamics of the output-generating RNN reads

$$x_t = U\mathrm{diag}(s_t)U^T x_{t-1} + B_{xu}u_t,$$

so passing to the low-rank mode $h := U^T x$ yields the equation

$$h_t = s_t \odot h_{t-1} + U^T B_{xu} u_t \tag{3.17}$$

where $s_t = \sigma(W_s z_t + b_s)$. Cosmetically, Equation 3.17 resembles the cell state update equation (Equation 3.14) for the LSTM, and the neuromodulatory signal s_t (a function of z_t) appears to play the role of the forget gate. (Moreover, the input term $U^T B_{xu} u_t$ is a transformation of the input signal, akin to the input gating term $i_t \odot \tilde{c}_t$ of the LSTM.) This seems to suggest that x_t resembles the cell state (c_t) of an LSTM, while z_t may resemble the hidden state (h_t) of the LSTM. The schematics of the NM-LR-RNN and the LSTM, with their similar components depicted by the same shaded colors, are illustrated in Figure 3.2.

In fact, we may mathematically formalize this connection: when we consider suitable linearized analogs of the NM-LR-RNN and the LSTM, we can in fact view the NM-LR-RNN as effectively carrying out the computations of an LSTM. This result will illustrate a curious connection between the two gated models, suggesting that the NM-LR-RNN may enjoy some of the advantages of LSTMs over ordinary RNNs, owing to their similar gating mechanisms. We begin by defining suitable linearizations of the models in question.

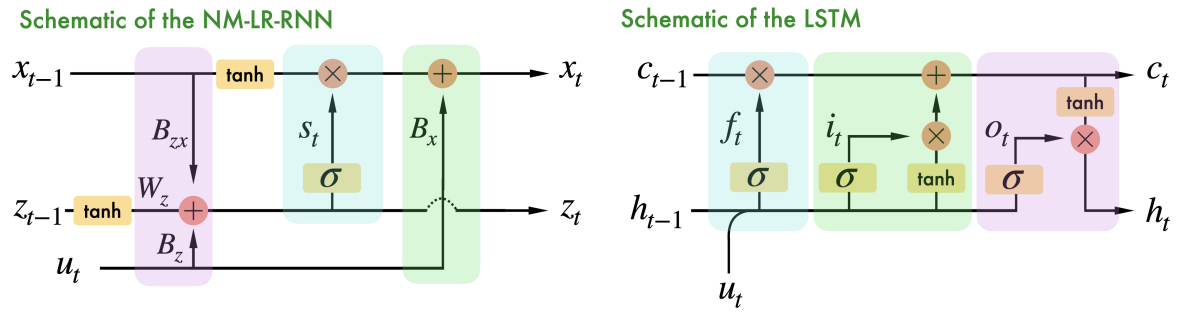


Figure 3.2 Visual comparison of an NM-LR-RNN and an LSTM. Corresponding parts of the networks are highlighted with shaded rectangles. Blue: a forget gate computation. Green: an input gate to recurrent dynamics. Purple: recurrent feedback onto the modulatory state variable.

Definition 2. *We define a* ***fully linearized coupled NM-LR-RNN*** *to be a NM-LR-RNN given by the following (discretized) dynamics equations:*

$$x_t = \frac{1}{N} U S_t V^T x_{t-1} + B_{xu} u_t \tag{3.18}$$

$$S_t = \text{diag}(s_t), \ \ \textit{where } s_t = \sigma(W_s z_t + b_s) \tag{3.19}$$

$$z_t = \left((1 - \frac{1}{\tau_z}) I_{R \times R} + \frac{1}{\tau_z} W_{zz} \right) z_{t-1} + \frac{1}{\tau_z} \left(B_{zx} x_{t-1} + B_{zu} u_t \right) \tag{3.20}$$

$$y_t = C x_t + r \tag{3.21}$$

The above model is an NM-LR-RNN with $\tau_x = 1$ and where all nonlinear transfer functions have been made to be the identity function. Notably, we have also added a *feedback coupling* term (namely, $B_{zx} x_t$, with no bias vector) from the output-generating RNN to the neuromodulatory RNN; this will serve to enhance the connection between this model and the LSTM, although we may simply impose the constraint $B_{zx} = 0 \in \mathbb{R}^{M \times N}$ if we wish to avoid feedback coupling. As before, $x_t \in \mathbb{R}^N$ denotes the hidden state of the output-generating network, $z_t \in \mathbb{R}^M$ denotes the hidden state of the neuromodulatory sub-network, and $y_t \in \mathbb{R}^{N_{out}}$ denotes the output generated by the network. Here, the parameters $U, V \in \mathbb{R}^{N \times R}, B_{xu} \in \mathbb{R}^{N \times N_{in}}, W_s \in \mathbb{R}^{R \times M}, b_s \in \mathbb{R}^R, W_{zz} \in \mathbb{R}^{M \times M}, B_{zx} \in \mathbb{R}^{M \times N}, B_{zu} \in \mathbb{R}^{M \times N_{in}}, C \in \mathbb{R}^{N_{out} \times N}$, and $r \in \mathbb{R}^{N_{out}}$ are learnable, and σ denotes the sigmoid activaton function. Next, we define a suitable linearization of the LSTM.

Definition 3. *Define a* ***semilinear LSTM*** *to be an LSTM whose gated dynamics are governed by the following equations:*

$$f_t = \sigma(W_f[h_{t-1}, u'_t] + b_f) \tag{3.22}$$

$$i_t = \sigma(W_i[h_{t-1}, u'_t] + b_i) \tag{3.23}$$

$$\tilde{c}_t = W_c[h_{t-1}, u'_t] + b_c \tag{3.24}$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \tag{3.25}$$

$$o_t = W_o[h_{t-1}, u'_t] + b_o \tag{3.26}$$

$$h_t = o_t \odot c_t \tag{3.27}$$

Here, σ denotes the sigmoid nonlinearity, $u'_t \in \mathbb{R}^{N_{in}}$ denotes the input, $h_t \in \mathbb{R}^{N_{LSTM}}$ denotes the hidden state, and $c_t \in \mathbb{R}^{N_{LSTM}}$ denotes the cell state. The notation $[\cdot, \cdot]$ signifies concatenation of vectors.

In other words, a semilinear LSTM is one whose "tanh" nonlinearities and the sigmoid nonlinearity of o_t are removed. Each of the parameters W_k, b_k, for $k \in \{f, i, c, o\}$, are learnable.

Now, we formalize the connection between these two classes of models. It will be revealing to consider the case where $U = V$.

Theorem 4. *Consider a rank-R linearized coupled NM-LR-RNN, where the hidden size of the (linearized) output-generating network is N, and the hidden size of the neuromodulatory sub-network is M. Moreover, assume that $U = V$ and that the columns of $\frac{1}{\sqrt{N}}U$ are pairwise orthogonal, in the sense that $\frac{1}{N}U^T U = I_{R\times R}$. Finally, across all input sequences u_t on which the NM-LR-RNN is tested, assume that the components of the states x_t and z_t are uniformly bounded over all timesteps. Then, this model's underlying R-dimensional dynamics (given by the low-rank variable $v_t := \frac{1}{N}V^T x_t$), as well as its neuromodulatory states z_t and outputs $y_t \in \mathbb{R}^{N_{out}}$, can be reproduced within a semilinear LSTM with a linear readout, whose inputs are $u'_t := \begin{bmatrix} u_t \\ u_{t-1} \end{bmatrix} \in \mathbb{R}^{2N_{in}}$. Here, $u_t \in \mathbb{R}^{N_{in}}$ denotes the input to the NM-LR-RNN at time t (with $u_{-1} := 0 \in \mathbb{R}^{N_{in}}$ by convention). Moreover, such a semilinear LSTM can be made to have hidden size $R + M + N_{in}$ and utilize $O\left(\max\{R^2, M^2, N_{in}R, N_{in}M, N_{out}R, N_{out}N_{in}\}\right)$ learnable parameters.*

Remark 5. *Upon inspection, we observe that the fully linearized (coupled) NM-LR-RNN*

(with linear readout) utilizes $O\left(\max\left\{NR, N_{in}N, MR, M^2, MN, N_{in}M, N_{out}N\right\}\right)$ *parameters. In particular, we find that the equivalent semilinear LSTM typically uses much fewer parameters, owing to the fact that we have captured the output-generating RNN's dynamics as an* R*-dimensional system, rather than an* N*-dimensional system.*

More specifically, assuming N_{in}*,* N_{out} *are sufficiently small relative to* R *and* M*, and that* $R = o(N)$ *and* $M = o(N)$*, then the semilinear LSTM (with linear readout) uses* $O\left(\left(\max\{R, M\}\right)^2\right)$ *parameters, which is asymptotically less than the* $O(N \max\{R, M\})$ *parameters used by the NM-LR-RNN.*

Proof of Theorem 3.4. Assuming x_t has uniformly bounded components, there exists some constant $L \in \mathbb{R}_+$ such that $(x_t)_i < L$ for each $i \in \{1, \ldots, N\}$ and all t. Consequently, this means $\frac{1}{\sqrt{N}}||x_t|| < L$ for all t. Now, consider the update equation for x_t in the linearized NM-LR-RNN:

$$x_t = \frac{1}{N} V S_t V^T x_{t-1} + B_{xu} u_t.$$

For V_i the ith column of V, the orthogonality condition implies that $\frac{||V_i||^2}{N} = 1$, or $\frac{1}{\sqrt{N}}||V_i|| = 1$. Defining $v_t := \frac{1}{N} V^T x_t \in \mathbb{R}^R$ to be the low-rank representation corresponding to x_t, we have that $|(v_t)_i| = \frac{1}{N}\left|V_i^T x_t\right| \leq \frac{1}{N}||V_i|| \cdot ||x_t|| = \frac{1}{\sqrt{N}} \cdot ||x_t|| < L$. That is, the low-rank mode corresponding to x_t also has uniformly bounded components over time, i.e., the underlying R-dimensional dynamics of the output-generating network do not diverge.

Now, to show that the dynamics of a linearized NM-LR-RNN satisfying the conditions given in the theorem statement can be replicated by a semilinear LSTM, we must effectively show that each of the update equations for the linearized NM-LR-RNN given in Definition 2 can be suitably replicated through the LSTM architecture. In essence, we must suitably map the parameters of the given NM-LR-RNN onto those of a semilinear LSTM.

First, we analyze the update equation for x_t in the output-generating network. Its corresponding R-dimensional (low-rank) update is

$$v_t = \frac{1}{N} V^T V S_t v_{t-1} + \frac{1}{N} V^T B_{xu} u_t$$

$$\implies v_t = S_t v_{t-1} + \frac{1}{N} V^T B_{xu} u_t.$$

Accordingly, define the cell state c_t of the corresponding semilinear LSTM to be $c_t = \begin{bmatrix} v_{t-1} \\ \mathbf{1}_M \\ \mathbf{1}_{N_{in}} \end{bmatrix} \in \mathbb{R}^{R+M+N_{in}}$, where $\mathbf{1}_\alpha$ denotes the vector of 1's in $\mathbb{R}^\alpha$ for $\alpha \in \{M, N_{in}\}$ (and, by convention, we set $v_{-1} := 0 \in \mathbb{R}^R$). In particular, note that x_t can be recovered from the cell state via the relation $x_t = \frac{1}{N} V S_t V^T x_{t-1} + B_{xu} u_t = V S_t v_{t-1} + B_{xu} u_t = V S_t \text{Proj}_R c_t + B_{xu} u_t$, where Proj_R denotes the projection matrix that sends c_t onto its first R components, namely, v_{t-1}.

We may also set all the parameters in W_i and b_i (part of the input gate i_t) to be 0, so that $i_t = \frac{1}{2}\mathbf{1}_{R+M+N_{in}}$ has a constant output. Then, define $\tilde{c}_t$ so that it equals $\begin{bmatrix} \frac{2}{N} V^T B_{xu} u_{t-1} \\ \mathbf{0}_M \\ \mathbf{0}_{N_{in}} \end{bmatrix} \in \mathbb{R}^{R+M+N_{in}}$. Indeed,

$$\tilde{c}_t = W_c[h_{t-1}, u_t'] + b_c = W_c[h_{t-1}, u_t, u_{t-1}] + b_c$$

can be made to have its first R entries form the vector $\frac{2}{N} V^T B_{xu} u_{t-1}$ if we set $b_c = 0$ and zero out all columns of W_c that do not correspond to u_{t-1}, and further by zeroing out the last $M + N_{in}$ rows of W_c. In block matrix form, we have defined

$$W_c = \left(\begin{array}{c|c|c} 0 & 0 & \frac{2}{N} V^T B_{xu} u_{t-1} \\ \hline 0 & 0 & 0 \\ \hline 0 & 0 & 0 \end{array}\right) \in \mathbb{R}^{(R+M+N_{in}) \times (N+M+3N_{in})}.$$

Before explicitly computing the other gates f_t, we first analyze how the semilinear LSTM might reproduce the update for the neuromodulatory state $z_t \in \mathbb{R}^M$:

$$z_t = \left((1 - \frac{1}{\tau_z}) I_{R \times R} + \frac{1}{\tau_z} W_{zz}\right) z_{t-1} + \frac{1}{\tau_z} \left(B_{zx} x_{t-1} + B_{zu} u_t\right)$$

$$= \left((1 - \frac{1}{\tau_z}) I_{R \times R} + \frac{1}{\tau_z} W_{zz}\right) z_{t-1} + \frac{1}{N \tau_z} B_{zx} V V^T x_{t-1} + \frac{1}{\tau_z} B_{zx} \left(I_{N \times N} - \frac{1}{N} V V^T\right) x_{t-1} + \frac{1}{\tau_z} B_{zu} u_t$$

$$= \left((1 - \frac{1}{\tau_z}) I_{R \times R} + \frac{1}{\tau_z} W_{zz}\right) z_{t-1} + \frac{1}{\tau_z} B_{zx} V v_{t-1} + \frac{1}{\tau_z} B_{zx} \left(I_{N \times N} - \frac{1}{N} V V^T\right) x_{t-1} + \frac{1}{\tau_z} B_{zu} u_t.$$

Note further that

$$\left(I_{N\times N} - \frac{1}{N}VV^T\right) x_{t-1} = x_{t-1} - \frac{1}{N}VV^T x_{t-1}$$

$$= \frac{1}{N}VS_{t-1}V^T x_{t-2} + B_{xu}u_{t-1} - \frac{1}{N}VV^T\left(\frac{1}{N}VS_{t-1}V^T x_{t-2} + B_{xu}u_{t-1}\right)$$

$$= \frac{1}{N}VS_{t-1}V^T x_{t-2} + B_{xu}u_{t-1} - \frac{1}{N}VS_{t-1}V^T x_{t-2} - \frac{1}{N}VV^T B_{xu}u_{t-1}$$

$$= \left(I_{N\times N} - \frac{1}{N}VV^T\right) B_{xu}u_{t-1}.$$

Thus, upon substituting this expression into our update equation for z_t above, we have that

$$z_t = \frac{1}{\tau_z}B_{zx}Vv_{t-1} + \left((1-\frac{1}{\tau_z})I_{R\times R} + \frac{1}{\tau_z}W_{zz}\right) z_{t-1} + \frac{1}{\tau_z}B_{zu}u_t + \frac{1}{\tau_z}B_{zx}\left(I_{N\times N} - \frac{1}{N}VV^T\right) B_{xu}u_{t-1} \quad (3.28)$$

Accordingly, define the output gate o_t of the LSTM to be

$$o_t := \begin{bmatrix} \frac{1}{2}\mathbf{1}_R \\ \left((1-\frac{1}{\tau_z})I_{R\times R} + \frac{1}{\tau_z}W_{zz}\right) z_{t-1} + \frac{1}{\tau_z}B_{zu}u_t + \frac{1}{\tau_z}B_{zx}\left(I_{N\times N} - \frac{1}{N}VV^T\right) B_{xu}u_{t-1} \\ u_{t-1} \end{bmatrix} \in \mathbb{R}^{R+M+N_{in}}.$$

This is achieved by setting $b_o = 0$ and zeroing out the first R rows of W_o. The next M rows can be set to produce the middle entry in o_t shown above, and the last N_{in} rows can similarly be set so as to produce the vector u_{t-1}. That is, in block matrix form, we claim that we may take W_o to be

$$W_o = \left(\begin{array}{c|c|c} 0 & 0 & 0 \\ \hline \left((1-\frac{1}{\tau_z})I_{R\times R} + \frac{1}{\tau_z}W_{zz}\right) W_{zh} & \frac{1}{\tau_z}B_{zu} & \frac{1}{\tau_z}B_{zx}\left(I_{N\times N} - \frac{1}{N}VV^T\right) B_{xu} \\ \hline 0 & 0 & I_{N_{in}\times N_{in}} \end{array}\right)$$

$$\in \mathbb{R}^{(R+M+N_{in})\times(R+M+3N_{in})}$$

where $W_{zh} \in \mathbb{R}^{M\times(R+M+N_{in})}$ is a suitable matrix (defined below) mapping the hidden state h_t of the semilinear LSTM to z_t, so that $o_t = W_o\,[h_{t-1}, u_t, u_{t-1}]$. Then, computing

$h_t = o_t \odot c_t \in \mathbb{R}^{R+M+N_{in}}$ gives

$$h_t = \begin{bmatrix} \frac{1}{2}v_{t-1} \\ \left((1-\frac{1}{\tau_z})I_{R\times R} + \frac{1}{\tau_z}W_{zz}\right) z_{t-1} + \frac{1}{\tau_z}B_{zu}u_t + \frac{1}{\tau_z}B_{zx}\left(I_{N\times N} - \frac{1}{N}VV^T\right)B_{xu}u_{t-1} \\ u_{t-1} \end{bmatrix}$$

$$\in \mathbb{R}^{R+M+N_{in}}.$$

Now, observe that $z_t = W_{zh}h_t$, where

$$W_{zh} = \left(\frac{2}{\tau_z}B_{zx}V \;\middle|\; I_{M\times M} \;\middle|\; 0\right) \in \mathbb{R}^{M\times(R+M+N_{in})}.$$

In particular, the equation $z_t = W_{zh}h_t$ precisely corresponds to the update equation for z_t in the linearized NM-LR-RNN given by Equation 3.28. Thus, the hidden state update for the corresponding semilinear LSTM implicitly involves performing the update for z_t in the linearized NM-LR-RNN. Moreover, we see that the output gate o_t may indeed be constructed as a linear function of h_{t-1}, u_t, and u_{t-1}, and so the hidden state update equation $h_t = o_t \odot c_t$ detailed above is valid.

Finally, we set the forget gate so that it evaluates to $f_t = \begin{bmatrix} s_{t-1} \\ \mathbf{1}_{M+N_{in}} \end{bmatrix}$. Recalling that $f_t = \sigma(W_f[h_{t-1}, u_t'] + b_f) \in \mathbb{R}^{R+M+N_{in}}$, we can achieve this gating by taking the first R entries of f_t to be $s_{t-1} = \sigma(W_s z_{t-1} + b_s) = \sigma(W_s W_{zh}h_{t-1} + b_s)$. We can also ensure that the last $M + N_{in}$ entries of f_t form the vector $\mathbf{1}_{M+N_{in}}$ by zeroing out the last $M + N_{in}$ rows of W_f and making the last $M + N_{in}$ entries of the bias vector b_f to be sufficiently large (so that applying the sigmoid function to these entries effectively yields 1). In other words, in block matrix form, we have

$$W_f = \left(\begin{array}{c|c|c} W_s W_{zh} & 0 & 0 \\ \hline 0 & 0 & 0 \\ \hline 0 & 0 & 0 \end{array}\right) \in \mathbb{R}^{(R+M+N_{in})\times(R+M+3N_{in})}$$

and $b_f = \begin{bmatrix} b_s \\ Q\mathbf{1}_{M+N_{in}} \end{bmatrix}$ where $Q \in \mathbb{R}+$ may be chosen such that

$$Q >> M \max_{1\leq i\leq R, 1\leq j\leq M} \left|(W_s)_{ij}\right| \sup_{1\leq m\leq M, t, u_t} |(z_t)_m| \geq ||W_s z_t||_\infty = \max_{1\leq k\leq R} |(W_s z_t)_k| \,.$$

(The above supremum is taken over all components of z_t, all timesteps t – finitely or infinitely many – and all input sequences $(u_t)_{t\geq 1}$ being considered. We then invoke uniform boundedness of the components of z_t to grant the existence of such a Q.)

Putting together our gate computations, the cell state update equation for the semilinear LSTM (Equation 3.25) follows from the low-rank update equation of the linearized NM-LR-RNN's output-generating network:

$$v_t = S_t v_{t-1} + \frac{1}{N} V^T B_{xu} u_t$$

$$\iff \begin{bmatrix} v_{t-1} \\ \mathbf{1}_{M+N_{in}} \end{bmatrix} = \begin{bmatrix} s_{t-1} \\ \mathbf{1}_{M+N_{in}} \end{bmatrix} \odot \begin{bmatrix} v_{t-2} \\ \mathbf{1}_{M+N_{in}} \end{bmatrix} + \frac{1}{2}\mathbf{1}_{R+M+N_{in}} \odot \begin{bmatrix} \frac{2}{N} V^T B_{xu} u_{t-1} \\ \mathbf{0}_{M+N_{in}} \end{bmatrix}$$

$$\iff c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t.$$

Having seen that the corresponding semilinear LSTM is capable of replicating the low-rank update equation for v_t as well as (implicitly) leveraging the update equation for z_t, we at last turn to analyzing the output states y_t. The output readout for the NM-LR-RNN (at time $t-1$, where $t \geq 2$) can be expressed as

$$y_{t-1} = Cx_{t-1} + r = C\left(\frac{1}{n} V V^T x_{t-1} + \left(I_{N\times N} - \frac{1}{N} V V^T\right) x_{t-1}\right) + r$$

$$= CV v_{t-1} + C\left(I_{N\times N} - \frac{1}{N} V V^T\right) x_{t-1} + r$$

$$CV v_{t-1} + C\left(I_{N\times N} - \frac{1}{N} V V^T\right) B_{xu} u_{t-1} = Dh_t + r,$$

where we have used our earlier equation $\left(I_{N\times N} - \frac{1}{N} V V^T\right) x_{t-1} = \left(I_{N\times N} - \frac{1}{N} V V^T\right) B_{xu} u_{t-1}$ and where $D \in \mathbb{R}^{N_{out}\times(R+M+N_{in})}$ is given in block matrix form as

$$D = \left(2CV \;\middle|\; 0 \;\middle|\; C\left(I_{N\times N} - \tfrac{1}{N} V V^T\right) B_{xu}\right).$$

Therefore, we find that the outputs of the NM-LR-RNN can be reproduced via a linear readout from the semilinear LSTM.

Here, it should be noted that the semilinear LSTM we have constructed "lags" behind the NM-LR-RNN by one timestep, i.e., the hidden state h_t is used to produce the $(t-1)$th output state y_{t-1}. This is a consequence of the fact that at each timestep in the NM-LR-RNN, the z_t state is updated before the x_t state, whereas in the semilinear LSTM, the cell state c_t (loosely corresponding to x_t) is updated before the hidden state h_t (loosely corresponding to z_t); as a result, the outputs of the semilinear LSTM are staggered by one timestep. In practice, this does not change the fact that the semilinear LSTM can replicate the outputs of the NM-LR-RNN.

As a final step, we count the total number of parameters used by the corresponding semilinear LSTM (along with its linear readout):

1. f_t: W_f only transforms the hidden state h_t (i.e., $W_f[h_{t-1}, u'_t] = W_s W_{zh} h_t$), giving us $R(R+M+N_{in})$ parameters. The bias vector b_f gives an additional $R+M+N_{in}$ parameters, for a total of $(R+1)(R+M+N_{in})$ parameters.

2. i_t: We zero out all of these parameters, giving us a count of 0.

3. $\tilde{c}_t$: The only parameters used in this computation are those that linearly transform u_{t-1} into R-dimensional space (i.e., the elements of $\frac{2}{N} V^T B_{xu}$), so the parameter count here is $N_{in}R$.

4. o_t: The bias term b_o was set to 0 and we defined the matrix W_o so that the middle M rows of W_o contained nontrivial entries, and the bottom N_{in} rows have N_{in} nontrivial parameters from the identity matrix $I_{N_{in} \times N_{in}}$. Consequently, we obtain a contribution of $M(R+M+3N_{in}) + N_{in}$ parameters from the output gate computation.

5. Readout: We have that $r \in \mathbb{R}^{N_{out}}$ and D uses a total of $N_{out}(R+N_{in})$ nontrivial parameters, giving us a total of $N_{out}(R+1+N_{in})$ parameters used.

Thus, the effective number of parameters used (and updated) by this LSTM is

$$(R+1)(R+M+N_{in}) + N_{in}R + M(R+M+3N_{in}) + N_{in} + N_{out}(R+1+N_{in})$$

$$= O\left(\max\{R^2, M^2, N_{in}R, N_{in}M, N_{out}R, N_{out}N_{in}\}\right).$$

Having showed that the parameters of the fully linearized NM-LR-RNN (potentially with feedback coupling) can be mapped onto the parameters of a semilinear LSTM, we conclude that the former can be viewed as an instance of a linearized LSTM operating in $(R + M + N_{in})$-dimensional space.

■

Remark 6. *Similarly to the LSTM, one could imagine incorporating input and output modulation into the NM-LR-RNN's design (in addition to the forget gate-like modulation of recurrent weights). This would likely allow the neuromodulatory sub-network to possess even greater flexibility in gating the dynamics of the output-generating RNN. We leave this exploration for future work.*

The upshot of this analysis is that we may view the NM-LR-RNN as a gated recurrent model, akin to the LSTM. In particular, it suggests that the NM-LR-RNN may be capable of overcoming the shortcomings of ordinary RNNs in modeling dependencies over long sequences. It also suggests that the NM-LR-RNN may be able to use its neuromodulatory gating to selectively filter input-related information and store certain kinds of information over long durations of time. In Chapter 4, we will see how the NM-LR-RNN indeed learns roughly as quickly as LSTM on a task that RNNs struggle to solve; moreover, we will take a closer look at how neuromodulatory gates might serve to "attend" to salient input information.

Chapter 4

Empirical Results

Having introduced and built a basic intuition for Neuromodulated Low-Rank RNNs in the previous chapter, we seek to understand how these models perform on different tasks of relevance to neuroscience. Crucially, we aim to study how well this class of RNNs is able to adapt to different contextual stimuli as well as generalize to out-of-sample inputs. Moreover, we aim to understand how these models may be advantageous over low-rank (and full-rank) RNNs, and we also revisit the NM-LR-RNN's connection to the LSTM (see Section 3.4) in practice. Following up on the motivation from Chapter 2, we first study the performance of NM-LR-RNNs on a time interval reproduction task (Measure-Wait-Go) and subsequently analyze how this class of models fares against non-neuromodulated RNNs as well as the LSTM on an "element finding" task.

4.1 Time Interval Reproduction

Chapter 2 introduced time estimation as a key task in which neural activity must flexibly keep track of different time intervals based on specific contextual inputs. In this section, we analyze how NM-LR-RNNs manage to satisfy this goal and, moreover, generalize to unseen time intervals. We also evaluate the relative advantages of incorporating neuromodulation by comparing these results against those of low-rank RNNs that do not use neuromodulation.

4.1.1 Measure-Wait-Go Revisited

Since Chapter 2 explored the ability of low-rank RNNs (with tonic contextual inputs) to exhibit out-of-sample generalization on the Measure-Wait-Go (MWG) task, we again revisit this task. In our experiments, however, we do not add tonic contextual input signals to each input sequence, aiming to discover whether NM-LR-RNNs may reliably generalize *without* augmenting training inputs with contextual cues. Ultimately, our results will demonstrate that neuromodulation is indeed capable of facilitating generalization.

The MWG training data was designed with the following requirements:

1. The total number of timesteps over which models operated was $T = 110$.

2. Each input sequence $u_0, \dots, u_T$ satisfied $u_i \in \mathbb{R}^3 \ \forall i = 0, \dots, T$, where all $u_i = 0 \in \mathbb{R}^3$ except for during the 'measure', 'wait', and 'go' timesteps. That is, if t_M, t_W, and t_G denote these timesteps, then we have $u_{t_M} = (10, 0, 0)^T$, $u_{t_W} = (0, 10, 0)^T$, and $u_{t_G} = (0, 0, 10)^T$.

3. The 'measure' cue was constrained to arrive at a some timestep t_M between 10 and 20. The target time interval T_* was chosen from the set $\{12, 14, 16, 18\}$, meaning that the 'wait' input cue arrived at timestep $t_W = t_M + T_*$. The 'go' cue always arrived after a 15-timestep delay period, i.e., $t_G = t_W + 15$.

4. For any given input sequence, the target output signal was a linear ramp given by

$$f(t) = \begin{cases} -\frac{1}{2} & t \leq t_G \\ \frac{1}{T_*}(t - t_G) - \frac{1}{2} & t_G \leq t \leq t_G + T_* \\ \frac{1}{2} & t \geq t_G + T_* \end{cases}$$

5. The training dataset consisted of all possible input sequences that satisfy the three requirements above. Training was performed over 50,000 iterations of full-batch backpropagation through time using the Adam optimizer with a learning rate of 0.001.

A sample MWG input-output pair (along with the corresponding loss mask) is shown in Figure 4.1.

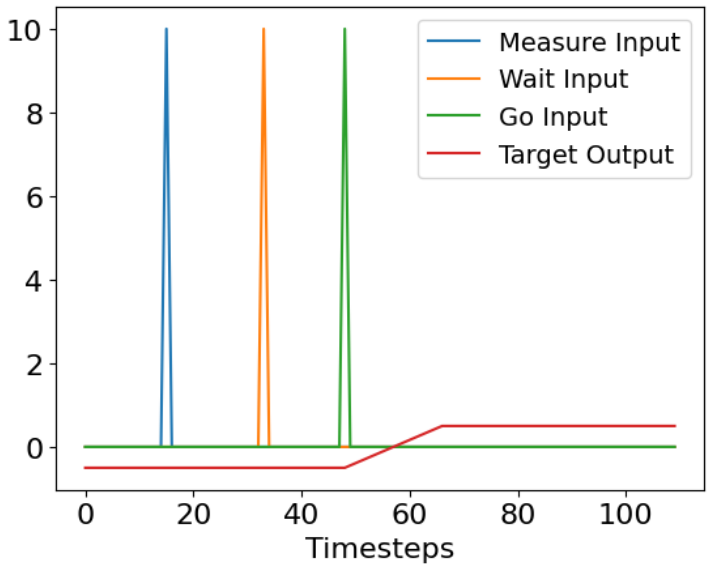
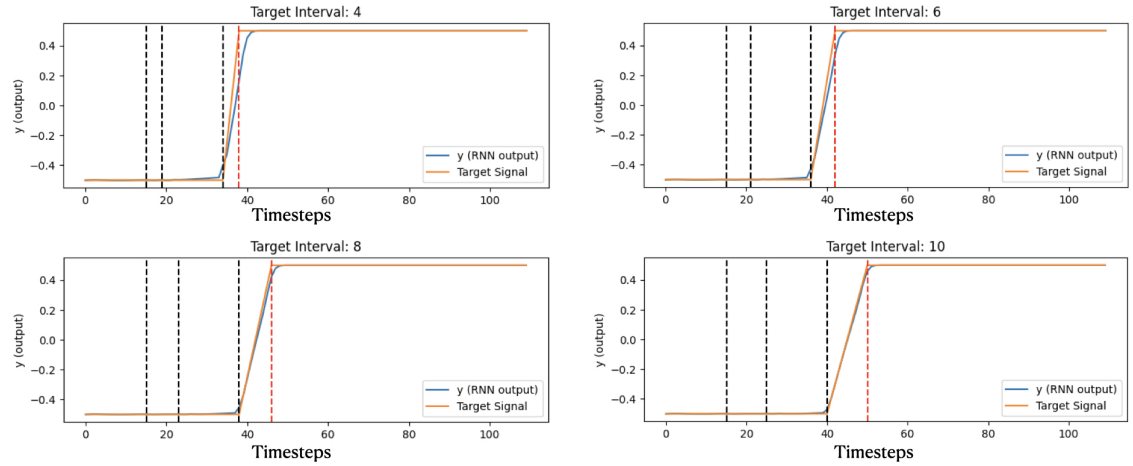


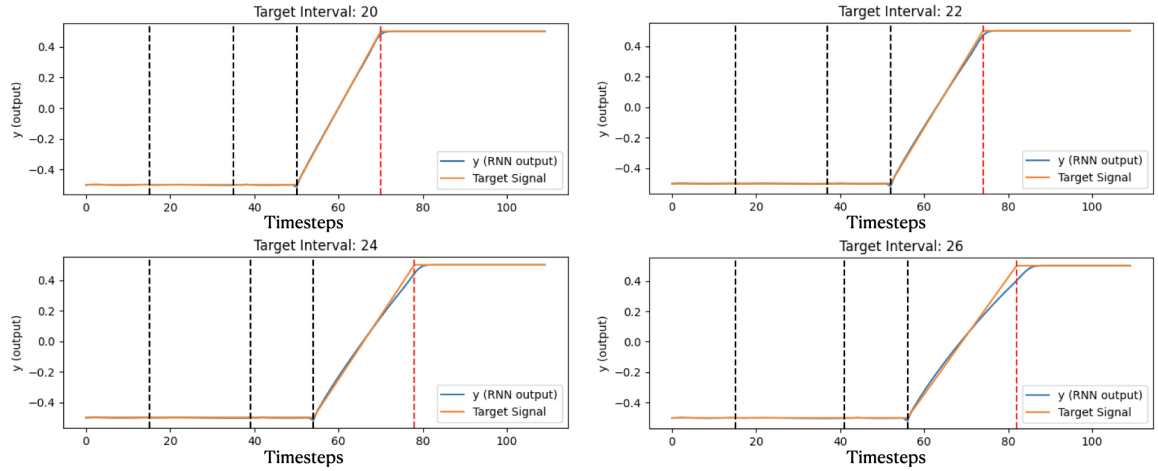
Figure 4.1 Sample inputs and outputs representing when $t_M = 15$, $t_W = 33$, and $t_G = 48$; here, $T_* = 18$ and so the target output signal linearly ramps from $t = 48$ to $t = 66$.

4.1.2 Low-Rank Neuromodulated RNNs can Generalize

We presently illustrate the ability of rank-2 and rank-3 NM-LR-RNNs to solve the MWG task and generalize to unseen intervals; in Chapter 5, we conduct a detailed analysis of rank-1 neuromodulated RNNs (which also turn out to successfully extrapolate). First, we train a rank-3 neuromodulated RNN ($M = 5$, $N = 100$, $R = 3$, $\tau_x = 1$, $\tau_z = 10$), as defined in Section 3.1, on the MWG task. As seen in Figure 4.2 and Figure 4.3 (right), this model is largely able to extrapolate to both lower and higher intervals (that were unseen during training). Figure 4.3 (left) also shows how the three components of $s(t) \in \mathbb{R}^3$ change across the timesteps of computation. $s(t)$ appears to remain fixed until the 'wait' cue arrives (marked in green for each time interval), at which point $s(t)$ starts moving along a new trajectory. Finally, upon arrival of the 'go' cue (marked in red for each time interval), the dynamics of s change yet again, settling to some new fixed point. Interestingly, the third component appears to settle to 0 quickly, whereas the first component settles to 0 over a larger timescale. Moreover, the changes undergone by any given component of $s(t)$ qualitatively appears to be similar across different time intervals, suggesting that $s(t)$ may be exhibiting the same general trend, simply staggered across time (based on when the 'go' cue arrives for replicating a given target time interval).



(a) Extrapolation to smaller time intervals.



(b) Extrapolation to larger time intervals.

Figure 4.2 Plots of NM-LR-RNN output compared against target output ramp for various time intervals that were not seen during training. 'Measure', 'wait', and 'go' cues are shown as dashed black lines (in that order), and a dashed red line marks the end of the target ramping period. (a) Rank-3 neuromodulated RNNs can generalize to novel intervals below those seen during training. (b) Rank-3 neuromodulated RNNs can generalize to larger intervals as well.

To confirm this intuition, we study the time-varying behavior of the vector $s(t)$ across different target time intervals by visualizing its top 2 principal components (Figure 4.4). This allows us to more clearly observe that the $s(t)$ trajectories corresponding to different time intervals are all roughly the same, starting at the same location (marked in orange) and ending at the same location (marked in black). It should also be noted that the trajectories are "temporally staggered," meaning that trajectories for smaller time intervals remain ahead of those corresponding to larger time intervals; this is more apparent when the individual components of $s(t)$ are visualized, as in Figure 4.3 (left).

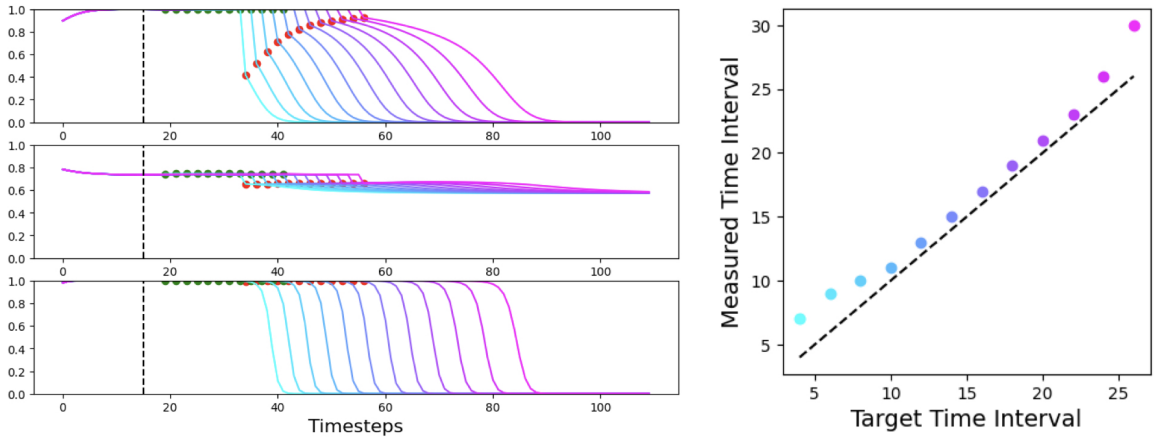


Figure 4.3 Performance of a rank-3 NM-LR-RNN ($M = 5$, $N = 100$, $R = 3$) on the MWG task. Left panel: the three neuromodulatory signal components (of $s(t)$) for measuring different time intervals. Measure cue shown as black dashed line, wait cue is marked in green, and go cue is marked in red. Right panel: accuracy of measured time interval. The measured interval (on the y-axis) was calculated by measuring the duration between the 'go' cue and when the model's output value first exceeded $0.5 - \epsilon$, where the tolerance ϵ was taken to be 0.01. The dashed line indicates the locations of the ground truth time intervals. For both panels, shorter intervals are represented in blue and longer intervals are represented in pink.

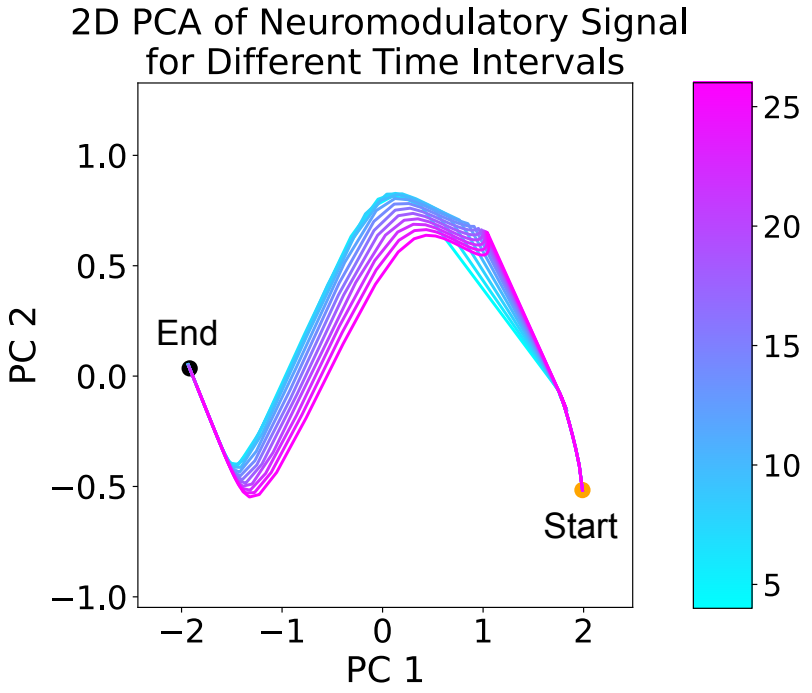


Figure 4.4 Top two principal components of $s(t)$ over the timesteps of the MWG task, visualized across the target time intervals $4, 6, \ldots, 24, 26$ (the color of the trajectory corresponds to the target interval length). Regardless of the time interval, $s(t)$ roughly follows the same trajectory. (PC's shown account for 99% of the variance in the data.)

In fact, we find that reducing the rank of the above network by 1 also yields a network that exhibits out-of-sample generalization, as illustrated by Figure 4.5 (where the NM-LR-RNN is initialized with $M = 5$, $N = 100$, $R = 2$, $\tau_x = 1$, and $\tau_z = 10$).

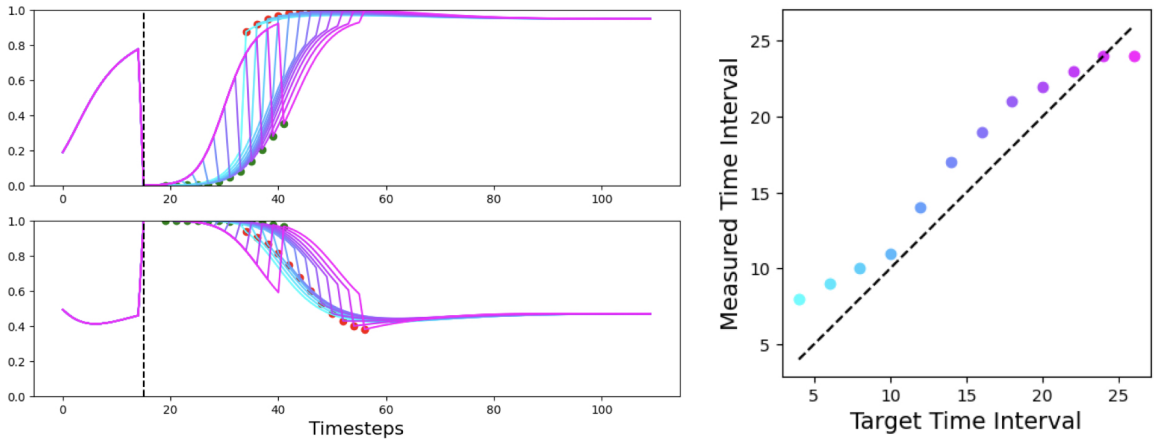


Figure 4.5 Performance of a rank-2 NM-LR-RNN ($M = 5$, $N = 100$, $R = 2$) on the MWG task. Left panel: the two neuromodulatory signal components (of $s(t)$) for measuring different time intervals. Measure cue shown as black dashed line, wait cue is marked in green, and go cue is marked in red. Right panel: accuracy of measured time interval. The measured interval (on the y-axis) was measured by the duration between the 'go' cue and when the model's output first exceeded 0.49 (where the tolerance level was $\epsilon = 0.01$). The dashed line indicates the locations of the ground truth time intervals. For both panels, shorter intervals are represented in blue and longer intervals are represented in pink.

4.1.3 Non-Neuromodulated RNNs Struggle to Generalize

As a benchmark for comparison with the previous results, we demonstrate that ordinary low-rank RNNs (without neuromodulation) can exhibit a certain degree of generalization, but generally fail to extrapolate as well as NM-LR-RNNs. Specifically, to compare against the rank-3 NM-LR-RNN ($M = 5$, $N = 100$, $R = 3$) studied in Section 4.1.2, we train an ordinary low-rank RNN with the same total number of parameters ($N = 106$, $R = 3$) on the MWG task, varying the dynamics constant τ across multiple training runs. (Recall that the dynamics equation for the low-rank RNN is $\tau \frac{dx}{dt} = -x + UV^T\varphi(x) + B_{xu}u$; we analyze the performance of rank-3 RNNs for different values of τ.)

Figure 4.6 depicts the ability of such a low-rank RNN to correctly reproduce various target time intervals, across different values of τ. Some of these networks exhibit slight extrapolation ability (e.g., the model with $\tau = 3$ extrapolates reasonably well to

larger intervals). However, none of the networks tested manage to successfully reproduce the trained time intervals *and* generalize to reproducing higher or lower time intervals, consistent with the findings of Beiran et al (2023) [52]. This suggests that having a neuromodulatory sub-network helps in reliably facilitating out-of-sample generalization.

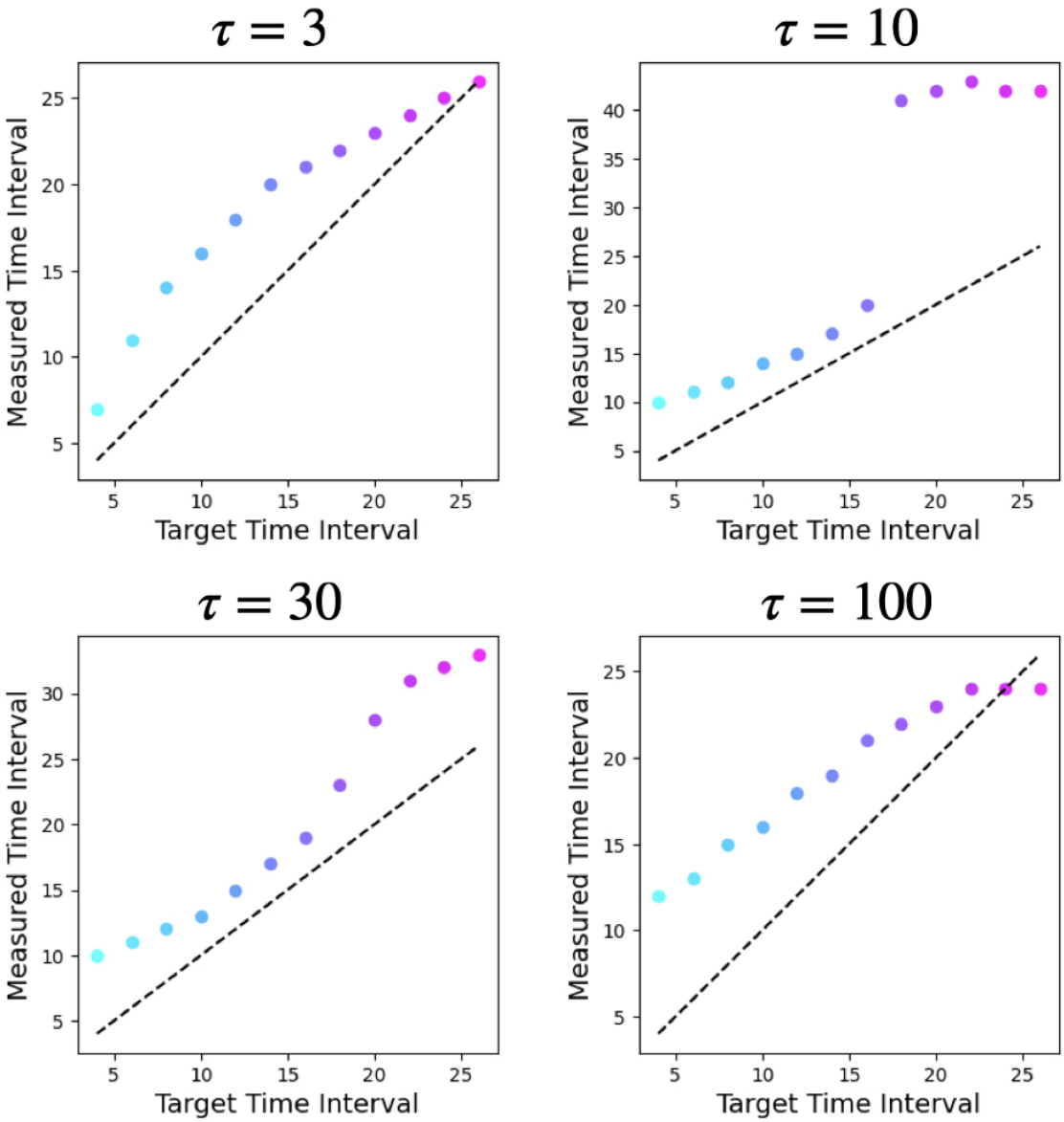


Figure 4.6 Assessing the extrapolation ability of regular rank-3 RNNs (with no neuromodulation; $N = 106, R = 3$) with the same total number of parameters as the NM-LR-RNN whose results are shown in Figure 4.3. Despite varying τ (the timescale for the dynamics of the rank-3 RNN), these models struggle to extrapolate to both higher and lower time intervals. Measured time intervals were computed as the duration between the 'go' cue and when the RNN's output value first exceeded 0.49 (i.e., the tolerance level was $\epsilon = 0.01$). Dashed line ($y = x$) depicts the locations of the true time intervals.

4.2 Element Finding Task

Inspired by the similarity between the NM-LR-RNN and the LSTM (see Section 3.4), we seek to identify a sequence-related task on which LSTMs are able to outperform ordinary RNNs, owing to their ability to more accurately capture dependencies over long sequences. As such, we might expect the NM-LR-RNN – whose neuromodulatory gating resembles the forget gate of an LSTM – to outperform ordinary full-rank and low-rank RNNs on such a task. In particular, in this section, we investigate the NM-LR-RNN's ability to perform what we name the *Element Finder Task*. In particular, we will show that the NM-LR-RNN – when endowed with suitable feedback coupling from the low-rank RNN to the neuromodulatory RNN – does indeed demonstrate LSTM-like performance on the Element Finder Task, while vanilla RNNs (with the same total parameter count) fail to solve this task. Moreover, we will study how the NM-LR-RNN manages to solve this task by carefully analyzing its internal states. Crucially, we will arrive at a picture of how neuromodulation can add flexibility to neural computations by means of "attending" to the query index.

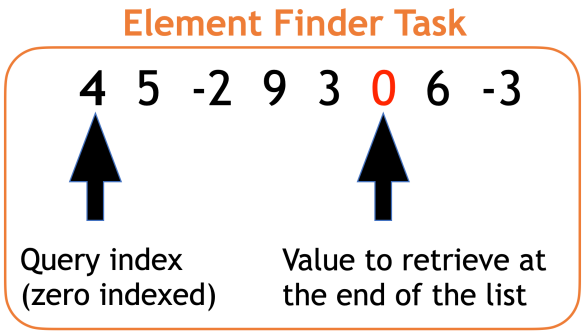


Figure 4.7 This figure illustrates the input stream for a sample sequence of length 7 (plus one query index) that could be used for the Element Finder Task. Each number (a one-dimensional input) is presented at a different timestep.

The Element Finder Task (see Figure 4.7) is designed as a variant of the task of retrieving the first element in a long sequence of numbers, originally presented in [56] in which the LSTM was introduced. The input stream consists of a sequence of l integers, uniformly sampled from some set of consecutive integers (e.g., $\{-10, -9, \ldots, 9, 10\}$) along with a query index, $q \in \{0, 1, \ldots, l-1\}$. At the first timestep $t = 0$, the query index q is passed as a one-dimensional input to the model; the goal of the model is to recover the value of the element at index q along the sequence of integers (where we use the

convention of starting sequences at the index 0, and so on). At each time $t = i$, for $i \geq 1$, the ith element of the sequence is passed as a one-dimensional input to the model. Finally, at time $t = l$, the model must output (as a one-dimensional quantity) the value of the element at index q (i.e., the $(q + 1)$th element in the sequence). Given that a model must learn "attend" to elements in potentially large sequences, it is conceivable that ordinary RNNs may not be sufficient to solve this task, whereas gated recurrent models may prove effective.

As an additional remark, the Element Finder Task may be loosely viewed as an abstracted version of a timing task (like the Measure-Wait-Go task). Indeed, the query index determines how long the model must wait until selecting an element value, and moreover, this element value must be retained until the end of the input sequence is reached. In particular, a model that solves the Element Finder Task must learn to 'wait' across different time intervals (or timesteps) based on the contextual input cue – the query index – provided at the start of any input sequence.

4.2.1 Models of Comparison

When training models on the Element Finder Task, the following models were used as a benchmark for comparison:

1. The LSTM, as defined in Section 3.4, along with a linear readout $y_t = Ch_t$ from the hidden state (with no readout bias vector).

2. Vanilla RNNs of specified rank R, defined via the following dynamics equations (see Chapter 2):

$$\tau \frac{dx}{dt} = -x + \frac{1}{N} UV^T \varphi(x) + B_{xu} u$$

$$\text{and } y = Cx.$$

   Here, $U, V \in \mathbb{R}^{N \times R}$, and for simplicity, we do not use a readout bias vector (for computing y).

The NM-LR-RNNs that were tested were defined as in Section 3.1, except with the addition of a feedback coupling term from the output-generating RNN to the neuromodulatory network (inspired by this model's connection to the LSTM that was explored in

Section 3.4). That is, the update to the neuromodulatory state $z \in \mathbb{R}^R$ was given by

$$\tau_z \frac{dz(t)}{dt} = -z(t) + W_{zz}\varphi(z(t)) + (W_{zx}\varphi(x(t)) + b_z) + B_{zu}u(t) \tag{4.1}$$

where $W_{zx} \in \mathbb{R}^{M\times N}, b_z \in \mathbb{R}^M$ are learnable weights and biases for the feedback coupling term. Similarly to the other models used, the NM-LR-RNNs used a linear readout of the form $y(t) = Cx(t)$.

4.2.2 NM-LR-RNNs can be LSTM-like Learners

The performances of multiple NM-LR-RNNs on this task were evaluated. The parameters (M, N, R) were varied, though $\tau_x = 10$ and $\tau_z = 2$ for all NM-LR-RNNs that were initialized. All the models tested (including the benchmark LSTM and RNN models) had roughly the same total parameter count (~ 500 parameters). Accordingly, for the LSTM, a hidden size of $N_{LSTM} = 10$ was used and, a full-rank RNN with $R = N = 16$ and $\tau = 10$ was used as a baseline. Additional low-rank RNNs were also trained as a baseline (shown in Table 4.1). Training was performed on 20,000 batches of sequences of length 25 (excluding the query index), where each batch contained 128 sequences. Each integer in each sequence was uniformly sampled at random from the set $\{-10, -9, \ldots, 9, 10\}$. Finally, all models were trained using gradient descent with the Adam optimizer; the learning rate for the LSTM and the NM-LR-RNNs was 1e-2 and was varied for the the RNN. All models tested are shown in Table 4.1 below. In particular, Table 4.1 shows the minimum and average mean-squared error (MSE) losses attained by each model after 20,000 batches of gradient descent training.

In particular, both full-rank and low-rank RNNs were unable to successfully solve the Element Finder Task (after 20,000 batches of training), as evidenced by their overall inability to attain low MSE loss (by each of the three metrics in the table). The LSTM readily and consistently attains a MSE loss that is close to zero, indicative of high task performance. Interestingly, a number of NM-LR-RNNs are also able to solve this task. Although these models do not appear to solve the task as consistently as the LSTM or attain as low a mean-squared error loss as the LSTM, they still demonstrate a high task performance that RNNs and LR-RNNs lack. This characteristic is highlighted in the categorical plot in Figure 4.8.

Model Performance on Element Finder Task			
Model Description	Lowest Loss Attained (after 20,000 batches)	Average Minimum Loss Attained (after 20,000 batches)	Proportion of Runs with Minimum Loss < 1
LSTM ($N_{LSTM} = 10$)	0.0125	0.0262	10/10
RNN (R=N=16, lr = 0.001)	22.67	24.088	0/10
RNN (R=N=16, lr = 0.01)	6.945	20.650	0/10
RNN (R=N=16, lr = 0.1)	23.037	24.776	0/10
LR-RNN (N=23, R=10, lr = 0.01)	3.069	20.991	0/10
LR-RNN (N=31, R=7, lr = 0.01)	18.809	23.018	0/10
LR-RNN (N=50, R=4, lr = 0.01)	23.065	23.906	0/10
LR-RNN (N=83, R=2, lr = 0.01)	22.803	23.877	0/10
NM-LR-RNN (M=15, N=6, R=5)	0.264	16.942	1/10
NM-LR-RNN (M=10, N=14, R=5)	0.276	9.901	6/10
NM-LR-RNN (M=10, N=12, R=7)	0.262	9.684	6/10
NM-LR-RNN (M=5, N=13, R=12)	0.145	6.836	7/10
NM-LR-RNN (M=5, N=18, R=8)	0.175	6.866	7/10

Table 4.1: Summary statistics for different models trained on the Element Finder Task for 20,000 batches. NM-LR-RNNs with relatively smaller neuromodulatory RNNs and relatively larger output-generating RNNs appear to learn the task better.

In addition to these results, the specific MSE loss curves over batch training was visualized for one particular initialization of the LSTM, vanilla (full-rank) RNN, and selected NM-LR-RNNs (Figure 4.9). Each of the NM-LR-RNNs appear to learn the task to a high degree of accuracy within 20,000 batches of training. Moreover, two of the selected neuromodulated RNNs appear to exhibit a steep descent in their respective MSE loss curves – and, thus, an ability to solve the task fairly well – around the same

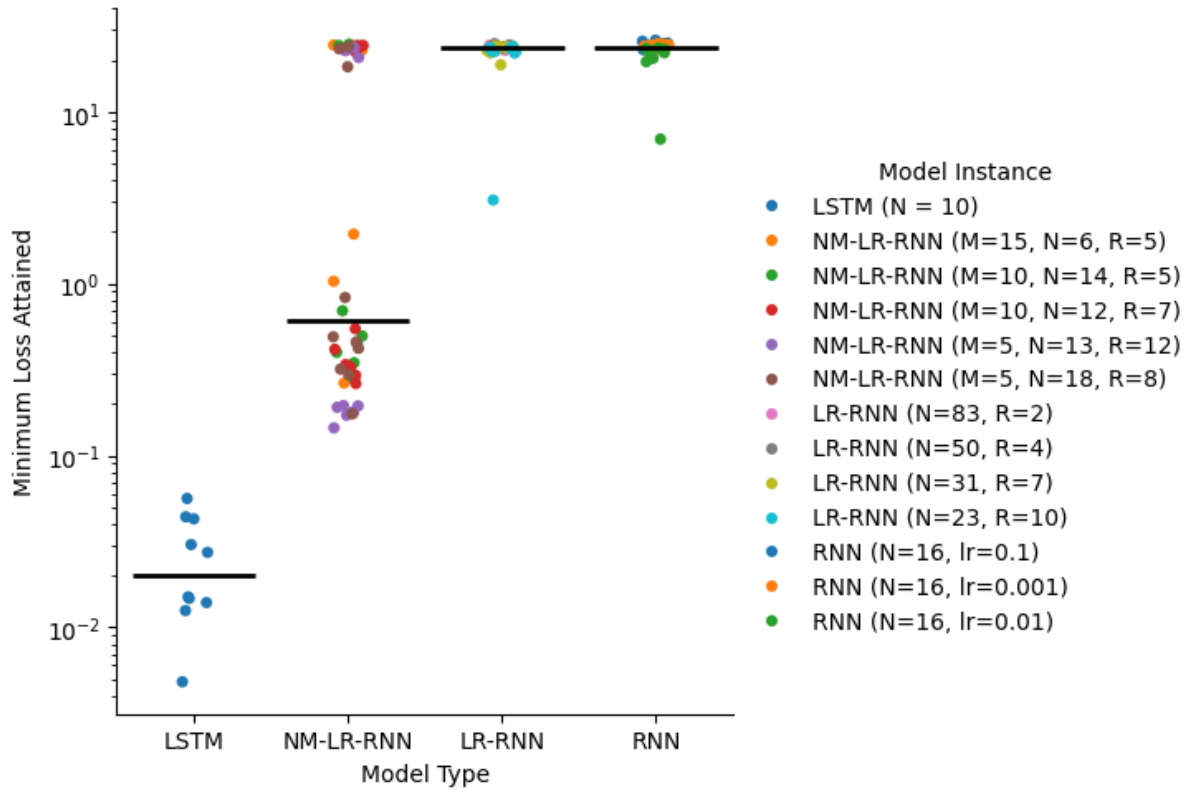


Figure 4.8 Categorical plots show the distribution of minimum losses attained on runs of an LSTM (10 total runs), full-rank RNNs (30 total runs), low-rank RNNs (40 total runs), and NM-LR-RNNs (50 total runs), after training for 20,000 batches of gradient descent. Whereas full- and low-rank RNNs exhibit high minimum loss, the minimum loss profile for NM-LR-RNNs is characterized by a bimodal distribution, in which most models attain a minimum MSE loss below 1.0. Median minimum loss attained is shown as a black line for each model type.

time that the LSTM's loss curve steeply drops.

4.2.3 How do Neuromodulated RNNs Solve the Element Finder Task?

Having seen that NM-LR-RNNs can possess the same learning advantages that LSTMs have over ordinary RNNs, we seek to understand how the NM-LR-RNN is solving the element finding task. As a starting point, consider one particular instance of an NM-LR-RNN with $M = 5$, $N = 18$, and $R = 8$. Simply stated, the two central questions we seek to answer are:

1. What computation is the neuromodulatory sub-network performing?

2. What computation is the output-generating network performing?

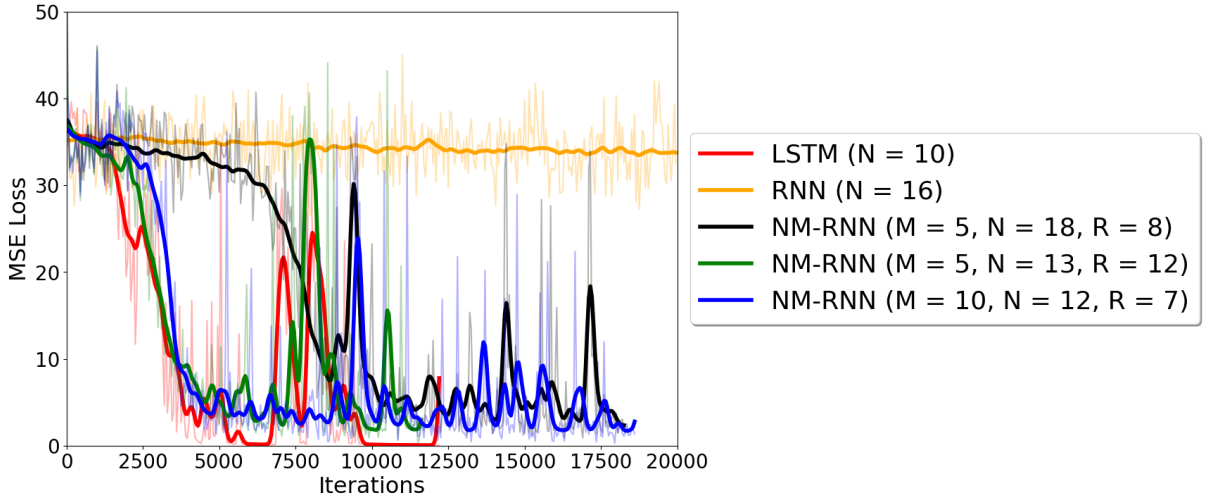


Figure 4.9 Training loss curves are shown for one run of an LSTM, a full-rank RNN, and three different NM-LR-RNNs. All three neuromodulated RNNs achieve low loss (like the LSTM), and two of them appear to exhibit similar learning dynamics to the LSTM; on the other hand, the RNN fails to learn the task. For each gated model, loss curves are truncated near the iteration on which the lowest loss (over 20,000 batches) is attained.

We can think of the Element Finder Task as having two key variables of interest: the query index and the target element value. By correctly using both of these variables, a network is able to solve the task. Thus, as a crude first pass at answering the questions posed above, one can visualize the states $s(t)$, $x(t)$, and $y(t)$ across time, first while fixing the query index q and varying the target element value, and subsequently by fixing the target element value and varying the query index. This will enable us to determine how each of these two variables – when isolated from the other – affects the states of the model.

Figure 4.10 shows the components of s, x, and y when fixing the query index to 10 and varying the target element value. Upon inspection, it appears that the behavior of the components of $s(t)$ (Figure 4.10; left) do not vary as the element value is varied. That is, the neuromodulatory network may function independently of the element value. Inspection of the output-generating network's state variable x (Figure 4.10; middle) shows that its components appear to scale and flip as the element value ranges from -10 to 10. Finally, turning to the output state y (Figure 4.10; right), the output state appears to follow the same initial trajectory until the query timestep arrives, at which point it rapidly settles to a fixed point – namely, the target element's value.

On the other hand, Figure 4.11 shows the components of s, x, and y when fixing the target element value to be 5 and varying the query index presented at the start of the

sequence. Inspecting the components of the neuromodulatory signal $s(t)$ (Figure 4.11; left), we see that the onset of the query timestep appears to facilitate a *switching* of components from 0 to 1, and vice versa. That is, the neuromodulatory network seems to mediate context-dependent gating (where the "context" refers to the query index). Looking to the x and y states (Figure 4.11; middle, right), we see that the components of x and y appear to settle around 0 before the query timestep arrives. Then, both variables rapidly settle to a fixed point; in particular, y settles to the target element value of 5.

Thus, a cursory analysis of the NM-LR-RNNs state variables suggests that the neuromodulatory sub-network may be receptive to the query index while the output-generating network serves to retrieve the target element's value based on the gating supplied by the neuromodulatory sub-network. In subsequent sections, we dive into a deeper analysis of the neuromodulatory and output-generating networks, and ultimately build further compelling evidence that will validate this hypothesis.

4.2.4 Analysis of the Neuromodulatory Sub-network

In this section, we carry out a detailed analysis of the neuromodulatory sub-network when performing the Element Finder Task. Given that the nonlinearly transformed output vector $s(t)$ of the neuromodulatory network is changing the low-rank modes of the output-generating network, we first study how components of s behave over the timesteps of the Element Finder Task. Each of the four plots in Figure 4.12 shows the behavior of one selected component of s (for a NM-LR-RNN with $(M, N, R) = (5, 18, 8)$) across different query indices. The four plots shown capture the typically observed behaviors for components of s: switching from 0 to 1 (or vice versa) upon onset of the query index, remaining at 0 (or 1) across all timesteps, or gradually transitioning from 1 to 0 (or vice versa) so that the transition is finished around the time the query index arrives. Overall, the state s is strongly attuned to the query index. Moreover, the upper two plots of Figure 4.12 clearly illustrate the notion of s as a gating variable, that can "turn on" and "turn off" rank-1 factors of the recurrence matrix W_{xx} as needed. The lower left plot shows that s can also act by permanently leaving a rank-1 mode "on" or "off," indicating that it has the power to further affect the effective rank of W_{xx}. Finally, the lower right plot suggests that certain components of s may serve a preparatory function, ramping up or down in anticipation of the query index. We also see that components of s are able to

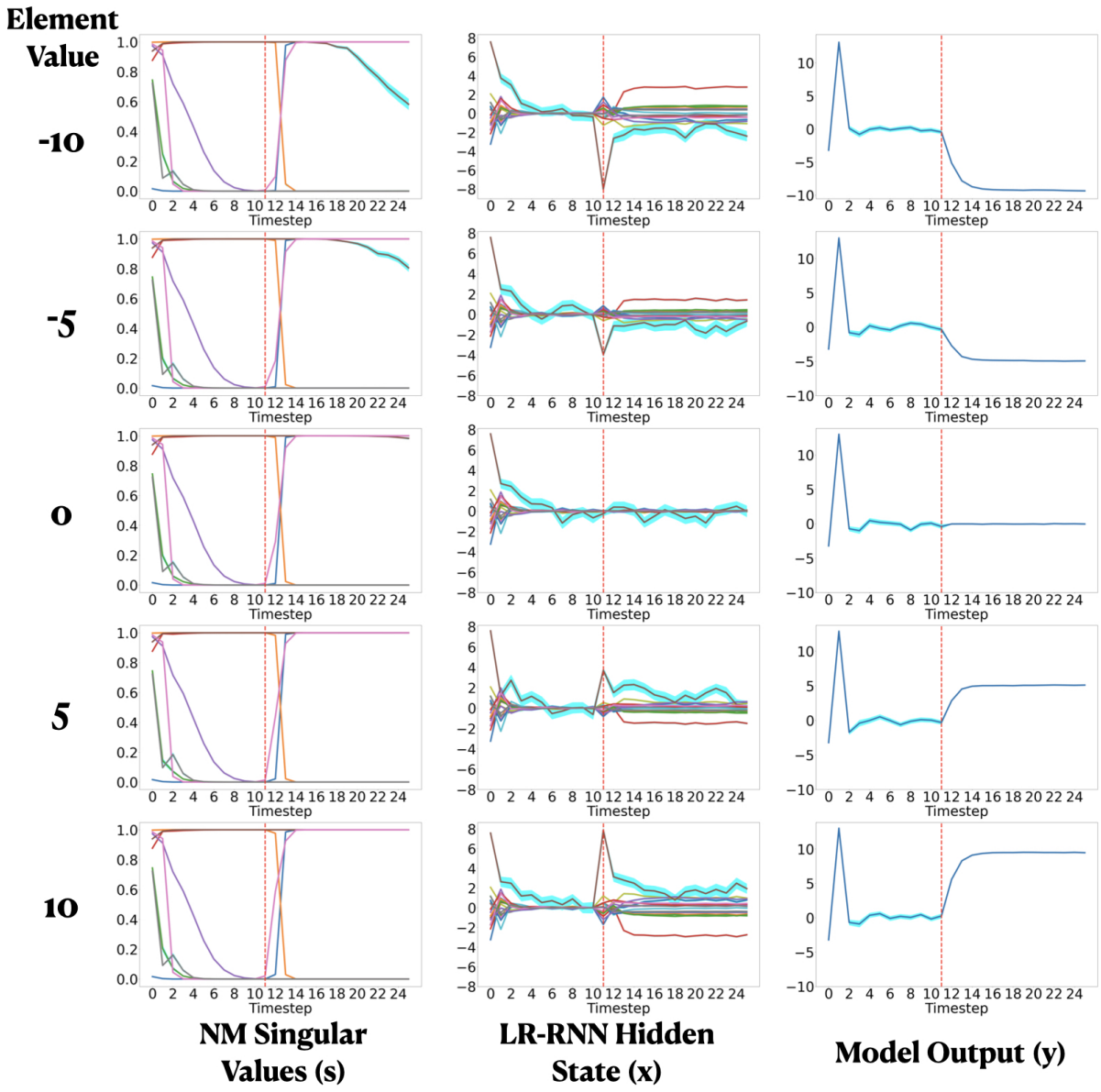


Figure 4.10 Sample internal states of a particular trained NM-LR-RNN ($M = 5$, $N = 18$, $R = 8$) across the timesteps of computation, shown for 5 different element values (-10, -5, 0, 5, and 10). Each plot shows how all of the components of one of the vectors $s(t)$ (left), $x(t)$ (middle), $y(t)$ (right) vary through time. The query index is fixed to be 10, as indicated by the red dashed line in each plot. Each line shown is averaged over 100 independent runs of the model (standard error shown in cyan).

switch at fast and slow timescales. Additionally, it should be noted that while s itself may exhibit discrete switching behavior, this really corresponds to the pre-sigmoidal variable $s_{pre} := W_s z + b_s = \sigma^{-1}(s)$ smoothly crossing 0 (see Figure 4.12, left).

Having analyzed the neuromodulatory signal s provided to the output-generating network, we turn to analyzing the internal state $z(t)$ that produces $s(t)$. Figure 4.13 shows plots for each component of z across different query indices, for a particular NM-

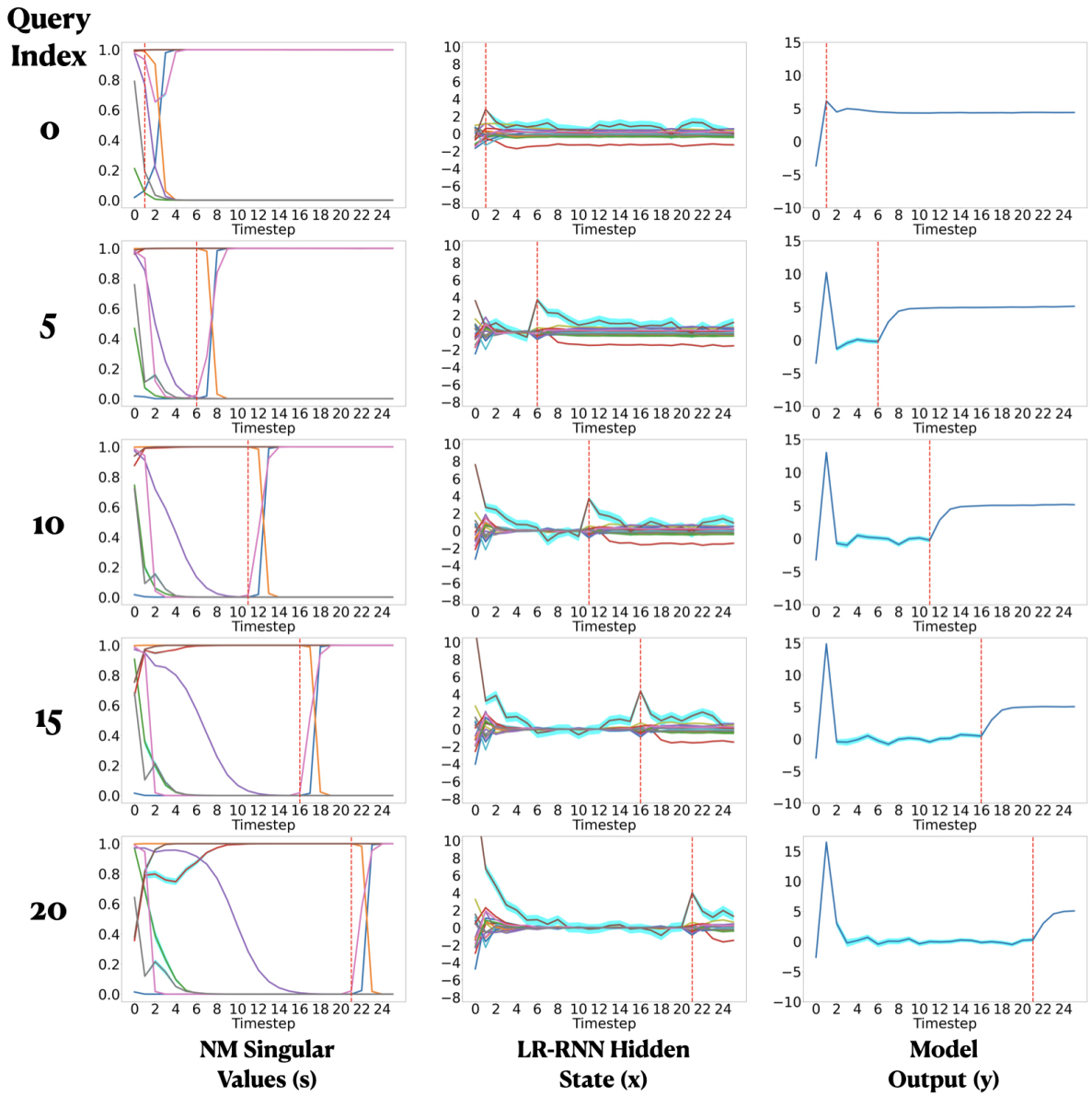


Figure 4.11 Sample internal states of a particular trained NM-LR-RNN ($M = 5$, $N = 18$, $R = 8$) across the timesteps of computation, shown for 5 different query indices (0, 5, 10, 15, and 20), while fixing the target element value to be 5 in each case. Each plot shows how all of the components of one of the vectors $s(t)$ (left), $x(t)$ (middle), $y(t)$ (right) vary through time. In each plot, the onset of the query index is indicated by the red dashed line. Each line shown is averaged over 100 independent runs of the model (standard error shown in cyan).

LR-RNN that has learned the Element Finder Task. In particular, each component follows a similar trend across different query indices, though for any given component of z, there is consistently an an offset between the value of this component across different query indices. This suggests that, regardless of the query index, the neuromodulatory RNN is performing the same overall kind of computation.

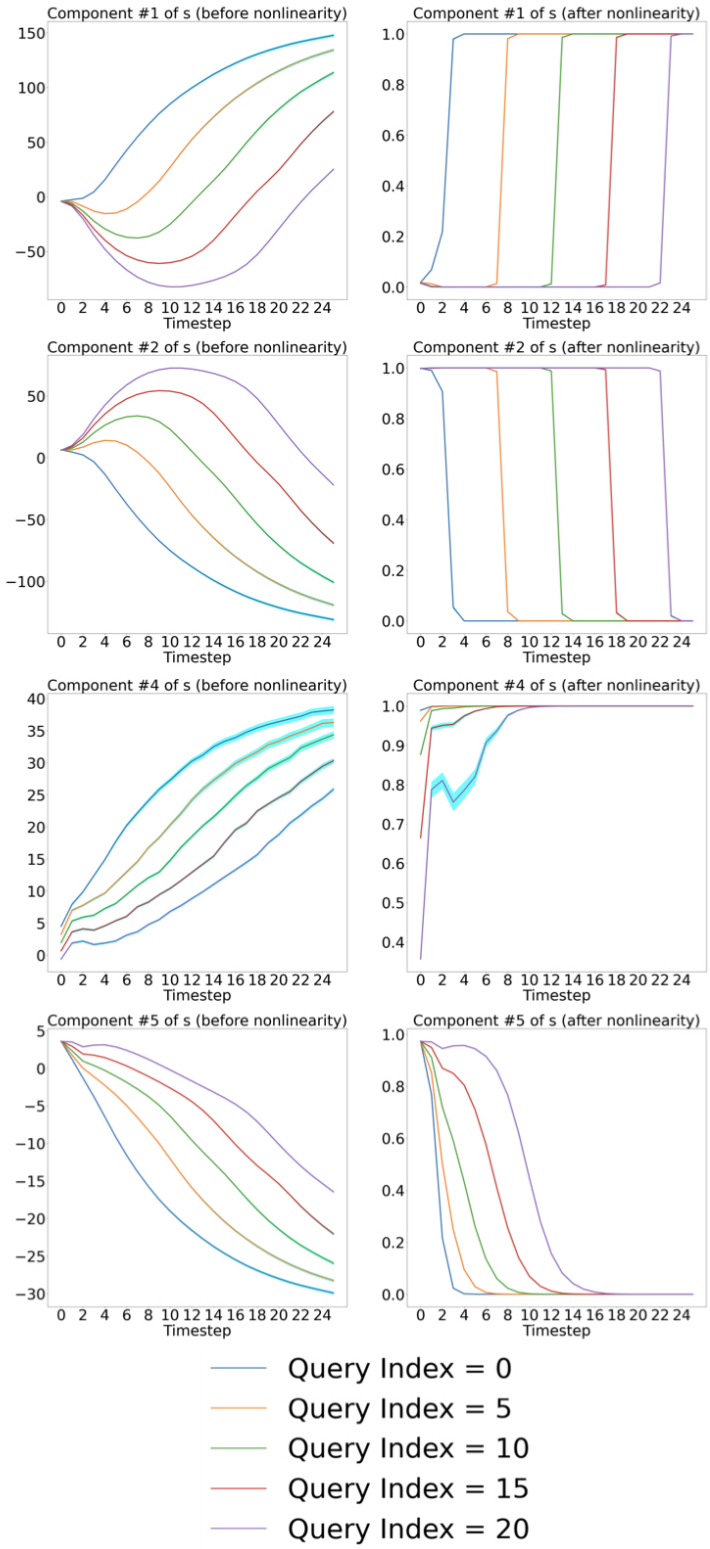


Figure 4.12 Plots of the selected components of the readout vector $s_{pre}(t) := W_s z(t) + b_s$ (left) and the corresponding nonlinear transformation $s(t) = \sigma(s_{pre})$ (right) for an NM-LR-RNN with $M = 5$, $N = 18$, and $R = 8$. Unlike the components of $s(t)$, which may exhibit gate-switching behavior, those of $s_{pre}(t)$ exhibit continuous, graded changes; in particular, a switching of the ith component $s_i(t)$ from 0 to 1 (or vice versa) corresponds to $(s_{pre})_i(t)$ smoothly crossing 0. Results are averaged over 100 independent trials; standard error is shown in cyan.

To bring further clarity to this last point, Figure 4.14 shows the (trial-averaged) behavior of the top two principal components of $z(t)$ for different query indices. It reveals that the $z(t)$ follows a similar sort of curved trajectory (across different query indices) that ends up falling on an approximate line attractor-like structure, ordered by the query index. All trajectories initially start on the same linear path that rapidly diverges into multiple curved trajectories, indicating that the presentation of the query index at the very beginning of the input sequence "kicks" the dynamics onto different possible trajectories. This underscores the idea that the neuromodulatory sub-network attends to the query index and gates the dynamics of the output-generating network accordingly.

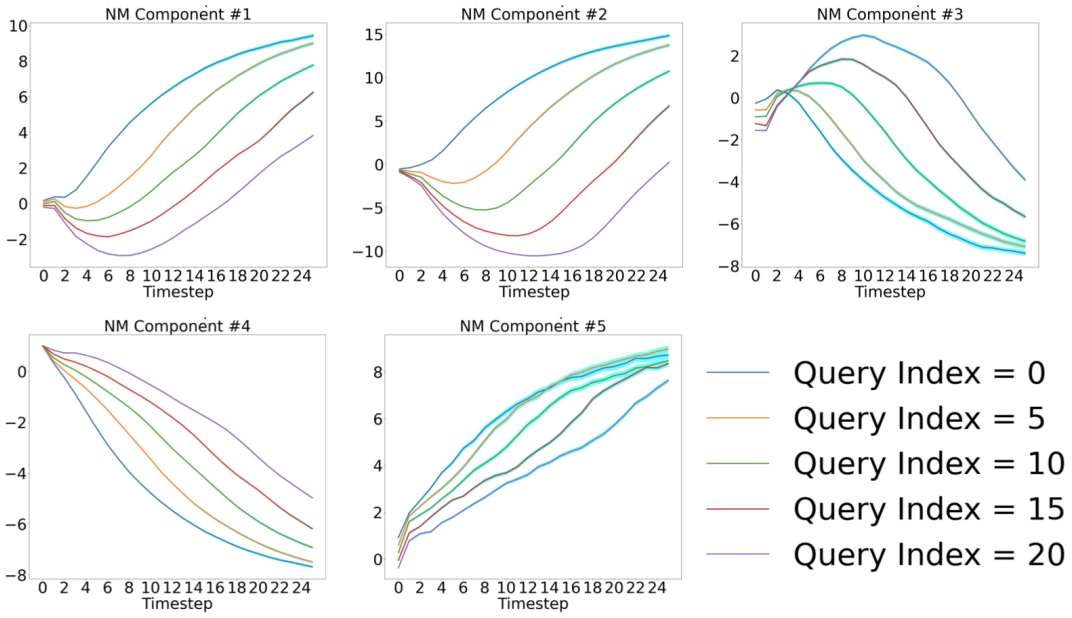


Figure 4.13 Plots of selected components of the vector $z(t)$ (for an NM-LR-RNN with $(M, N, R) = (5, 18, 8)$) during the Element Finder Task, where each plot shows one particular component of z across different query indices. Each line shown is averaged over 100 independent runs (standard error is shown in cyan).

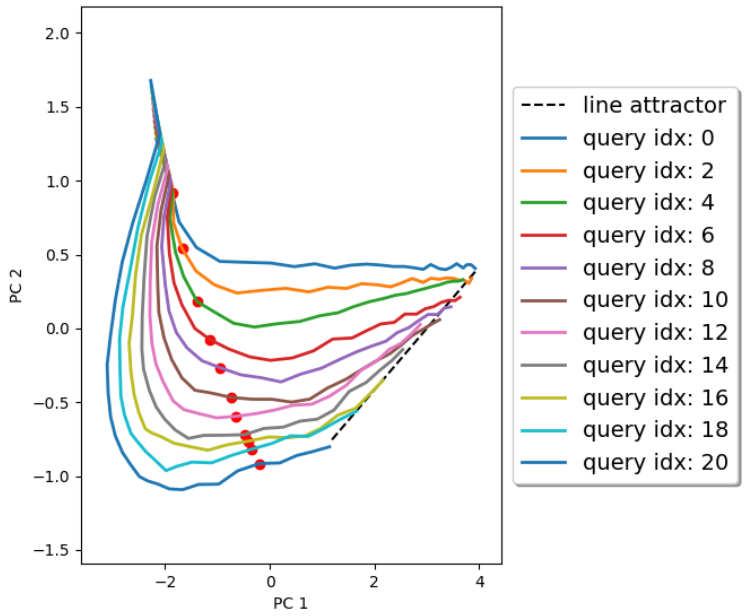


Figure 4.14 Two-dimensional PCA representation of $z(t)$, shown across different query indices. (Using the top 2 PC's accounted for over 99% of the variance in the data.) Each curve is averaged over 100 independent runs of a trained NM-LR-RNN with $(M, N, R) = (5, 18, 8)$. All trajectories start in the upper left of the figure at time $t = 0$ and end up falling on an approximate line attractor (shown as a dashed line). For each trajectory, the timestep given by the query index is marked in red.

4.2.5 Analysis of the Output-generating Network

Now that we have highlighted the neuromodulatory sub-network's role in attending to the query index, we aim to understand how neuromodulation affects the dynamics of the output-generating RNN. Figure 4.11 (middle) illustrates that for a fixed element value (5 in this case), varying the query index directly affects the amount of time the components of $x(t)$ remain settled around 0. Then, right before the query index arrives and when the query arrives, certain components of $x(t)$ quickly grow in magnitude, suggesting that $x(t)$ spirals away from the origin (and, furthermore, the middle column in Figure 4.10 indicates that the size and direction of this spiral scales with the element value). This is followed by a rapid settling of the components of $x(t)$ to a fixed point. This general

trend remains the same when the element value remains fixed; varying the query index only temporally offsets the moment when $x(t)$ leaves the origin and settles to a new fixed point.

To bring clarity to these trends, we visualize its top 2 principal components of $x(t)$ for different trajectories that correspond to different query indices and target element values (Figure 4.15). The top panel illustrates how all trajectories with the same query index ($q = 15$) start at the same location; these trajectories then spiral into the origin, remaining there until the query timestep arrives. Then, depending on the value of the target element, each trajectory rapidly settles to a fixed point on an approximate line attractor. Interestingly, this line attractor-like structure encodes a mostly accurate ordering of the numbers $-10, -9, \ldots, 9, 10$. The bottom panel of Figure 4.15 further illustrates the dynamics of $x(t)$ for sequences with different query indices (5, 10, and 15). Trajectories that possess different query indices have different starting locations in the PC space. Still, all trajectories appear to converge around the origin; once the query index arrives, each trajectory converges to a point on the line attractor corresponding to the target element value for that respective sequence.

Through these analyses, we now illustrate how the NM-LR-RNN is able to solve the Element Finder Task. First, when the query index is presented at the start of an input sequence, the neuromodulatory network "attends" to this index. Some of its components stay fixed permanently at 0 or 1, while others will abruptly switch upon arrival of the query timestep, while others will slowly ramp up or down until the onset of the query index. It is also possible that the output-generating network aids in this computation. Indeed, in the right column of Figure 4.11, the y state exhibits a brief spike (within the first few timesteps) whose magnitude increases as the query index increases, suggesting that the initial trajectory traced out by $x(t)$ could help the neuromodulatory RNN to store a trace of the query index (owing to the feedback coupling from x to z). Next, the output-generating network leverages the power of switching nonlinear dynamics: while $x(t)$ originally settles to 0 (or near 0), the onset of the query index produces gate-switching in $s(t)$, which in turn abruptly sends $x(t)$ to a new fixed point.

As we observed earlier (e.g., in Figure 4.12), regardless of the value and query index, the components of $s(t)$ change in the same exact way (just at different times, based on the query index). This suggests that the neuromodulatory network is able to switch the

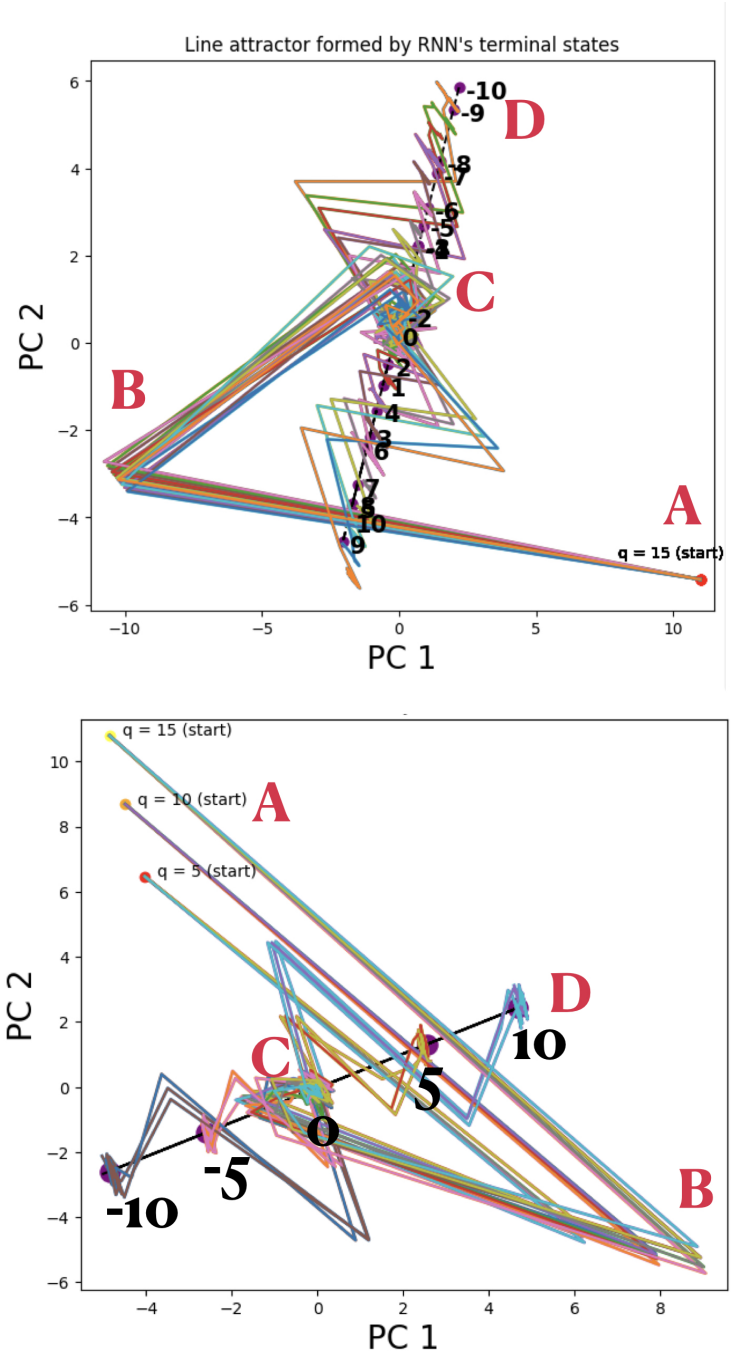


Figure 4.15 2-dimensional PCA representation of the low-rank RNN's internal state $x(t)$ for different target element values (where $M = 5$, $N = 18$, and $R = 8$). Distinct trajectories (averaged over 100 independent runs of the model) are shown for each of values in $\{-10, -9, \ldots, 9, 10\}$. For both figures: **A** The starting location for each trajectory is determined by the query index. **B** All trajectories initially follow the zig-zag path (roughly). **C** All trajectories spiral into the origin. **D** Upon arrival of the query index, each trajectory rapidly leaves the origin and settles to a fixed point that lies on an approximate line attractor ($R^2 > 0.90$ in both plots). For the bottom figure, different trajectories for various query indices and target element values are shown. (For both figures, using 2 principal components accounts for over 80% of the variance in the data.)

dynamical landscape of the output-generating RNN, by the modulating rank-1 factors of the recurrence matrix W_{xx} (via $s(t)$). Since the output-generating network rapidly settles to a fixed point following the query timestep, there is reason to believe that the output-generating network now follows dynamics such that the first input value seen since the query timestep – namely, the target element itself – is best retained until the end of the input sequence is reached.

Thus, we have evidence that the neuromodulatory sub-network is able to change the dynamical structure defined by the output-generating network. This can have the effect of changing the fixed point structure, as observed in Figure 4.15: fixed points at or near the origin are kicked to new fixed points on an approximate line attractor as a consequence of neuromodulatory gates opening and closing (during the query timestep). In Chapters 5 and 6, we dive deeper into the fixed point structure of the NM-LR-RNN, crucially seeking to understand how neuromodulation can alter the stability of fixed points. Furthermore, in Chapter 7, we will explore the notion of the NM-LR-RNN as a latent inference model that can flexibly switch between different dynamical systems.

Chapter 5

Building a Theory of Neuromodulation

Our empirical analyses from Chapter 4 warrant a deeper mathematical analysis of how the NM-LR-RNN uses neuromodulation to perform sequence-related tasks, like timing and element finding. In Section 3.3.2, we began to get a glimspe into the role of neuromodulation in affecting the timescales of neural dynamics. Presently, we seek to carry out a mathematical analysis of how the network's neuromodulatory signal is constrained by the target output signal (as required by the task). Using the theory developed in this section, we revisit the time interval reproduction task, and verify that the theoretically predicted neuromodulatory signal is indeed observed in practice. Finally, we will study the fixed point structure of our model, crucially exploring how the addition of neuromodulation to RNNs affects the existence and stability of fixed points.

In this chapter, we largely restrict our analyses to the case of rank-1 networks (i.e., $R = 1$), as this allows us to precisely identify the neuromodulatory signal $s(t) \in \mathbb{R}$; when $R > 1$, there may be infinitely many neuromodulatory signals $s(t) \in \mathbb{R}^R$ that produce a given output signal.

5.1 How is Neuromodulation Constrained by the Output Signal?

We now analyze the general rank-1 NM-LR-RNN where the readout signal $y(t) \in \mathbb{R}$ is a scalar. Because $R = 1$, we introduce have $U \in \mathbb{R}^{n\times 1}, V \in \mathbb{R}^{n\times 1}$. Furthermore, treating U

and V as vectors in $\mathbb{R}^n$, let $U = ||U||_2\hat{U}, V = ||V||_2\hat{V}$, where $\hat{U}, \hat{V}$ are unit vectors, and define $\tilde{s}(t) = \frac{1}{N}||U||_2||V||_2 s(t) \in \mathbb{R}$. Additionally, let $u(t) \in \mathbb{R}^{N_{in}}$ denote the input signal over time. Then, for $t > 0$, the dynamics of the output-generating network read:

$$\tau_x \frac{dx}{dt} = -x + \frac{1}{N} U s V^T \varphi(x) + B_{xu} u$$

$$= -x + \tilde{s}\hat{U}\hat{V}^T\varphi(x) + B_{xu}u.$$

Similarly to what was done in Section 3.3, we define the rank-1 dynamics variable $h := \hat{U}^T x$ and a residual mode $h^\perp := (I - \hat{U}\hat{U}^T)x$, so that $x = h\hat{U} + h^\perp$ is a combination of a rank-1 mode and a residual component. This gives us the dynamics equations

$$\tau_x \frac{dh}{dt} = \hat{U}^T\left(-x + \tilde{s}\hat{U}\hat{V}^T\varphi(x) + B_{xu}u\right) = -h + \tilde{s}\hat{V}^T\varphi\left(h\hat{U} + h^\perp\right) + \hat{U}^T B_{xu}u$$

$$\text{and } \tau_x \frac{dh^\perp}{dt} = \left(I - \hat{U}\hat{U}^T\right)\left(-x + \tilde{s}\hat{U}\hat{V}^T\varphi(x) + B_{xu}u\right) = -h^\perp + \left(I - \hat{U}\hat{U}^T\right)B_{xu}u.$$

Using the basic theory of ordinary differential equations, we may actually solve the latter ODE to obtain

$$h^\perp(t) = e^{-t/\tau_x}\left(h^\perp(0) + \frac{1}{\tau_x}\int_0^t e^{s/\tau_x}\left(I - \hat{U}\hat{U}^T\right)B_{xu}u(s)ds\right) \tag{5.1}$$

For instance, in the absence of any inputs, we would have $h^\perp(t) = h^\perp(0)e^{-t/\tau_x}$. For ease of notation, we define the function $J : \mathbb{R} \to \mathbb{R}^N$ by

$$J(t) = \frac{1}{\tau_x}e^{-t/\tau_x}\int_0^t e^{s/\tau_x}\left(I - \hat{U}\hat{U}^T\right)B_{xu}u(s)ds \tag{5.2}$$

$J(t)$ is a term quantifying how inputs signals shift the residual dynamics. Now, suppose our desired output signal is of the form $y = f(t)$. If our readout is given (with bias) as $y = c^T x + r = (c^T\hat{U})h + c^T h^\perp + r = (c^T\hat{U})h + (c^T h^\perp(0))e^{-t/\tau_x} + c^T J + r$, then solving for h and differentiating yields the equations

$$h = \frac{f(t) - c^T h^\perp(0)e^{-t/\tau_x} - c^T J - r}{c^T\hat{U}} \tag{5.3}$$

$$\tau_x \frac{dh}{dt} = \frac{\tau_x f'(t) + c^T h^\perp(0)e^{-t/\tau_x} - \tau_x(c^T\nabla J)}{c^T\hat{U}} \tag{5.4}$$

Now, observe that

$$s = \frac{N\tilde{s}}{||U|| \cdot ||V||} = N\frac{h + \tau_x \frac{dh}{dt} - \hat{U}^T B_x u}{||U|| \cdot ||V|| \cdot \hat{V}^T \varphi\left(h\hat{U} + h^{\perp}\right)},$$

(where $||\cdot||$ denotes the standard ℓ_2 norm), meaning that

$$s = N\frac{h + \tau_x \frac{dh}{dt} - \hat{U}^T B_x u}{||U|| \cdot V^T \varphi\left(h\hat{U} + h^{\perp}\right)} \tag{5.5}$$

If we substituted our previous expressions for h and $\tau_x \frac{dh}{dt}$ into the formula for s, we would obtain the formula

$$s = N\frac{f(t) + \tau_x f'(t) - \tau_x(c^T \nabla J) - r - \hat{U}^T B_x u}{c^T U \cdot V^T \varphi\left(\frac{f(t) - c^T h^{\perp}(0) e^{-t/\tau_x} - c^T J - r}{c^T U} U + h^{\perp}(0) e^{-t/\tau_x} + J\right)}$$

$$\approx N\frac{f(t) + \tau_x f'(t) - \tau_x(c^T \nabla J) - r - \hat{U}^T B_x u}{c^T U \cdot V^T \varphi\left(\frac{f(t) - c^T J - r}{c^T U} U + J\right)},$$

where in the latter approximation we have assumed that for $t \geq t_G$, $h^{\perp}(0)e^{-t/\tau_x}$ is effectively 0. Moreover, in the absence of any inputs to the system, the neuromodulatory signal would be

$$s = N\frac{f(t) + \tau_x f'(t) - r}{c^T U \cdot V^T \varphi\left(\frac{f(t) - c^T h^{\perp}(0) e^{-t/\tau_x} - r}{c^T U} U + h^{\perp}(0) e^{-t/\tau_x}\right)} \tag{5.6}$$

Furthermore, if we also know that that τ_x is sufficiently small and that $h^{\perp}(0)$ has sufficiently small entries, then (owing to the exponential decay of $h^{\perp}$) we may further approximate s as

$$s(t) \approx N\frac{f(t) + \tau_x f'(t) - r}{c^T U \cdot V^T \varphi\left(\frac{f(t) - r}{c^T U} U\right)}.$$

5.2 Timing Task Revisited

With the theory we have built, we revisit the Measure-Wait-Go (MWG) task (introduced in Chapter 2, aiming to verify whether the theoretically predicted neuromodulatory signals match with those of rank-1 networks in practice. In particular, we find that even

linearized rank-1 neuromodulated RNNs demonstrate the ability to generalize their performance to previously unseen time intervals, which suggests that the mathematical expression derived for their neuromodulatory signal $s(t)$ is a general solution learned by the neuromodulatory sub-network that facilitates generalization. Before diving into the analyses presented subsequently, it should be noted that all models trained satisfied the same basic methodological criteria presented in Section 4.1.1.

5.2.1 Linearized Rank-1 Networks

For NM-LR-RNNs in which the output-generating network's nonlinearity is removed (i.e. we define $\phi(x) := x$), Equation 5.6 implies that, in the absence of inputs, the neuromodulatory signal is

$$s = N\frac{f(t) + \tau_x f'(t) - r}{(V^T U)\left(f(t) - c^T h^{\perp}(0)e^{-t/\tau_x} - r\right) + (c^T U)\left(V^T h^{\perp}(0)e^{-t/\tau_x}\right)}.$$

If we assume further that $h^{\perp}(0)$ is sufficiently small and τ_x is also sufficiently small, then we may make the approximation

$$s \approx \frac{f(t) + \tau_x f'(t) - r}{\frac{V^T U}{N}\left(f(t) - r\right)} = \frac{N}{V^T U}\left(1 + \frac{\tau_x f'(t)}{f(t) - r}\right) \tag{5.7}$$

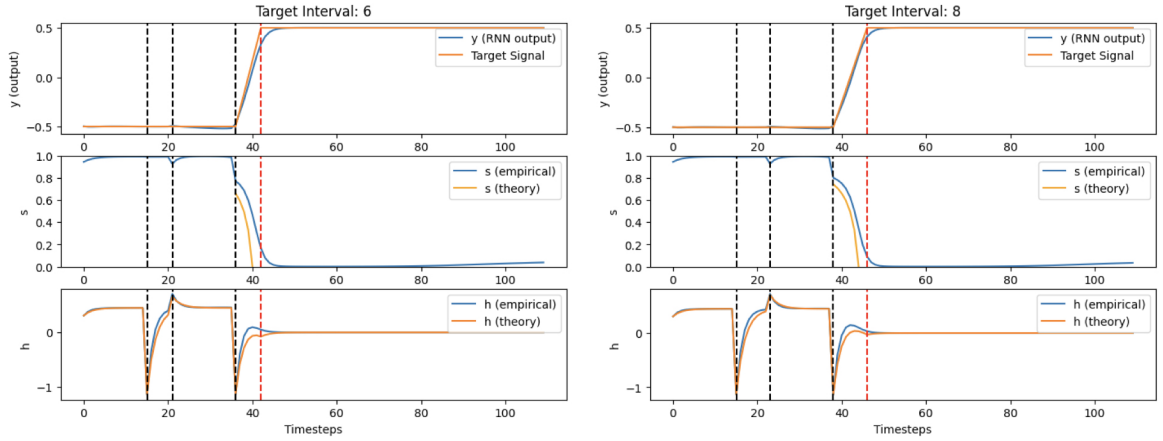
We now verify how well this theoretically predicted neuromodulatory signal matches with the empirically observed signal in an actual NM-LR-RNN. In particular, in the MWG task, no inputs are provided to the network during the ramping period following the 'go' cue, indicating that Equation 5.7 applies. Moreover, given that rank-2 and rank-3 neuromodulated RNNs demonstrate an ability to generalize (see Section 4.1.2), it is natural to wonder whether this result extends to rank-1 neuromodulated RNNs as well. Thus, we begin by training a linearized NM-LR-RNN ($M = 20$, $N = 100$, $R = 1$, $\tau_x = 2$, $\tau_z = 10$) on the MWG task (following the specifications presented in Section 4.1.1). As illustrated in Figure 5.1 such a network reliably exhibits the ability to extrapolate to larger time intervals, though not as reliably for smaller time intervals (Figure 5.1a); for these smaller intervals, the neuromodulatory signal differs from the theoretically computed signal s (as in Equation 5.7) needed to produce the desired linear ramp. For larger time intervals, the empirically observed neuromodulatory signal closely follows the theoretically prediction

from Equation 5.7. Moreover, the empirically produced neuromodulatory signal during the ramping period (following the 'go' cue) matches closely with the theoretical prediction made by Equation 5.7.

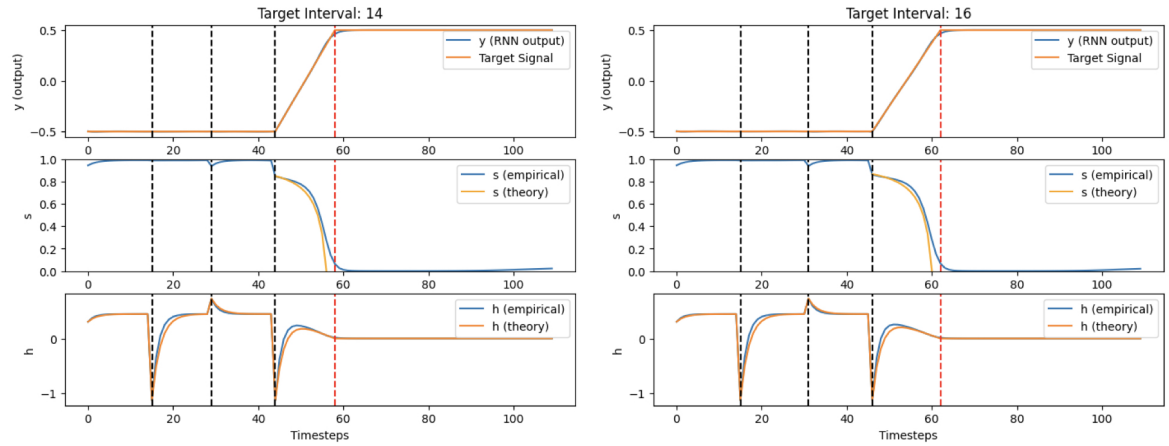
Having observed that linearized rank-1 neuromodulated RNNs *do* have the ability to generalize (primarily for larger time intervals), we hypothesize that this phenomenon emerges due to the specific nature of the neuromodulatory signal s during the ramping period. Specifically, if we shift the time axis so that $t = 0$ occurs at the 'go' cue, then from $t = 0$ to $t = T_*$, we have that $f(t) = \frac{1}{T_*}t - \frac{1}{2}$ and $f'(t) = \frac{1}{T_*}$, giving us the equation

$$s \approx \frac{N}{V^T U}\left(1 + \frac{\tau_x/T_*}{\frac{1}{T_*}t - \frac{1}{2} - r}\right).$$

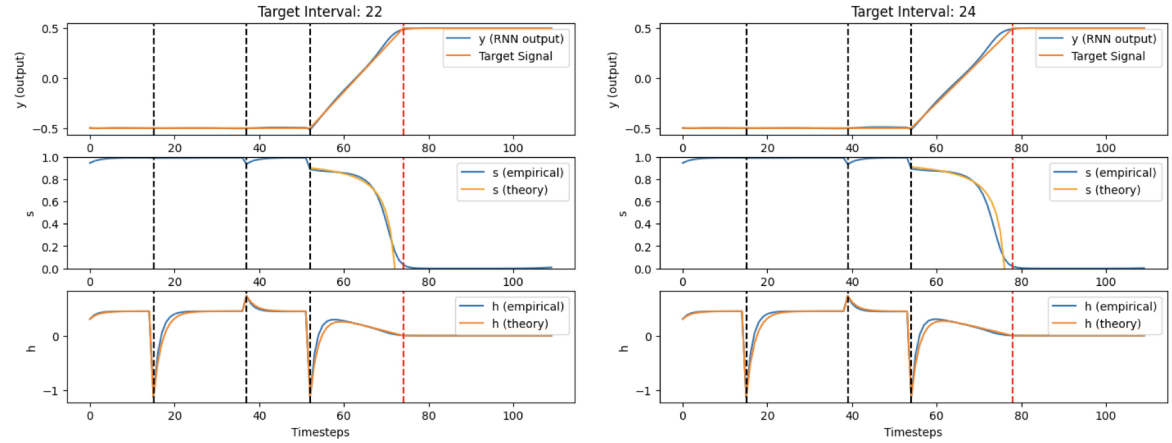
That is, s must learn to follow a kind of power law. It is then natural to wonder if learning a function of this form facilitates generalization. However, this function also depends on the vectors $U, V \in \mathbb{R}^N$, which are unknown quantities after learning. Thus, to simplify our analysis while investigating the above hypothesis, we turn to considering a further special case within the class of linearized rank-1 neuromodulated RNNs: linearized rank-1 networks with a symmetric recurrence matrix W_{xx}.



(a) Extrapolation to smaller time intervals.



(b) Time intervals on which model was trained.



(c) Extrapolation to larger time intervals.

Figure 5.1 Plots of model output (y), neuromodulatory signal (s), and the rank-1 mode h, shown across selected time intervals below, trained on, and above. For each plot, the theoretically predicted s (in order to produce the desired linear ramp) during the ramping period, as well as the theoretically required h (calculated via Equation 5.3), are shown.

5.2.2 Linearized Symmetric Rank-1 Networks

Suppose that $U = V \in \mathbb{R}^{N\times 1}$ and that $||U||_2 = ||V||_2 = \sqrt{N}$. Then, $\frac{N}{V^T U} = \frac{N}{||U||_2^2} = 1$, and so our theoretical approximation for s further simplifies to

$$s \approx 1 + \frac{\tau_x/T_*}{\frac{1}{T_*}t - \frac{1}{2} - r} \tag{5.8}$$

Conveniently, this formula is independent of the learned weight vectors $U = V$. Note firstly that we must have $|r| > \frac{1}{2}$, or else s will diverge in the interval $[0, T_*]$. Moreover, since $s \in (0, 1)$, we have that for all $t \in [0, T_*]$,

$$1 + \frac{\tau_x/T_*}{\frac{1}{T_*}t - \frac{1}{2} - r} \leq 1$$

$$\Leftrightarrow \frac{\tau_x/T_*}{\frac{1}{T_*}t - \frac{1}{2} - r} \leq 0$$

$$\Leftrightarrow \frac{1}{T_*}t - \frac{1}{2} - r \leq 0.$$

This means that $r > \frac{1}{2} \Leftrightarrow s \leq 1$.

Furthermore, after imposing the constraint that $r > \frac{1}{2}$, we must also account for the condition $s \geq 0$, i.e.,

$$1 + \frac{\tau_x/T_*}{\frac{1}{T_*}t - \frac{1}{2} - r} \geq 0$$

$$\Leftrightarrow \frac{\tau_x/T_*}{\frac{1}{T_*}t - \frac{1}{2} - r} \geq -1$$

$$\Leftrightarrow r + \frac{1}{2} - \frac{1}{T_*}t \geq \frac{\tau_x}{T_*} \ \forall t \in [0, T_*] \text{ (note that } \frac{1}{T_*}t - \frac{1}{2} - r < 0)$$

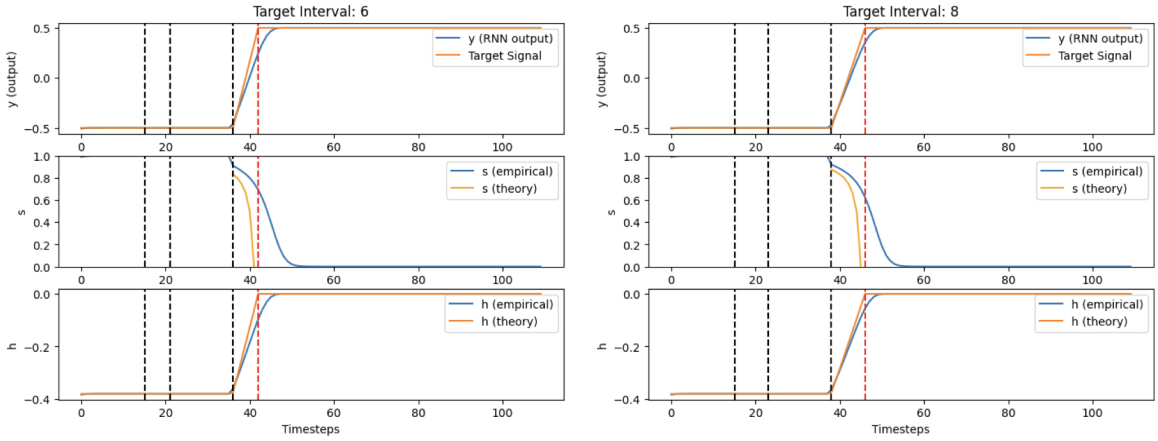
$$\Leftrightarrow r - \frac{1}{2} \geq \frac{\tau_x}{T_*}$$

$$\Leftrightarrow r \geq \frac{1}{2} + \frac{\tau_x}{T_*}.$$

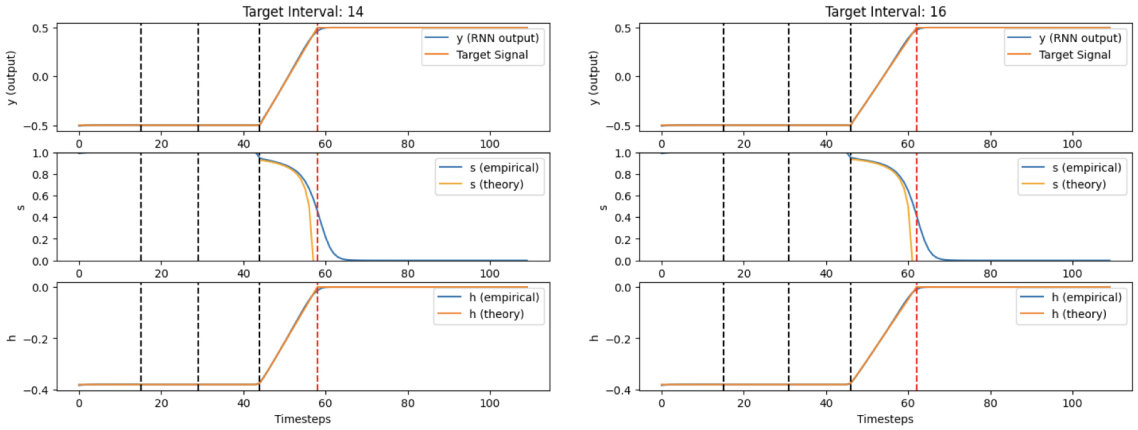
Thus, we find that $r \geq \frac{1}{2} + \frac{\tau_x}{T_*}$ for all target time intervals T_* being considered is equivalent to (the necessary condition that) $0 \leq s \leq 1$ for all such intervals T_*. Succintly, we may write that $r \geq \frac{1}{2} + \frac{\tau_x}{T_*^{\min}} \Leftrightarrow 0 \leq s \leq 1$, where $T_*^{\min}$ is the minimum target time interval that we desire to reproduce. In particular, for linearized rank-1 networks, having a nonzero readout bias r is necessary in order to correctly solve the MWG task.

Subject to the above constraint on the readout bias, we observe that if the linearized symmetric rank-1 neuromodulated RNN is able to solve the MWG task, then s must follow a decaying power law-type profile during the ramping period. Presently, we verify that rank-1 linearized symmetric NM-LR-RNNs can indeed solve the MWG task, produce the theoretically expected neuromodulatory signal, and exhibit a certain degree of generalization. We train such a model with $M = 10$, $N = 100$, $R = 1$, $\tau_x = 1$, and $\tau_z = 10$, following the training specifications presented in Section 4.1.1. Much like in the case of general linearized rank-1 networks, the linearized symmetric network exhibits reasonable generalization ability, extrapolating particularly well for larger time intervals. For these larger intervals, we observe (again) that the theoretically postulated signal matches closely the empirically produced signal.

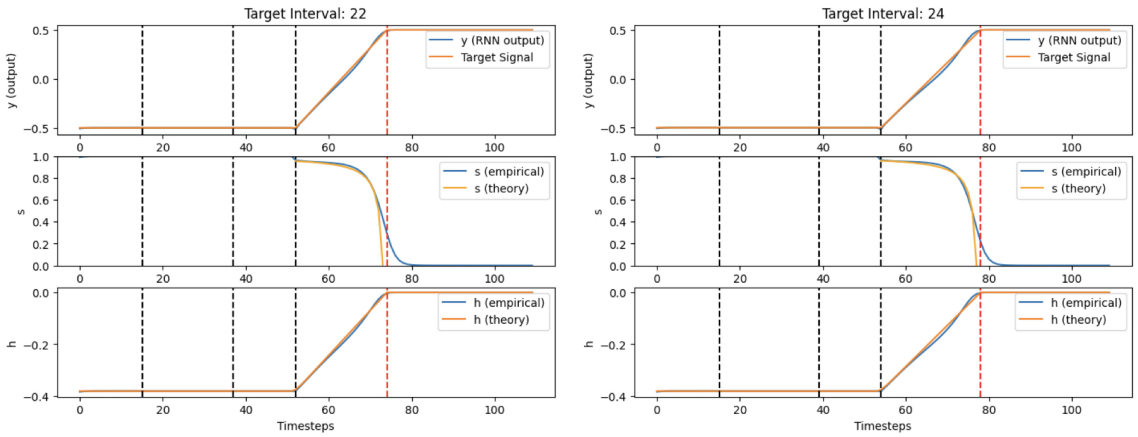
Building on these empirical results, we propose the following conjecture: if the neuromodulatory sub-network learns to produce the decaying power law function characterized by Equation 5.8, then the resulting rank-1 linearized symmetric NM-LR-RNN will be capable of solving the MWG task for general time intervals.



(a) Extrapolation to smaller time intervals.



(b) Time intervals on which model was trained.



(c) Extrapolation to larger time intervals.

Figure 5.2 Plots of a linearized symmetric NM-LR-RNN's output (y), neuromodulatory signal (s), and the rank-1 mode h, shown across selected time intervals below, trained on, and above. For each plot, the theoretically required s (computed via Equation 5.8) during the ramping period, as well as the theoretically required h (calculated via Equation 5.3), are shown.

5.3 Fixed Point Analysis

We now turn to analyzing the fixed point structure of two special classes of neuromodulated RNNs: linearized NM-LR-RNNs with a symmetric recurrence matrix W_{xx} and general rank-1 NM-LR-RNNs. For simplicity, we assume that there are no inputs to the network; while this may be seen as a limitation of our analysis, our present analysis will nonetheless shed insight into how NM-LR-RNNs perform on tasks like Measure-Wait-Go (where the inputs are zero except for at times t_M, t_W, and t_G) and, more broadly, into the conditions under which fixed points are attained.

5.3.1 Linear Symmetric NM-LR-RNNs

First, we analyze the special case in which the output-generating network has no nonlinearity and that $U = V \in \mathbb{R}^{N\times R}$ has orthonormal columns. In this case, recall from Section 3.3.2 that we may define the low-rank variable $h := U^T x \in \mathbb{R}^R$ and the decaying mode $h^\perp := \left(I - UU^T\right) x \in \mathbb{R}^N$ so that $x = Uh + h^\perp$. Moreover, recall the transformed equations governing the dynamics of a rank R linearized NM-LR-RNN with symmetric recurrence matrix (in the absence of any inputs):

$$\tau_x \frac{dh}{dt} = -(I - S)h$$

$$\text{and } \tau_x \frac{dh^\perp}{dt} = -h^\perp.$$

In particular, $h^\perp$ always decays to 0 (exponentially fast), so we focus on on the dynamics of h. Having a fixed point[1] means that $h = Sh$, which means that for each $i = 1, \ldots, R$, either $h_i = 0$ (and s_i is unconstrained) or that $s_i = 1$(and h_i is unconstrained). We can further evaluate the stability of fixed points by linearizing around any given fixed point $(h^\star, z^\star)$. For $F(h^\star, z^\star) = -(I - S)h$, $\frac{dh}{dt}|_{(h,z)=(h^\star,z^\star)} = 0 \Leftrightarrow F(h^\star, z^\star) = 0$, so for (h, z) in a sufficiently small neighborhood of $(h^\star, z^\star)$, we can linearize to obtain

$$F(h, z) \approx F(h^\star, z^\star) + (\nabla_h F)(h - h^\star) + (\nabla_z F)(z - z^\star)$$

[1]Technically, a true fixed point is one where h *and* z attain fixed points, but given that h is the only variable influencing the output y, we call the point $(h, z) \in \mathbb{R}^{R+M}$ a fixed point if $\frac{dh}{dt} = 0$ (and $\frac{dz}{dt}$ need not be zero).

$$= -(I - S)(h - h^\star) + (\nabla_z F)(z - z^\star).$$

Now, observe that, by the chain rule,

$$\nabla_z F(h^\star, z^\star) = \nabla_s F(h^\star, z^\star) \cdot \nabla_z s(z^\star),$$

and since $F(h, z) = -(I-S)h$, $\nabla_s F(h^\star, z^\star) = \text{diag}(h^\star)$. Next, noting that $s = \sigma(W_s z + b_s)$, we find that

$$\nabla_z s(z^\star) = \text{diag}\left(\sigma(W_s z^\star + b_s) \odot (1 - \sigma(W_s z^\star + b_s))\right) \cdot W_s$$

$$= \text{diag}(s^\star \odot (1 - s^\star)) \cdot W_s.$$

Consequently, we find that

$$\nabla_z F(h^\star, z^\star) = \text{diag}\left(h^\star \odot s^\star \odot (1 - s^\star)\right) \cdot W_s.$$

Thus, substituting this equation back into our linearization yields, that for all (h, z) sufficiently close to $(h^\star, z^\star)$,

$$\tau_x \frac{dh}{dt} = F(h, z) \approx -(I - S^\star)(h - h^\star) + \left(\text{diag}\left(h^\star \odot s^\star \odot (1 - s^\star)\right) \cdot W_s\right)(z - z^\star) \quad (5.9)$$

From this equation, we note the following interesting phenomena:

1. Supposing that for each $i = 1, \ldots, R$, it is true that $h_i^\star = 0$, $s_i^\star = 0$ (which implies that $h_i^\star = 0$), or $s_i^\star = 1$, the z-term above becomes zero and we find that $h^\star$ is a stable fixed point, as $-(I - S)$ has all of its eigenvalues nonpositive. Thus, if $z^\star$ is sufficiently positive or negative, $s^\star$ is effectively 1 or 0, respectively, and consequently produces a stable fixed point $h^\star$.

2. In particular, we crucially remark that $z^\star$ need *not* be a fixed point of the neuromodulatory subnetwork! As long as each z_i is sufficiently positive or sufficiently negative so that s_i is effectively 1 or 0, respectively, we may obtain a fixed point for h that is stable.

3. Suppose now that $0 < s_j^\star < 1$ and $h_j^\star \neq 0$ for some $j \in \{1, \ldots, R\}$, and that $M \in \mathbb{R}^{R \times N_M}$ is a nonzero real matrix, where N_M is the hidden dimension of the

neuromodulatory subnetwork. Then, assuming that $W_s \in \mathbb{R}^{R\times M}$ is nonzero, the second term $(\text{diag}\,(h^\star \odot s^\star \odot (1-s^\star)) \cdot W_s)\,(z-z^\star)$ need not be 0 anymore, for z close to $z^\star$. If $z^\star$ is a stable fixed point of the neuromodulatory sub-network's dynamics $\tau_z \frac{dz}{dt} = -z + W_{zz}\varphi(z)$, then for z in a suitable neighborhood of $z^\star$, z will decay back to $z^\star$ and h will decay back to $h^\star$. However, if $z^\star$ is not a stable fixed point of the neuromodulatory sub-network, then $h^\star$ is not necessarily a stable fixed point of the output-generating network.

5.3.2 Rank-1 Neuromodulated RNNs

Having built intuition by studying linearized symmetric NM-LR-RNNs, we now carry out a fixed point analysis for general rank-1 networks. To assess the fixed point structure in the case of rank-1 (not necessarily linear) dynamics (and in the absence of inputs), recall the network dynamics equations

$$\tau_x \frac{dx}{dt} = -x + \frac{1}{N} U s V^T \varphi(x) = -x + \tilde{s}\hat{U}\hat{V}^T\varphi(x)$$

$$\text{and } \tau_z \frac{dz}{dt} = -z + W_{zz}\varphi(z),$$

where $\tilde{s} := \frac{1}{N}||u||||v||$, $\hat{U} := \frac{U}{||U||}$, and $\hat{V} := \frac{V}{||V||}$.

As before, we define $h := \hat{u}^T x \in \mathbb{R}$ and $h^\perp := \left(I_{N\times N} - \hat{U}\hat{U}^T\right) x$. Then, as before, we find that $h^\perp$ always has a unique fixed point (that is stable) at 0, as

$$\tau_z \frac{dh^\perp}{dt} = \tau_z \left(I_{N\times N} - \hat{U}\hat{U}^T\right) \frac{dx}{dt} = -h^\perp.$$

Further the dynamics of h read:

$$\tau_x \frac{dh}{dt} = \hat{U}^T\left(-x + \tilde{s}\hat{U}\hat{V}^T\varphi\left(x\right)\right) = -h + \tilde{s}\hat{V}^T\varphi\left(h\hat{U} + h^\perp\right).$$

Because $h^\perp$ exponentially decays to 0, in the large time limit, we will have that $\tau_x \frac{dh}{dt} = -h + \tilde{s}\hat{V}^T\varphi\left(h\hat{U}\right) =: F(h,z)$. As before, we may linearize around any point $(h^\star, z^\star) \in \mathbb{R}^{1+M}$ that yields a fixed point $h^\star$ (of the output-generating network) to obtain

$$\tau_x \frac{dh}{dt} \approx (\nabla_h F)\,(h-h^\star) + (\nabla_z F)\,(z-z^\star)$$

$$= -\left(1 - \tilde{s}^\star \hat{V}^T \text{diag}(\varphi'(h^\star \hat{U}))\hat{U}\right)(h - h^\star) + (\nabla_s F) \cdot (\nabla_z s)(z - z^\star)$$

$$\tau_x \frac{dh}{dt} \approx -\left(1 - s^\star \cdot \frac{1}{N} V^T \text{diag}(\varphi'(h^\star \hat{U}))U\right)(h - h^\star) + \frac{||U|| \cdot s^\star(1 - s^\star)}{N} (W_s(z - z^\star)) V^T \varphi\left(h^\star \hat{U}\right) \quad (5.10)$$

In this case, we end up with the following scenarios regarding fixed point stability:

1. If $\frac{1}{N} V^T \text{diag}(\varphi'(h^\star \hat{U}))U \leq 1$ and $h^\star = 0$ or $s^\star = 1$ (note that $s^\star = 0$ implies that h decays to zero, so that $h^\star = 0$), we obtain a (marginally) stable fixed point $h^\star$, since the second term in the above equation vanishes and the term $-\left(1 - s^\star \cdot \frac{1}{N} V^T \text{diag}(\varphi'(h^\star \hat{U}))U\right)$ is nonpositive.

2. If $s^\star = 1$ and $\frac{1}{N} V^T \text{diag}(\varphi'(h^\star \hat{U}))U > 1$, then $h^\star$ is an unstable fixed point, as the second term in the above equation disappears and $-\left(1 - s^\star \cdot \frac{1}{N} V^T \text{diag}(\varphi'(h^\star \hat{U}))U\right)$ becomes positive.

3. Much like as in Section 5.3.1, if $h^\star \neq 0$ and $0 < s^\star < 1$, then the second term $\frac{||U|| \cdot s^\star(1-s^\star)}{N} (W_s(z - z^\star)) V^T \varphi\left(h\hat{U}\right)$ above may inject instability into the dynamics of h.

From these observations, as well as those presented in Section 5.3.1, we make note of a few key ideas regarding fixed point stability. First, both times in which we have linearized the dynamics around fixed points, the linearization term involving z contains a factor of $s^\star(1 - s^\star)$, suggesting that many (if not all) stable fixed points should satisfy $s^\star = 0$ or $s^\star = 1$. Qualitatively, this ties back to our analysis of neuromodulatory sub-network when solving the Element Finder Task (4.2.4). Indeed, when solving this task, the components of s usually remained at 0 or 1, and occasionally switched between the two.

Second, the above fixed point analyses provide some intuition for how the NM-LR-RNN exhibits switching nonlinear dynamics, in that the stability of fixed points can change as s (or, equivalently, z) changes. For example, in the case of rank-1 networks, suppose that $h = h^\star \neq 0$ is a fixed point, but the neuromodulatory sub-network's dynamics cause z to change so that $s_{pre} := W_s z + b_s \in \mathbb{R}$ switches from being sufficiently positive to being sufficiently negative; this corresponds to a switching of s from

1 to 0. Moreover, as s switches from 1 to 0, the z-term in Equation 5.10 – namely, $\frac{||U|| \cdot s^\star(1-s^\star)}{N} \left(W_s(z - z^\star)\right) V^T \varphi\left(h^\star \hat{U}\right)$ – will very likely become nonzero, causing $\frac{dh}{dt}$ to become nonzero. Consequently, we see that a switch from $s = 1$ to $s = 0$ can cause h to move away from $h^\star$ (and settle to a new fixed point); that is, the neuromodulatory drive s has the power to change the dynamical landscape of the low-rank mode h, and in particular, can alter the fixed point structure of the output-generating network's underlying R-dimensional dynamics.

While we primarily restricted ourselves to the case of rank-1 networks in this chapter, in the next chapter we study the setting of larger-rank *random* networks in which $N \to \infty$. In this so-called *mean-field* setting, we will similarly carry out a fixed point analysis.

Chapter 6

A Foray into Dynamical Mean Field Theory

Much of our existing theoretical analyses of the neuromodulatory signal have focused on the case of rank 1 networks (and linearized symmetric networks). This leaves the question of how neuromodulation shapes the dynamical landscape when R and N are potentially large. Luckily, there is a well-known technique for analyzing the dynamics and fixed point structure of large recurrent networks with random connections: the dynamical mean field theory (DMFT) [35, 37, 38]. In the limiting case where the size of a recurrent network tends to infinity, the DMFT averages over the disorder in such a network, enabling us to succinctly capture the prototypical dynamics by leveraging tools such as the Central Limit Theorem.

In recent years, the DMFT framework has been extended to studying low-rank RNNs [46, 58, 59]. This body of work comprehensively analyzes the mean field dynamics of rank 1 RNNs and details the possible fixed point structures arising in low-rank RNNs. In this chapter, we primarily focus on the summarizing the fixed point structure (and stability) in such networks, and subsequently extend aspects of this theory to neuromodulated low-rank RNNs. Crucially, we aim to demonstrate that incorporating neuromodulation into the low-rank RNN framework can only produce a greater number of stable fixed points – scattered across different values of the neuromodulatory signal $s(t)$ – thus enabling the NM-LR-RNN to produce richer dynamics. In particular, as we have already illustrated in earlier chapters, we will find that varying the neuromodulatory signal can cause the fixed point structure of the network to change, allowing the NM-LR-RNN to expressively

produce dynamics that involve switching from one fixed point to another.

6.1 Low-Rank Dynamics in Low-Rank RNNs

In this section, we introduce the groundwork needed to carry out the DMFT calculations for evaluating the fixed point structure in low-rank RNNs. Crucially, we capture the notion of rank-R RNNs as effectively operating in R-dimensional space (provided that the inputs are largely absent). Primarily following work by Beiran et al (2021) [58], we start with the equation governing the dynamics of a low-rank RNN:

$$\tau_x \frac{dx}{dt} = -x + \frac{1}{N} U V^T \varphi(x) + Iu.$$

Here, we use $I \in \mathbb{R}^{N \times N_{in}}$ to denote the input weights B_{xu}, for ease of notation, and φ denotes the tanh nonlinearity. We also use $u^{(1)}, \ldots, u^{(R)}$ and $v^{(1)}, \ldots, v^{(R)}$ to denote the columns of U and V, respectively. We assume that the $u^{(r)}$ are mutually orthogonal. (Note that there is a distinction between $u^{(r)}$ – a column vector of U) and u – the time-varying input to the network.) Then, the recurrence matrix may be expressed as $\frac{1}{N} \sum_{r=1}^{R} u^{(r)} v^{(r)T}$. In particular, the dynamics equation for any component x_i of x (for $i \in \{1, \ldots, N\}$) is

$$\tau_x \frac{dx_i}{dt} = -x_i + \frac{1}{N} \sum_{r=1}^{R} u_i^{(r)} \left(\sum_{j=1}^{N} v_j^{(r)} \varphi(x_j) \right) + \sum_{s=1}^{N_{in}} I_{is} u_s.$$

In particular, the neuron corresponding to x_i is characterized by its interactions with the weights $(\underline{u}_i, \underline{v}_i, \underline{I}_i) := \left(\{u_i^{(r)}\}_{r=1,\ldots,R}, \{v_i^{(r)}\}_{r=1,\ldots,R}, \{I_{is}\}_{s=1,\ldots,N_{in}} \right)$. In the setting where we apply the DMFT, we assume that the network's weights are random, so we may imagine that the weights $(\underline{u}_i, \underline{v}_i, \underline{I}_i)$ are sampled from some multivariate probability distribution, given by some mean $a \in \mathbb{R}^{2R+N_{in}}$ and covariance $\Sigma \in \mathbb{R}^{(2R+N_{in}) \times (2R+N_{in})}$:

$$a = (a_{u_1}, \ldots, a_{u_R}, a_{v_1}, \ldots, a_{v_R}, I_1, \ldots, I_{N_{in}}) \in \mathbb{R}^{2R+N_{in}}$$

$$\Sigma_{mn} = \mathbb{E}\left[(m - a_m)(n - a_n) \right]$$

across all $m, n \in \{u_i^{(r)}\}_{r=1,\ldots,R} \cup \{v_i^{(r)}\}_{r=1,\ldots,R} \cup \{I_{is}\}_{s=1,\ldots,N_{in}}$. We also define $\sigma_{uv} \in \mathbb{R}^{R \times R}$ to denote the subset of Σ that represents the covariances of u components with v com-

ponents. We similarly define σ_{uI} and σ_{vI}.

Now, observe that the hidden state $x(t)$ is always operating in some $(R + N_{in})$-dimensional subspace of $\mathbb{R}^N$, because x is in the column space spanned by U and I (and any component of x initially othogonal to this subspace will exponentially decay to 0). Thus, for each $i \in \{1, \ldots, N\}$, we may write

$$x = \sum_{r=1}^{R} \kappa_r u^{(r)} + \sum_{s=1}^{N_{in}} \kappa_{I_s} I_\perp^{(s)},$$

where $I^{(s)}$ denotes the sth column of I, and $I_\perp^{(s)}$ denotes the component of $I^{(s)}$ that is orthogonal to the column space of U. Thus, we may capture the overall dynamics of $x \in \mathbb{R}^N$ by writing out the dynamics of the κ_r and κ_{I_s}:

$$\tau_x \frac{d\kappa_r}{dt} = \tau_x \frac{u^{(r)^T}}{||u^{(r)}||^2} \frac{dx}{dt} = \frac{u^{(r)^T}}{||u^{(r)}||^2} \left(-x + \frac{1}{N} U V^T \varphi(x) + Iu \right)$$

$$\implies \tau_x \frac{d\kappa_r}{dt} = -\kappa_r + \frac{1}{N} v^{(r)^T} \varphi \left(\sum_{r'=1}^{R} \kappa_{r'} u^{(r')} + \sum_{s=1}^{N_{in}} \kappa_{I_s} I_\perp^{(s)} \right) + \frac{1}{||u^{(r)}||^2} u^{(r)^T} Iu \tag{6.1}$$

$$\text{and } \tau_x \frac{d\kappa_{I_s}}{dt} = \tau_x \frac{I_\perp^{(s)^T}}{||I_\perp^s||^2} \frac{dx}{dt} = \frac{I_\perp^{(s)^T}}{||I_\perp^s||^2} \left(-x + \frac{1}{N} U V^T \varphi(x) + Iu \right)$$

$$\implies \tau_x \frac{d\kappa_{I_s}}{dt} = -\kappa_{I_s} + \frac{1}{||I_\perp^{(s)}||^2} I_\perp^{(s)^T} Iu \tag{6.2}$$

Before diving into the DMFT calculations, we introduce one final assumption: the input signal $u(t) = 0$ for all t sufficiently large. This way, occasional nonzero inputs for small enough t might "kick" the dynamics into a new region of space, but to study the fixed points of the system in the large-time limit, we may effectively ignore the inputs. Thus, for each $r = 1, \ldots, R$, we concern ourselves with analyzing the following equation:

$$\tau_x \frac{d\kappa_r}{dt} = -\kappa_r + \frac{1}{N} v^{(r)^T} \varphi \left(\sum_{r'=1}^{R} \kappa_{r'} u^{(r')} \right) \tag{6.3}$$

6.2 Fixed Point Analysis for Low-Rank RNNs

To carry out our fixed point analysis, let us assume for simplicity that $a = 0 \in \mathbb{R}^{2R+N_{in}}$ and moreover that, for each $i = 1, \ldots, N$, $(\underline{\mathrm{u}}_i, \underline{\mathrm{v}}_i, \underline{\mathrm{I}}_i) \sim \mathcal{N}(0, \Sigma)$. Then, in the limit of $N \to \infty$, we have that

$$\kappa_r^{rec} := \frac{1}{N} v^{(r)T} \varphi\left(\sum_{r'=1}^{R} \kappa_{r'} u^{(r')}\right) \to \int d\underline{\mathrm{u}}_i d\underline{\mathrm{v}}_i P(\underline{\mathrm{u}}_i, \underline{\mathrm{v}}_i) v_i^{(r)} \varphi\left(\sum_{r'=1}^{R} \kappa_{r'} u_i^{(r')}\right) \tag{6.4}$$

for any $i = 1, \ldots, N$, where $P(\cdot)$ denotes the multivariate Normal probability density. In the limit of large N (or the *mean-field limit*), we may then invoke the following lemma:

Lemma 7. *Suppose that $(\underline{u}_i, \underline{v}_i) \sim \mathcal{N}(0, \Sigma_{u,v})$ for each $i = 1, \ldots, N$, where $\Sigma_{u,v} \in \mathbb{R}^{2R \times 2R}$ is a covariance matrix. Let σ_{uu} denote the $R \times R$ matrix embedded in $\Sigma_{u,v}$ that consists of all covariances between components of u, and let $\kappa := (\kappa_1, \ldots, \kappa_R)^T \in \mathbb{R}^R$. In the mean-field limit, we have*

$$\int d\underline{u}_i d\underline{v}_i P(\underline{u}_i, \underline{v}_i) v_i^{(r)} \varphi\left(\sum_{r'=1}^{R} \kappa_{r'} u^{(r')}\right) = \left\langle \varphi'(0, \kappa^T \sigma_{uu} \kappa) \right\rangle \left(\sum_{r'=1}^{R} \sigma_{u^{(r')} v^{(r)}} \kappa_{r'}\right),$$

where $\left\langle \varphi'(0, \kappa^T \sigma_{uu} \kappa) \right\rangle$ is given by

$$\left\langle \varphi'(0, \kappa^T \sigma_{uu} \kappa) \right\rangle = \int \mathcal{D}w \varphi'\left(w \sqrt{\kappa^T \sigma_{uu} \kappa}\right).$$

Here, $\mathcal{D}w = \frac{1}{\sqrt{2\pi}} e^{\frac{-w^2}{2}} dw$ denotes the standard Gaussian measure.

Proof. We largely follow the ideas presented in the supplement of Dubreuil et al (2022) [59], but filling in the details relevant to our current setting. Note first that because $\Sigma_{u,v} \in \mathbb{R}^{2R \times 2R}$ is a covariance matrix, it admits a Cholesky decomposition $\Sigma = BB^T$. Consequently, for each $a_i \in \{u_i^{(1)}, \ldots, u_i^{(R)}, v_i^{(1)}, \ldots, v_i^{(R)}\}$, we may write

$$a_i = \sum_{d=1}^{2R} b_{a,d} X_i^{(d)}$$

where $X_i^{(1)}, \ldots, X_i^{(2R)} \sim \mathcal{N}(0, 1)$ are iid Gaussian variables and where $b_a := (b_{a,1}, \ldots, b_{a,2R})$ is the row of B corresponding to a_i. As $\Sigma_{u,v} = BB^T$, we also have that, for any $a, b \in \{u_i^{(1)}, \ldots, u_i^{(R)}, v_i^{(1)}, \ldots, v_i^{(R)}\}$, the $\sigma_{ab} := (\Sigma_{u,v})_{ab} = b_a^T b_b$. Thus, the integral representing

κ_r^{rec} in Equation 6.4 becomes

$$\kappa_r^{rec} = \int \left(\prod_{j=1}^{2R} \mathcal{D}X_i^{(j)}\right) \left(\sum_{d=1}^{2R} b_{v^{(r)},d} X_i^{(d)}\right) \varphi\left(\sum_{r'=1}^{R} \kappa_{r'} \left(\sum_{d'=1}^{2R} b_{u^{(r')},d'} X_i^{(d')}\right)\right)$$

$$= \sum_{d=1}^{2R} b_{v^{(r)},d} \int \prod_{j\neq d} \mathcal{D}X_i^{(j)} \left(\int \mathcal{D}X_i^{(d)} X_i^{(d)} \varphi\left(\sum_{r'=1}^{R} \kappa_{r'} \left(\sum_{d'=1}^{2R} b_{u^{(r')},d'} X_i^{(d')}\right)\right)\right).$$

Here, $\mathcal{D}$ denotes the standard Gaussian measure.

We now invoke Stein's lemma for Gaussian integrals – namely, that

$$\int \mathcal{D}x\, xf(x) = \int \mathcal{D}x\, f'(x)$$

for f a bounded differentiable function. Applying Stein's lemma to the inner integral over $X_i^{(d)}$, the above expression simplifies to

$$\kappa_r^{rec} = \sum_{d=1}^{2R} b_{v^{(r)},d} \int \left(\prod_{j=1}^{2R} \mathcal{D}X_i^{(j)}\right) \left(\sum_{r'=1}^{R} \kappa_{r'} b_{u^{(r')},d}\right) \varphi'\left(\sum_{r'=1}^{R} \kappa_{r'} \left(\sum_{d'=1}^{2R} b_{u^{(r')},d'} X_i^{(d')}\right)\right)$$

$$= \left(\sum_{r'=1}^{R} \kappa_{r'} \sum_{d=1}^{2R} b_{u^{(r')},d} b_{v^{(r)},d}\right) \int \left(\prod_{j=1}^{2R} \mathcal{D}X_i^{(j)}\right) \varphi'\left(\sum_{d'=1}^{2R} \left(\sum_{r'=1}^{R} \kappa_{r'} b_{u^{(r')},d'}\right) X_i^{(d')}\right).$$

Now, by the Cholesky decomposition of $\Sigma_{u,v}$, we have that $\sum_{d=1}^{2R} b_{u^{(r)},d} b_{v^{(r)},d} = \sigma_{u^{(r)}v^{(r)}}$. Moreover, observe that, above, we are integrating a function of a weighted sum of iid Gaussian variables, meaning that we may replace this weighted sum by a single Gaussian variable whose variance is the sum of the variances of the original independent Gaussian variables:

$$\int \left(\prod_{i=1}^{n} \mathcal{D}x_i\right)\, f\left(\sum_{j=1}^{n} \theta_j x_j\right) = \int \mathcal{D}x\, f\left(x\sqrt{\sum_{j=1}^{n} \theta_j^2}\right),$$

where $x, x_1, \ldots, x_n \sim \mathcal{N}(0,1)$ are iid and $\theta_1, \ldots, \theta_n \in \mathbb{R}$. In our present setting, we have $n = 2R$ and $\theta_j := \sum_{r'=1}^{R} \kappa_{r'} b_{u^{(r')},d'}$ for each $j \in \{1, \ldots, 2R\}$. Accordingly, we first compute

$$\sum_{j=1}^{2R} \theta_j^2 = \sum_{j=1}^{2R} \left(\sum_{r'=1}^{R} \kappa_{r'} b_{u^{(r')},j}\right)^2$$

$$= \sum_{j=1}^{2R} \left(\sum_{p=1}^{R} \kappa_p b_{u^{(p)},j}\right) \left(\sum_{q=1}^{R} \kappa_q b_{u^{(q)},j}\right)$$

$$= \sum_{p=1}^{R} \sum_{q=1}^{R} \kappa_p \kappa_q \sum_{j=1}^{2R} b_{u^{(p)},j} b_{u^{(q)},j}$$

$$= \sum_{p=1}^{R} \sum_{q=1}^{R} \kappa_p \kappa_q \sigma_{u^{(p)},u^{(q)}}$$

$$= \kappa^T \sigma_{uu} \kappa.$$

Thus, substituting these simplified expressions into the equation for κ_r^{rec} gives

$$\kappa_r^{rec} = \left(\sum_{r'=1}^{R} \kappa_{r'} \sigma_{u^{(r')},v^{(r)}} \right) \int \mathcal{D}x \, \varphi' \left(x \sqrt{\kappa^T \sigma_{uu} \kappa} \right),$$

which precisely means that

$$\kappa_r^{rec} = \left\langle \varphi'(0, \kappa^T \sigma_{uu} \kappa) \right\rangle \left(\sum_{r'=1}^{R} \kappa_{r'} \sigma_{u^{(r')},v^{(r)}} \right).$$

■

Using this lemma, we may rewrite Equation 6.3 as:

$$\tau_x \frac{d\kappa_r}{dt} = -\kappa_r + \left\langle \varphi'(0, \kappa^T \sigma_{uu} \kappa) \right\rangle \left(\sum_{r'=1}^{R} \sigma_{u^{(r')}v^{(r)}} \kappa_{r'} \right) \tag{6.5}$$

Moreover, we may combine these equations across $r = 1, \ldots, R$ to obtain the following dynamics equation of $\kappa \in \mathbb{R}^R$:

$$\tau_x \frac{d\kappa}{dt} = -\kappa + \left\langle \varphi'(0, \kappa^T \sigma_{uu} \kappa) \right\rangle \sigma_{uv} \kappa \tag{6.6}$$

Thus, fixed points are characterized by solutions to the equation

$$\left\langle \varphi'(0, \kappa^T \sigma_{uu} \kappa) \right\rangle \sigma_{uv} \kappa = \kappa.$$

Certainly, $\kappa = 0$ is a trivial solution. Moreover, by nature of the above equation, any nontrivial solution for κ must be an eigenvector of σ_{uv}.

6.2.1 The Case of All Real Eigenvalues

For simplicity, let us suppose that all the eigenvalues of σ_{uv} are real and nonzero. Such a scenario is certainly plausible; for instance, if $u^{(r)} = v^{(r)}$ for each $r = 1, \dots, R$ (making W_{xx} symmetric), then $\sigma_{uv} = \sigma_{uu}$ becomes symmetric, and thus has real eigenvalues (that are nonzero with high probability). Then, consider any eigenvector κ of σ_{uv}; if we let $\hat{\kappa}$ denote the unit vector in the direction of κ, ρ denote the magnitude of κ, and λ denote the corresponding (real and nonzero) eigenvalue, then we have

$$\lambda \left\langle \varphi'(0, \rho^2 \hat{\kappa}^T \sigma_{uu} \hat{\kappa}) \right\rangle = 1 \tag{6.7}$$

Now, observe that $\left\langle \varphi'(0, \rho^2 \hat{\kappa}^T \sigma_{uu} \hat{\kappa}) \right\rangle < 1$ because for any $x \in \mathbb{R}$, $\varphi'(x) = 1 - \tanh^2(x) \in [0, 1)$. Thus, real eigenvalues that are at most 1 do not correspond to fixed points, whereas real eigenvalues that are greater than 1 may produce fixed points.

Furthermore, observe that for any $\rho > 0$, $\kappa_\rho := \pm\rho\hat{\kappa}$ is an eigenvector of σ_{uv} with eigenvalue λ. Then, as ρ goes from 0 to ∞, $\left\langle \varphi'(0, \rho^2 \hat{\kappa}^T \sigma_{uu} \hat{\kappa}) \right\rangle$ monotonically decreases from 1 to 0 (note that σ_{uu} is positive semidefinite). Thus, for any eigenvalue (of σ_{uv}) $\lambda > 1$, the left-hand side of Equation 6.7 varies from $\lambda > 1$ to 0 as $\rho \to \infty$. In particular, there is a unique solution $\rho = \rho_0$, producing the pair of fixed points $\kappa_0 := \pm\rho_0\hat{\kappa}$, that solve Equation 6.7. Thus, real eigenvalues λ that are greater than 1 produce a pair of fixed points that are symmetrically located about the origin. Since, in this case, there are R real, nonzero eigenvalues of σ_{uv}, we obtain at most $2R+1$ total fixed points (where the trivial fixed point $\kappa = 0$ is included).

More than simply searching for fixed points, we seek to understand which fixed points are *stable*. After all, stable fixed points (as opposed to unstable fixed points) are resistant to slight perturbations (e.g., from inputs) that could cause κ to leave such fixed points. As such, suppose that the R real, nonzero eigenvalues of σ_{uv} are $\lambda_1 < \lambda_2 < \cdots < \lambda_R$, and if any given λ_r is greater than 1, let $\hat{\kappa}_r$ and ρ_r denote the corresponding unit eigenvector and fixed point magnitude. We may evaluate the stability of the fixed points each eigenvalue produces by linearizing around fixed points. That is, for $F(\kappa) := -\kappa + \left\langle \varphi'(0, \kappa^T \sigma_{uu} \kappa) \right\rangle \sigma_{uv} \kappa$, linearizing around a fixed point $\kappa^\star$ corresponding to the eigenvalue

$\lambda_r > 1$ gives us that

$$\tau_x \frac{d\kappa}{dt} = F(\kappa) \approx F(\kappa^\star) + \nabla_\kappa F(\kappa^\star)(\kappa - \kappa^\star) = \nabla_\kappa \left(-\kappa + \left\langle \varphi'(0, \kappa^T \sigma_{uu} \kappa) \right\rangle \sigma_{uv} \kappa\right) |_{\kappa=\kappa^\star} (\kappa - \kappa^\star)$$

$$=: S_r(\kappa - \kappa^\star),$$

where S_r denotes the *stability matrix* for the fixed point $\kappa^\star$. Having a stable fixed point precisely means that all eigenvalues of S_r are real and negative.

We may then simplify the above gradient via the chain rule:

$$S_r = \nabla_\kappa \left(-\kappa + \left\langle \varphi'(0, \kappa^T \sigma_{uu} \kappa) \right\rangle \sigma_{uv} \kappa\right) |_{\kappa=\kappa^\star}$$

$$= -I_{R\times R} + \left\langle \varphi'(0, \kappa^{\star T} \sigma_{uu} \kappa^\star) \right\rangle \sigma_{uv} + \left[\sigma_{uv} \kappa \nabla_\kappa \left\langle \varphi'(0, \kappa^T \sigma_{uu} \kappa) \right\rangle^T\right] |_{\kappa=\kappa_0},$$

and, by differentiating under the integral, we may further compute

$$\left\langle \nabla_\kappa \varphi'(0, \kappa^T \sigma_{uu} \kappa) \right\rangle = \nabla_\kappa \int \mathcal{D}w \varphi'\left(w\sqrt{\kappa^T \sigma_{uu} \kappa}\right)$$

$$= \frac{\sigma_{uu}\kappa}{\sqrt{\kappa^T \sigma_{uu} \kappa}} \int \mathcal{D}w \varphi''\left(w\sqrt{\kappa^T \sigma_{uu} \kappa}\right) w$$

$$= \frac{\sigma_{uu}\kappa}{\sqrt{\kappa^T \sigma_{uu} \kappa}} \int \mathcal{D}w \varphi'''\left(w\sqrt{\kappa^T \sigma_{uu} \kappa}\right) \sqrt{\kappa^T \sigma_{uu} \kappa}$$

$$= \left\langle \varphi'''\left(\sqrt{\kappa^T \sigma_{uu} \kappa}\right) \right\rangle \sigma_{uu} \kappa.$$

Thus, substituting this expression into the previous gradient calculation yields

$$S_r = \nabla_\kappa \left(-\kappa + \left\langle \varphi'(0, \kappa^T \sigma_{uu} \kappa) \right\rangle \sigma_{uv} \kappa\right) |_{\kappa=\kappa^\star}$$

$$= -I_{R\times R} + \left\langle \varphi'(0, \kappa^{\star T} \sigma_{uu} \kappa^\star) \right\rangle \sigma_{uv} + \left[\left\langle \varphi'''\left(\sqrt{\kappa^T \sigma_{uu} \kappa}\right) \right\rangle \sigma_{uv} \kappa \kappa^T \sigma_{uu}\right] |_{\kappa=\kappa_0}$$

$$= -I_{R\times R} + \left\langle \varphi'(0, \kappa^{\star T} \sigma_{uu} \kappa^\star) \right\rangle \sigma_{uv} + \left\langle \varphi'''\left(\sqrt{\kappa^{\star T} \sigma_{uu} \kappa^\star}\right) \right\rangle \sigma_{uv} \kappa^\star \kappa^{\star T} \sigma_{uu}$$

$$\implies S_r = -I_{R\times R} + \left\langle \varphi'(0, \rho_r^2 \hat{\kappa}_r^T \sigma_{uu} \hat{\kappa}_r) \right\rangle \sigma_{uv} + \rho_r^2 \left\langle \varphi'''\left(\sqrt{\rho_r^2 \hat{\kappa}_r^T \sigma_{uu} \hat{\kappa}_r}\right) \right\rangle \sigma_{uv} \hat{\kappa}_r \hat{\kappa}_r^T \sigma_{uu} \quad (6.8)$$

$$\implies S_r = -I_{R\times R} + \frac{1}{\lambda_r}\sigma_{uv} + \rho_r^2\lambda_r \left\langle \varphi'''\left(\sqrt{\rho_r^2\hat{\kappa}_r^T\sigma_{uu}\hat{\kappa}_r}\right)\right\rangle \hat{\kappa}_r\hat{\kappa}_r^T\sigma_{uu} \tag{6.9}$$

where in the last step we have used Equation 6.7 and the fact that λ_r is the eigenvalue corresponding to $\hat{\kappa}_r$.

Observe first that S_r has $\hat{\kappa}_r$ as an eigenvector, which produces the eigenvalue

$$\gamma_r = \rho_r^2\lambda_r \left\langle \varphi'''\left(\sqrt{\rho_r^2\hat{\kappa}_r^T\sigma_{uu}\hat{\kappa}_r}\right)\right\rangle \left(\hat{\kappa}_r^T\sigma_{uu}\hat{\kappa}_r\right).$$

Note that, because σ_{uu} is positive semidefinite, $\hat{\kappa}_r^T\sigma_{uu}\hat{\kappa}_r \geq 0$. Moreover, we have the following lemma:

Lemma 8. *We have that*

$$\left\langle \varphi'''\left(\sqrt{\rho_r^2\hat{\kappa}_r^T\sigma_{uu}\hat{\kappa}_r}\right)\right\rangle < 0.$$

Putting these results yields $\gamma_r \leq 0$. To assess whether $\kappa^\star$ is a stable fixed point, it suffices to check that all other eigenvalues of S_r are nonpositive. In particular, since the eigenvectors $\{\hat{\kappa}_{r'}\}_{r'=1,\ldots,R}$ are linearly independent, they span $\mathbb{R}^R$, so it is enough to check that, for each $r' \in \{1,\ldots,R\}\ \{r\}$, the eigenvalues of the *projection of* S_r onto the plane spanned by $\hat{\kappa}_r$ and $\hat{\kappa}_{r'}$ are nonpositive. This projection matrix is of the form $\Pi_{r'}S_r|_{\text{span}(\hat{\kappa}_r,\hat{\kappa}_{r'})} : \text{span}\,(\hat{\kappa}_r,\hat{\kappa}_{r'}) \to \text{span}\,(\hat{\kappa}_r,\hat{\kappa}_{r'})$, where $\Pi_{r'} : \mathbb{R}^R \to \text{span}\,(\hat{\kappa}_r,\hat{\kappa}_{r'})$ is the projection onto the latter 2-dimensional subspace. This way, along each of these 2-dimensional subspaces, the dynamics about $\kappa^\star$ would be contractive, ensuring stability of this fixed point.

Thus, we compute the aforementioned projection of S_r:

$$\Pi_{r'}S_r|_{\text{span}(\hat{\kappa}_r,\hat{\kappa}_{r'})} = \Pi_{r'}\left(-I_{R\times R} + \frac{1}{\lambda_r}\sigma_{uv} + \rho_r^2\lambda_r\left\langle\varphi'''\left(\sqrt{\rho_r^2\hat{\kappa}_r^T\sigma_{uu}\hat{\kappa}_r}\right)\right\rangle\hat{\kappa}_r\hat{\kappa}_r^T\sigma_{uu}\right)|_{\text{span}(\hat{\kappa}_r,\hat{\kappa}_{r'})}$$

$$= -\Pi_{r'}|_{\text{span}(\hat{\kappa}_r,\hat{\kappa}_{r'})} + \frac{1}{\lambda_r}\Pi_{r'}\sigma_{uv}|_{\text{span}(\hat{\kappa}_r,\hat{\kappa}_{r'})} + \rho_r^2\lambda_r\left\langle\varphi'''\left(\sqrt{\rho_r^2\hat{\kappa}_r^T\sigma_{uu}\hat{\kappa}_r}\right)\right\rangle\hat{\kappa}_r\hat{\kappa}_r^T\sigma_{uu}|_{\text{span}(\hat{\kappa}_r,\hat{\kappa}_{r'})}.$$

For shorthand, we define $[S_r] := \Pi_{r'}S_r|_{\text{span}(\hat{\kappa}_r,\hat{\kappa}_{r'})}$. Taking the trace of this operator gives

$$\text{Tr}\,([S_r]) = -2 + \frac{1}{\lambda_r}(\lambda_r + \lambda_{r'}) + \rho_r^2\lambda_r\left\langle\varphi'''\left(\sqrt{\rho_r^2\hat{\kappa}_r^T\sigma_{uu}\hat{\kappa}_r}\right)\right\rangle\left(\hat{\kappa}_r^T\sigma_{uu}\hat{\kappa}_r\right)$$

$$= -1 + \frac{\lambda_{r'}}{\lambda_r} + \rho_r^2\lambda_r\left\langle\varphi'''\left(\sqrt{\rho_r^2\hat{\kappa}_r^T\sigma_{uu}\hat{\kappa}_r}\right)\right\rangle\left(\hat{\kappa}_r^T\sigma_{uu}\hat{\kappa}_r\right)$$

$$= -1 + \frac{\lambda_{r'}}{\lambda_r} + \gamma_r$$

where we use linearity of the trace and the fact that $\mathrm{Tr}(A)$ trace is the sum of the eigenvalues of the matrix A. (We also note that the third summand above, $\rho_r^2 \lambda_r \left\langle \varphi''' \left(\sqrt{\rho_r^2 \hat{\kappa}_r^T \sigma_{uu} \hat{\kappa}_r} \right) \right\rangle \hat{\kappa}_r \hat{\kappa}_r^T \sigma_{uu}|_{\mathrm{span}(\hat{\kappa}_r, \hat{\kappa}_{r'})}$, is a rank-1 operator with only nonzero eigenvalue γ_r.)

Thus, again invoking the fact that the trace gives the sum of the eigenvalues of an operator, the other eigenvalue of the operator $[S_r]$ is $\mathrm{Tr}([S_r]) - \gamma_r = -1 + \frac{\lambda_{r'}}{\lambda_r}$. This latter eigenvalue is nonpositive provided that $\lambda_{r'} < \lambda_r$ for all $r' \neq r$. Recalling that $\lambda_1 < \cdots < \lambda_R$, we see that this property only holds for the *maximum eigenvalue*, i.e., only the pair of fixed points corresponding to the maximal eigenvalue λ_R of σ_{uv} are stable! Finally, note that the trivial fixed point $\kappa = 0$ is stable because its stability matrix (as computed in Equation 6.8) simply reduces to $-I_{R \times R}$, all of whose eigenvalues are -1. In summary, we have found that for a mean-field low-rank RNN in which σ_{uv} has only real, nonzero eigenvalues (that are distinct), there are only 3 stable fixed points: 0 and the pair of fixed points corresponding to the maximal eigenvalue of σ_{uv}.

6.3 Incorporating Neuromodulation

Now that we have carried out a stability analysis of a random low-rank RNN in the mean-field regime, we incorporate neuromodulation and carry out a similar stability analysis. Again, for clarity, we carefully analyze a case involving only real eigenvalues. Again, such a case may be realized, e.g., if each $u^{(r)} = v^{(r)}$.

6.3.1 The Case of Real Eigenvalues (Revisited)

Similarly to before, we start with the premise of a mean-field NM-LR-RNN, in which there are no inputs. Moreover, suppose (again) that all the eigenvalues of σ_{uv} are real, nonzero, and distinct. However, now, Equation 6.3 becomes

$$\tau_x \frac{d\kappa_r}{dt} = -\kappa_r + \frac{1}{N} s_r v^{(r)^T} \varphi \left(\sum_{r=1}^{R} \kappa_r u^{(r)} \right) \tag{6.10}$$

where s_r is the rth component of the neuromodulatory signal $s(t)$. As before, in the mean-field limit, we have

$$\kappa_r^{rec} := \frac{1}{N} s_r v^{(r)^T} \varphi\left(\sum_{r=1}^{R} \kappa_r u^{(r)}\right) \to s_r \int d\underline{u} d\underline{v} P(\underline{u}, \underline{v}) v^{(r)^T} \varphi\left(\sum_{r=1}^{R} \kappa_r u^{(r)}\right),$$

where $P(\cdot)$ denotes the multivariate Normal probability density. Then, Lemma 7 gives that

$$s_r \int d\underline{u} d\underline{v} P(\underline{u}, \underline{v}) v^{(r)^T} \varphi\left(\sum_{r=1}^{R} \kappa_r u^{(r)}\right) = s_r \left\langle \varphi'(0, \kappa^T \sigma_{uu} \kappa) \right\rangle \left(\sum_{r'=1}^{R} \sigma_{u^{(r')} v^{(r)}} \kappa_{r'}\right).$$

Thus, the mean-field dynamics equation for κ becomes

$$\tau_x \frac{d\kappa}{dt} = -\kappa + \left\langle \varphi'(0, \kappa^T \sigma_{uu} \kappa) \right\rangle S \sigma_{uv} \kappa \tag{6.11}$$

where $S := \text{diag}(s)$. Then, aside from the trivial fixed point $\kappa = 0$, fixed points are characterized by solutions to

$$\left\langle \varphi'(0, \kappa^T \sigma_{uu} \kappa) \right\rangle S \sigma_{uv} \kappa = \kappa.$$

Now, suppose for now that S is fixed. Then, if we conduct the same analysis as in Section 6.2.1, except with $S\sigma_{uv}$ in place of σ_{uv}, we find that there are at most $2R$ nontrivial fixed points (where each pair is symmetrically positioned about the origin), and that only the pair of fixed points corresponding to the maximal eigenvalue of $S\sigma_{uv}$ is stable.

As in the case of low-rank RNNs, we assume that the eigenvalues of $S\sigma_{uv}$ are all real, distinct, and nonzero, and that they satsify $\lambda_1' < \cdots < \lambda_R'$. Moreover, let each λ_r' correspond to the unit eigenvector $\hat{\kappa}_r'$ and the magnitude ρ_r'. Now, consider any particular fixed point $\kappa^\star$, corresponding to the eigenvalue λ_r'; this means that $S^\star \sigma_{uv} \kappa^\star = \lambda_r' \kappa^\star \implies \sigma_{uv} \kappa^\star = \lambda_r' S^{\star -1} \kappa^\star$. If we repeat the exact same stability analysis that we did in low-rank RNNs, letting $F(\kappa, z) := -\kappa + \left\langle \varphi'(0, \kappa^T \sigma_{uu} \kappa) \right\rangle S \sigma_{uv} \kappa$ we can linearize the dynamics about a point $(\kappa^\star, z^\star)$:

$$\tau_x \frac{d\kappa}{dt} \approx S_r(\kappa - \kappa^\star) + \nabla_z F(z - z^\star)$$

where, similarly to before, S_r is given by

$$S_r = -I_{R\times R} + \frac{1}{\lambda_r'} S^\star \sigma_{uv} + \rho_r'^2 \lambda_r' \left\langle \varphi''' \left(\sqrt{\rho_r'^2 \hat{\kappa}_r'^T \sigma_{uu} \hat{\kappa}_r'} \right) \right\rangle \hat{\kappa}_r' \hat{\kappa}_r'^T \sigma_{uu} \tag{6.12}$$

Here, $\lambda_r' \left\langle \varphi'(0, \rho_r^2 \hat{\kappa}_r'^T \sigma_{uu} \hat{\kappa}_r') \right\rangle = 1$ (so $\lambda_r' > 1$). Moreover, using the chain rule (and noting that $s = W_s z + b_s$), we find that

$$\nabla_z F = \nabla_s F \cdot \nabla_z s = \left\langle \varphi'(0, \kappa^{\star T} \sigma_{uu} \kappa^\star) \right\rangle \mathrm{diag}\left(\sigma_{uv} \kappa^\star\right) \cdot S^\star (I_{R\times R} - S^\star) W_s$$

$$= \left\langle \varphi'(0, \kappa^{\star T} \sigma_{uu} \kappa^\star) \right\rangle \lambda_r' S^{\star -1} \mathrm{diag}(\kappa^\star) \cdot S^\star (I_{R\times R} - S^\star) W_s$$

$$= \mathrm{diag}(\kappa^\star)(I_{R\times R} - S^\star) W_s$$

where we have again invoked the fact that $\lambda_r' \left\langle \varphi'(0, \kappa^{\star T} \sigma_{uu} \kappa^\star) \right\rangle = 1$.

Putting these results together, we therefore have that, for (κ, z) sufficiently near the fixed point $(\kappa^\star, z)$ (corresponding to the eigenvalue λ_r' of $S^\star \sigma_{uv}$),

$$\tau_x \frac{d\kappa}{dt} \approx \left(-I_{R\times R} + \frac{1}{\lambda_r'} S^\star \sigma_{uv} + \lambda_r' \left\langle \varphi''' \left(\sqrt{\kappa^{\star T} \sigma_{uu} \kappa^\star} \right) \right\rangle \kappa^\star \kappa^{\star T} \sigma_{uu} \right) (\kappa - \kappa^\star)$$
$$+ \mathrm{diag}(\kappa^\star) \cdot (I_{R\times R} - S^\star) W_s (z - z^\star) \tag{6.13}$$

From this analysis, we make the following observations:

1. Based on our analysis in Section 6.2.1, we have that *each* possible fixed point $s^\star$ produced by the neuromodulatory sub-network produces a pair of fixed points that are stable in the dimensions of κ; these correspond to the maximal eigenvalue of $S^\star \sigma_{uv}$.

2. Fixed points that are stable in the dimensions of κ need not be stable in the dimensions of z unless the second summand in Equation 6.13 is zero. This latter condition can be met if for each $i = 1, \ldots, R$, $\kappa_i^\star = 0$ or $s_i^\star = 1$. It may also be met if $z^\star$ is a stable fixed point of the neuromodulatory sub-network.

3. If $z^\star$ is not a stable fixed point (or even just a fixed point), then perturbations in z can cause the dynamics to continue, causing the output-generating network to reach a different fixed point in κ.

As a special case, we note that if $S^\star = I_{R\times R}$, then we recover the fixed point structure of the mean-field low-rank RNN, giving us two stable fixed points besides the origin. However, as illustrated by the observations above, stable fixed points can often be achieved in many more ways. For instance, suppose that $u^{(r)} = v^{(r)}$ for each $r = 1, \ldots, R$ (i.e., the recurrence matrix W_{xx} is symmetric), so that $\sigma_{uv} = \sigma_{uu}$ is symmetric. Then, for any neuromodulatory signal s, the matrix $S\sigma_{uu}$ is symmetric, and hence has real eigenvalues. Accordingly, stable fixed points can be attained in many more ways than just when $S^\star = I_{R\times R}$; indeed, the condition of $\kappa_i^\star = 0$ or $s_i^\star = 1$ for each i (shown above) can be achieved in combinatorially many ways, and each of these configurations – if realized – would yield more stable fixed points. Thus, we conclude that the mean-field NM-LR-RNN has the potential to model richer dynamical systems than the mean-field low-rank RNN, owing to the former's ability to model switches in dynamics (mediated by the neuromodulatory signal s) and give rise to at least as many stable fixed points as the low-rank RNN (and potentially many more).

Chapter 7

Neuromodulated RNNs as Recurrent Switching Linear Dynamical Systems

Neuromodulated low-rank recurrent neural networks are useful models of study in neuroscience because they constitute a higher-level mechanistic framework for explaining neural mechanisms, like flexible time interval generation. However, this class of models also has a connection to lower-level descriptive frameworks that are used to infer the latent dynamics present in real neural data. In this chapter, we establish a connection between NM-LR-RNNs and Recurrent Switching Linear Dynamical Systems (rSLDS), a latent variable model that decomposes time-series data (arising from complex, nonlinear dynamics) into simpler linear dynamical systems that can switch between one another [47, 60]. In previous chapters, we have encountered the idea that the neuromodulatory signal $s(t)$ can facilitate "gate-switching" by transitioning from 0 to 1, or vice versa, which could alter the dynamical landscape of the output-generating network as well as the resulting fixed point structure. rSLDS precisely invokes this idea, whereby the dynamical landscape governing a continuous latent variable (that produces the output) can be altered by a second (discrete) latent variable. Furthermore, rSLDS has found applications in inferring the latent dynamics present in real neural data [61, 48]. As such, if we connect neuromodulated RNNs to the rSLDS framework, then we may view NM-LR-RNNs not only as mechanistic models of neural computation but also potentially as descriptive models that can be used to analyze real neural dynamics.

7.1 rSLDS Model Design

rSLDS is a framework that uses two time-varying latent variables – a continuous state x_t and a discrete state z_t – that use a set of parameters θ to update at each timestep. For any time-series data $y_t \in \mathbb{R}^D$, this is essentially treated as the output of the system; $x_t \in \mathbb{R}^N$ and $z_t \in \mathbb{R}^R$ are the latent variables giving rise to the observed time-series data (e.g., neural activity). At the timestep t, the update for $x_t \in \mathbb{R}^N$ is given by

$$x_t = A_{z_t} x_{t-1} + b_{z_t} + \epsilon_t \tag{7.1}$$

where $\epsilon_t \sim \mathcal{N}(0, \Sigma_{z_t})$ is Gaussian noise that is sampled iid across timesteps with covariance matrix $\Sigma_{z_t} \in \mathbb{R}^{N \times N}$, and where $A_{z_t} \in \mathbb{R}^{N \times N}, b_{z_t} \in \mathbb{R}^N$ are learnable parameters specific to the discrete state z_t. Next, the update for the predicted output (i.e., the prediction of the actual data itself for that timestep) is given by

$$y_t = C_{z_t} x_t + r_{z_t} + \delta_t \tag{7.2}$$

where $\delta_t \sim \mathcal{N}(0, \Delta_{z_t})$ is sampled iid across timesteps with covariance matrix $\Delta_{z_t} \in \mathbb{R}^{D \times D}$, and where $C_{z_t} \in \mathbb{R}^{D \times N}, r_{z_t} \in \mathbb{R}^D$ are learnable parameters. Finally, we update the discrete state z_t. In the ordinary Switching Linear Dynamical Systems (SLDS) setup, z_t switches according to the Markovian dynamics z_t [62, 63]; however, the point of adding recurrence to the system is to add a dependence of x_t to the update equation for z_t, so as to produce more "intelligent" switches in dynamics. To that end, the update for z_t is given by

$$z_t | z_{t-1}, x_{t-1} \sim \pi(\nu_t) \tag{7.3}$$

$$\nu_t = R_{z_{t-1}} x_{t-1} + q_{z_{t-1}} \tag{7.4}$$

where π denotes a chosen probability distribution and $R_{z_t} \in \mathbb{R}^{R \times N}, q_{z_t} \in \mathbb{R}^R$ are learnable.

Within this chapter, we assume that π denotes a multivariate Normal distribution with some fixed covariance, so that $z_t | z_{t-1}, x_{t-1} \sim \mathcal{N}(\nu_t, Q)$, for some covariance matrix $Q \in \mathbb{R}^{R \times R}$. Although the fact that we are modeling the latter conditional distribution as a continuous one is in conflict with the fact that z_t is traditionally modeled as a discrete variable in rSLDS, we may bypass this issue by viewing z_t as a limiting case where there

are infinitely many discrete states. That is, for $\alpha > 0$ sufficiently small, we may imagine partioning $\mathbb{R}^R$ into the countable collection of all rectangles of the form

$$R_{n_1 n_2 \ldots n_R} := [n_1\alpha, (n_1+1)\alpha] \times \cdots \times [n_R\alpha, (n_R+1)\alpha]$$

across all $n_1, \ldots, n_R \in \mathbb{Z}$ and subsquently identifying z_t with this discrete set of rectangles. This way, to define a notion of $z_t|z_{t-1}, x_{t-1}$, we first sample a specific value z_{t-1} inside the rectangle it belongs to. Then, we sample z_t from the Normal distribution $\mathcal{N}(\nu_t, Q)$. Finally, we identify which rectangle z_t belongs to and take this to be the updated discrete state. The process of discretizing z_t into rectangular "buckets" and sampling from them introduces compounding noise in measurements, but we may curb this noise by taking $\alpha \to 0$. (Another way to justify modeling the conditional distribution from Equation 7.3 as being Gaussian is to extend the rSLDS framework to allow z_t to be sampled from a set of uncountably infinite states, which characterizes the set $\mathbb{R}^R$.)

The graphical model for rSLDS is shown in Figure 7.1.

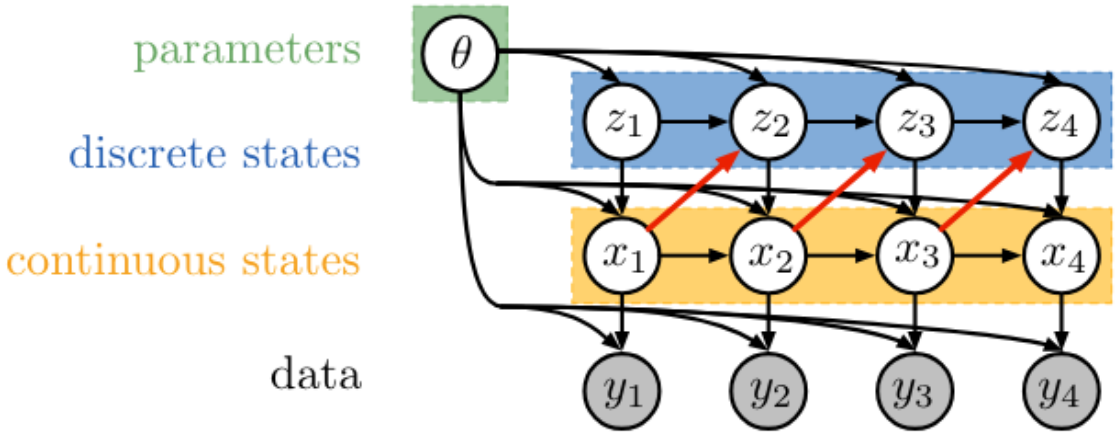


Figure 7.1 Graphical model illustrating the rSLDS framework [47]. The observed time-series data (y) is assumed to be generated by a continuous latent variable (x), which in turn depends on a discrete latent variable (z). In rSLDS, updating the latent state z also depends on x.

7.2 Mapping the Fully Linearized Coupled NM-LR-RNN onto the rSLDS Framework

Having defined the rSLDS framework, we presently turn to illustrating how the NM-LR-RNN is effectively an instance of rSLDS. Recall the dynamics equations governing the NM-LR-RNN *with* feedback coupling:

$$\tau_x \frac{dx}{dt} = -x + \frac{1}{N} USV^T \varphi(x) + B_{xu} u \tag{7.5}$$

$$\tau_z \frac{dz}{dt} = -z + W_{zz}\varphi(z) + (B_{zx}\varphi(x) + b_z) + B_{zu} u \tag{7.6}$$

Here, N is the dimension of the low-rank RNN, M is the dimension of the neuromodulatory network, and φ denotes the tanh nonlinearity unless otherwise stated. For the purposes of our argument, we assume that $M \leq R$, where R is the rank of the output-generating network. Furthermore, we may assume that $\tau_x = 1$ and $\tau_z > 1$. Moreover, for the present setting, assume that we have purely linear dynamics, i.e., $\varphi = \mathrm{Id}$ (rather than tanh) and that $b_z = 0$. That is, our model is the fully linearized and coupled NM-LR-RNN (see Definition 2), and has dynamics given by the (discretized) equations

$$x_{t+1} = \frac{1}{N} US_t V^T x_t + B_{xu} u_t \tag{7.7}$$

$$z_{t+1} = \left((1 - \frac{1}{\tau_z}) I_{M\times M} + \frac{1}{\tau_z} W_{zz} \right) z_t + \frac{1}{\tau_z} \left(B_{zx} x_t + B_{zu} u_t \right) \tag{7.8}$$

$$y_t = C x_t + r \tag{7.9}$$

Note that these dynamics equations are simply the discretized analogs of those in Definition 2 (with $\tau_x = 1, \tau_z > 1$), and note that we exclude a coupling bias term for the z_t update. Furthermore, note that the form of rSLDS we are considering involves no external inputs (e.g., u_t) and does involve noise; the model uses a generative process to predict the observed time-series data of interest. To get closer to this setting, we assume that the "objective" of the linearized coupled NM-LR-RNN is to faithfully reproduce some time-series data with its outputs y_t and that the inputs $u_t \sim \mathcal{N}(0, \Lambda)$ are Gaussian white noise (and iid over timesteps), where $\Lambda \in \mathbb{R}^{N_{in} \times N_{in}}$ is some fixed covariance matrix.

Then, we may rewrite the latent dynamics equations of the NM-LR-RNN as

$$x_{t+1} = \frac{1}{N} U S_t V^T x_t + \epsilon_t, \; S_t = \text{diag}(\sigma(W_s z_t + b_s)) \tag{7.10}$$

$$z_{t+1} = \left((1 - \frac{1}{\tau_z}) I_{M \times M} + \frac{1}{\tau_z} W_{zz} \right) z_t + \frac{1}{\tau_z} B_{zx} x_t + \delta_t \tag{7.11}$$

$$y_{t+1} = C x_t + r \tag{7.12}$$

where $\epsilon_t \sim \mathcal{N}(0, B_{xu} \Lambda \Lambda^T B_{xu}^T)$ and $\delta_t \sim \mathcal{N}(0, \frac{1}{\tau_z^2} B_{zu} \Lambda \Lambda^T B_{zu}^T)$ are sampled iid across timesteps. In this setting, we will show that the NM-LR-RNN is an instance of rSLDS.

Previous work by Valente et al (2022) has highlighted an equivalence between Linear Dynamical Systems (LDS) and linearized low-rank RNNs under suitable conditions [64]. In the following theorem, we take inspiration from the argument in Valente et al (2022) in showing that the linearized coupled NM-LR-RNN may be viewed as an instance of rSLDS:

Theorem 9. *Consider a fully linearized and coupled NM-LR-RNN whose update equations are given by Equations 7.10-7.12. Moreover, suppose that $M \leq R$ and that W_s has rank M. Then, this model is precisely an instance of an rSLDS (as detailed in Section 7.1) that has the same outputs y_t, and that uses the continuous latent states $v_t := H^T x_t \in \mathbb{R}^h$ and discrete[1] latent states $\eta_t := W_s z_t + b_s \in \mathbb{R}^R$. Here, $H : \mathbb{R}^h \to \mathbb{R}^N$ is some fixed matrix whose columns are orthonormal vectors and that collectively span the sum of the column spaces of U, V, and B_{xu} (note that, by definition, the number of columns of H is $h \leq 2R + N_{in}$).*

Proof. Using the matrix H as defined in the theorem statement, we may first view the dynamical equation for the output-generating network as a lower-dynamical linear dynamical system in the variable $v_t := H^T x_t$. (Note that, since U and V each have R columns, if we assume from the outset that $R << N$ and that the input dimension $N_{in} << N$, then v_t operates in some h-dimensional space, where $h \leq 2R + U << N$ dimensional space.) By construction of H, we also have $HH^T U = U$, $HH^T V = V$, and $HH^T B_{xu} = B_{xu}$. Then, observe moreover that

$$HH^T \frac{1}{N} U S V^T H H^T = \frac{1}{N} (HH^T U) S (HH^T V)^T = \frac{1}{N} U S V^T.$$

[1]Here, when we say 'discrete', we refer to the approach of discretizing a continuous variable over the countable collection of rectangles, as discussed in Section 7.1.

Then, we have

$$\begin{aligned} v_{t+1} = H^T x_{t+1} &= H^T \frac{1}{N} U S_{t+1} V^T x_t + H^T \epsilon_t \\ &= H^T H H^T \frac{1}{N} U S_{t+1} V^T H H^T x_t + H^T \epsilon \\ &= \frac{1}{N} H^T U S_{t+1} V^T H v_t + H^T \epsilon_t \\ &= A_{\eta_{t+1}} v_t + \epsilon'_t, \end{aligned}$$

where $A_{\eta_{t+1}} := \frac{1}{N} C^T U S_{t+1} V^T C = \frac{1}{N} C^T U \text{diag}(\sigma(\eta_{t+1})) V^T C$ (where $\eta_t = W_s z_t + b_s$ and σ denotes the sigmoid function) and $\epsilon'_t \sim \mathcal{N}\left(0, H^T B_{xu} \Lambda \Lambda^T B_{xu}^T H\right)$. Moreover, observe that $H v_t = H H^T x_t = v_t$ because x_t always lies in the column space of H. Thus, we may view v_t as the latent variable of a linear dynamical system with readout given by $y_t = C x_t + r = C H v_t + r$. That is, for v_t representing the continuous state of the rSLDS model, the readout parameters CH and r are constant across all possible values of η_t. Thus, we so far mapped the output-generating network's dynamics as well as the linear readout onto the rSLDS framework.

Note that the latent LDS dynamics of x_t are influenced by $S_t = \text{diag}(\sigma(\eta_t))$, whose diagonal components can "switch" depending on the sign of the state η_t; in turn, this affects the recurrence matrix $W_{xx} = \frac{1}{N} U S_t V$.

Now, we derive an update equation for η_t based on the update equation for z_t in the NM-LR-RNN. This is where the "recurrent switching" aspect of rSLDS enters the picture. Observe that

$$\begin{aligned} \eta_{t+1} &= W_s z_{t+1} + b_s \\ &= W_s \left(\left(1 - \frac{1}{\tau_z}\right) I_{M \times M} + \frac{1}{\tau_z} W_{zz} \right) z_t + \frac{1}{\tau_z} W_s B_{zx} x_t + W_s \delta_t + b_s \\ &= W_s \left(\left(1 - \frac{1}{\tau_z}\right) I_{M \times M} + \frac{1}{\tau_z} W_{zz} \right) W_s^\dagger (\eta_t - b_s) + \frac{1}{\tau_z} W_s B_{zx} H v_t + W_s \delta_t + b_s \\ &= \left(\left(1 - \frac{1}{\tau_z}\right) W_s W_s^\dagger + \frac{1}{\tau_z} W_s W_{zz} W_s^\dagger \right) \eta_t + \frac{1}{\tau_z} W_s B_{zx} H v_t \\ &\quad + \left(I_{M \times M} - \left(1 - \frac{1}{\tau_z}\right) W_s W_s^\dagger - W_s W_{zz} W_s^\dagger \right) b_s + \delta'_t, \end{aligned}$$

where $W_s^\dagger = (W_s^T W_s)^{-1} W_s^T$ is the left inverse of W_s. Indeed, since $W_s : \mathbb{R}^M \to \mathbb{R}^R$ with $R \geq M$ and W_s having rank M, W_s has $W_s^\dagger$ as a left inverse. Moreover, $\delta'_t \sim \mathcal{N}(0, \frac{1}{\tau_z^2} W_s B_{zu} \Lambda \Lambda^T B_{zu}^T W_s^T)$.

Owing to the Gaussian noise term δ_t, we may conditionally model $\eta_{t+1}|\eta_t, v_t \sim \mathcal{N}(\nu_t, G)$, with $\nu_t = Rv_t + r_{\eta_t}$, where

$$R = \frac{1}{\tau_z} W_s B_{zx} H$$

$$r_{\eta_t} = \left(\left(1 - \frac{1}{\tau_z}\right) W_s W_s^\dagger + \frac{1}{\tau_z} W_s W_{zz} W_s^\dagger\right) \eta_t + \left(I_{M\times M} - \left(1 - \frac{1}{\tau_z}\right) W_s W_s^\dagger - W_s W_{zz} W_s^\dagger\right) b_s$$

$$\text{and } G = \frac{1}{\tau_z^2} W_s B_{zu} \Lambda\Lambda^T B_{zu}^T W_s^T.$$

Thus, we have also mapped the update for the discretized state η_t onto rSLDS. We conclude that the fully linearized and coupled NM-LR-RNN is an instance of rSLDS. ■

The upshot of this analysis is that the linearized NM-LR-RNN can be viewed as a recurrent switching linear dynamical system, further highlighting the NM-LR-RNN's ability to switch the dynamical landscape of the output-generating network's state variable x_t based on the neuromodulatory latent variable z_t. Furthermore, given the applications of rSLDS to inferring the simple "component" dynamical systems that compose real neural dynamics, this suggests the potential for NM-LR-RNNs to serve as expressive models that capture the low-dimensional dynamical modes underlying high-dimensional neural activity.

Chapter 8

Conclusion

8.1 Summary

The goal of this thesis has been to develop and explore a mechanistic account of neuromodulation into the recurrent neural network (RNN) framework. Whereas ordinary RNNs typically utilize static weights, real synaptic strengths in the brain are constantly in flux, both at the level of individual neurons and at the population level. Neuromodulators play a crucial role in dynamically modulating the strengths of neurons and their connections, and in this work we have hypothesized that neuromodulation enables large neural ensembles to produce flexible, generalizable behavior.

To test this hypothesis, we have designed the neuromodulated low-rank RNN (NM-LR-RNN); consisting of a pair of coupled RNNs, this model involves a neuromodulatory sub-network providing a neuromodulatory signal that scales the low-rank recurrent weights of an output-generating low-rank RNN (Chapter 3). The fact that (a relatively low-dimensional) neuromodulatory signal is able to dynamically update each rank-1 factor of a low-rank recurrence matrix allows for greater interpretability of how neuromodulation affects the dynamics of the output-generating network as well as how it is constrained by the desired output signal (see Chapter 5). As implemented in the NM-LR-RNN, neuromodulation also constitutes a spatial and temporal gating mechanism that closely resembles the LSTM in machine learning, as well as the rSLDS framework, as it is used for inferring latent dynamics in real neural populations (Chapters 3 and 7).

Through empirical simulations, we do indeed find that the NM-LR-RNN demonstrates an advantage over low-rank and full-rank RNNs in generalizing to unseen data on a flexible control-based timing task (Measure-Wait-Go); even rank-1 (linearized) neuromodulated RNNs – when equipped with the right neuromodulatory signal – demonstrate the ability

to generalize (see Chapters 4 and 5). The NM-LR-RNN also succeeds in capturing long-term dependencies (much like the LSTM) on the Element Finder Task, unlike ordinary RNNs (Chapter 4). In particular, for this task, we find that the neuromodulatory and output-generating sub-networks take on dual roles, with the former attending to the query index and shifting the dynamics of the latter accordingly, so that the latter may successfully recover the correct target element value.

Finally, in a more theoretical setting, we have carried out fixed point analyses of the NM-LR-RNN (Chapters 5 and 6). These analyses illustrate how each component of the neuromodulatory signal $s(t)$ saturating to 0 or 1 tends to produce stable fixed points, as well as how changes in s can change the fixed point structure attained by the output-generating network. Our exploration of the NM-LR-RNN in the mean-field setting, in particular, has illustrated the ability of the NM-LR-RNN to produce a different fixed point configuration associated with each possible neuromodulatory signal, potentially enabling this model to model richer dynamical systems (and, in particular, *switches* between different sets of dynamics) than ordinary LR-RNNs in the mean-field regime.

Altogether, the NM-LR-RNN constitutes a promising first step towards understanding the role of neuromodulators in shaping neural dynamics. It provides several hypotheses about the role of neuromodulation (e.g., neuromodulation constitues a spatial and temporal gating mechanism) that may be tested in neuroscience experiments. However, given the range of different neuromodulators in the brain, and the different neural circuits in which these chemical agents are involved, much work remains to be done in studying the computational features of neuromodulation.

8.2 Future Work

On the modeling side, a number of steps for future work remain. For instance, the networks trained and tested in Chapters 4 and 5 were relatively small-scale. Future work would involve testing the scalability of these models, as well as analyzing their performance on more complex, higher-dimensional tasks – especially compared to ordinary RNNs and gated models like LSTMs. Additionally, neuromodulation need not only target the recurrent weights of the output-generating network. Consequently, future work would also involve incorporating neuromodulatory input and output gates to the NM-LR-RNN (in addition to the existing "forget gate"-like neuromodulation of W_{xx}); this

would likely further enhance the correspondence between this model and the LSTM.

Future work would also involve adding further neuromodulator-related biophysical details to the RNN framework. For example, one avenue for research would be to vary the sparsity level of the rank-1 recurrent factors of W_{xx} to account for the ability of neuromodulators to both target entire neural populations as well as localized subsets of neurons. Furthermore, one could also imagine allowing $s(t)$ to be positive or negative to account for neuromodulation being excitatory and inhibitory. Finally, any specific neuromodulator has its own computational features; to that end, future work could involve building an RNN model with multiple kinds of neuromodulatory signals, each with its own distinctive features.

More broadly, a future research direction is that of bridging mechanistic models (like RNNs) with descriptive models for analyzing real neural dynamics (like rSLDS). Having explored the connection between the NM-LR-RNN and rSLDS in Chapter 7, future work could involve comparing the internal states of the NM-LR-RNN against real neural data and neuromodulatory signals in the brain; moreover, mechanistic models like the NM-LR-RNN may even have the potential to act as descriptive models, being used to predict the neural activity *and* neuromodulatory signals that arise from various environmental stimuli.

Ultimately, we hope that the work presented in this thesis constitutes a further step towards demystifying the computational role of neuromodulators in the brain, especially with regards to how they may help facilitate the generation of flexible behaviors. While a full understanding of neuromodulation remains to be achieved, further studying the function and dysfunction of neuromodulators may have translational implications in the long-term.

References

[1] Evan D. Remington, Devika Narain, Eghbal A. Hosseini, and Mehrdad Jazayeri. Flexible sensorimotor computations through rapid reconfiguration of cortical dynamics. *bioRxiv*, 2018. doi: 10.1101/261214. URL https://www.biorxiv.org/content/early/2018/02/07/261214.

[2] Warren S. McCulloch and Walter Pitts. A logical calculus of the ideas immanent in nervous activity. *The Bulletin of Mathematical Biophysics*, 5(4):115–133, 1943. doi: 10.1007/bf02478259.

[3] F. Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408, 1958. ISSN 0033-295X. doi: 10.1037/h0042519. URL http://dx.doi.org/10.1037/h0042519.

[4] David E. Rumelhart, James L. McClelland, and CORPORATE PDP Research Group, editors. *Parallel distributed processing: explorations in the microstructure of cognition, vol. 1: foundations*. MIT Press, Cambridge, MA, USA, 1986. ISBN 026268053X.

[5] James L McClelland and Jeffrey L Elman. The trace model of speech perception. *Cognitive Psychology*, 18(1):1–86, 1986. ISSN 0010-0285. doi: https://doi.org/10.1016/0010-0285(86)90015-0. URL https://www.sciencedirect.com/science/article/pii/0010028586900150.

[6] David E. Rumelhart and James L. McClelland. On learning the past-tenses of english verbs: implicit rules or parallel distributed processing. 1986. URL https://api.semanticscholar.org/CorpusID:56499839.

[7] James L. McClelland and David E. Rumelhart. An interactive activation model of

context effects in letter perception: part 1.: an account of basic findings. 1988. URL https://api.semanticscholar.org/CorpusID:15118845.

[8] Jeffrey L. Elman. Distributed representations, simple recurrent networks, and grammatical structure. *Mach. Learn.*, 7(2–3):195–225, sep 1991. ISSN 0885-6125. doi: 10.1007/BF00114844. URL https://doi.org/10.1007/BF00114844.

[9] J. J. Hopfield. Neural networks and physical systems with emergent collective computational abilities. *Proceedings of the National Academy of Sciences of the United States of America*, 79(8):2554–2558, April 1982. ISSN 0027-8424. URL http://view.ncbi.nlm.nih.gov/pubmed/6953413].

[10] Bruce L McNaughton and Richard GM Morris. Hippocampal synaptic enhancement and information storage within a distributed memory system. *Trends in neurosciences*, 10(10):408–415, 1987.

[11] Randall C. O'Reilly and James L. McClelland. Hippocampal conjunctive encoding, storage, and recall: Avoiding a trade-off. *Hippocampus*, 4(6):661–682, 1994. doi: https://doi.org/10.1002/hipo.450040605. URL https://onlinelibrary.wiley.com/doi/abs/10.1002/hipo.450040605.

[12] James L. McClelland, Bruce L. McNaughton, and Randall C. O'Reilly. Why there are complementary learning systems in the hippocampus and neocortex: Insights from the successes and failures of connectionist models of learning and memory. *Psychological Review*, 102(3):419–457, 1995. doi: 10.1037/0033-295x.102.3.419.

[13] James L. McClelland and Timothy T. Rogers. The parallel distributed processing approach to semantic cognition. *Nature Reviews Neuroscience*, 4:310–322, 2003. URL https://api.semanticscholar.org/CorpusID:684322.

[14] Earl K. Miller and Jonathan D. Cohen. An integrative theory of prefrontal cortex function. *Annual Review of Neuroscience*, 24(Volume 24, 2001):167–202, 2001. ISSN 1545-4126. doi: https://doi.org/10.1146/annurev.neuro.24.1.167. URL https://www.annualreviews.org/content/journals/10.1146/annurev.neuro.24.1.167.

[15] Matthew M. Botvinick, Cameron S. Carter, Todd S. Braver, Deanna M. Barch, and Jonathan D. Cohen. Conflict monitoring and cognitive control. *Psychological Review*, 108(3):624–652, July 2001. ISSN 0033-295X. doi: 10.1037/0033-295X.108.3.624.

[16] Stephen Grossberg. Adaptive pattern classification and universal recoding: 1. parallel development and coding of neural feature detectors. *Biological cybernetics*, 23 (3):121–134, 1976.

[17] Stephen Grossberg. Adaptive pattern classification and universal recoding: 2. feedback, expectation, olfaction, illusions. *Biological cybernetics*, 23(4):187–202, 1976.

[18] David E. Rumelhart and David Zipser. Feature discovery by competitive learning*. *Cognitive Science*, 9(1):75–112, 1985. doi: https://doi.org/10.1207/s15516709cog0901_5. URL `https://onlinelibrary.wiley.com/doi/abs/10.1207/s15516709cog0901_5`.

[19] Chr. von der Malsburg. (1973) Chr. von der Malsburg, S̈elf-organization of orientation sensitive cells in the striate cortex,K̈ybernetik 14: 85-100. In *Neurocomputing, Volume 1: Foundations of Research*. The MIT Press, 04 1988. ISBN 9780262267137. doi: 10.7551/mitpress/4943.003.0018. URL `https://doi.org/10.7551/mitpress/4943.003.0018`.

[20] Timothy T. Rogers and James L. McClelland. Parallel distributed processing at 25: Further explorations in the microstructure of cognition. *Cognitive Science*, 38(6): 1024–1077, 2014. doi: 10.1111/cogs.12148.

[21] Blake Aaron Richards, Timothy P. Lillicrap, Philippe Beaudoin, Yoshua Bengio, Rafał Bogacz, Amelia J. Christensen, Claudia Clopath, Rui Ponte Costa, Archy O. de Berker, Surya Ganguli, Colleen J. Gillon, Danijar Hafner, Adam Kepecs, Nikolaus Kriegeskorte, Peter E. Latham, Grace W. Lindsay, Kenneth D. Miller, Richard Naud, Christopher C. Pack, Panayiota Poirazi, Pieter R. Roelfsema, João Sacramento, Andrew M. Saxe, Benjamin Scellier, Anna C. Schapiro, Walter Senn, Greg Wayne, Daniel L. K. Yamins, Friedemann Zenke, Joel Zylberberg, Denis Thérien, and Konrad Paul Kording. A deep learning framework for neuroscience. *Nature Neuroscience*, 22:1761 – 1770, 2019. URL `https://api.semanticscholar.org/CorpusID:204948269`.

[22] Andrew M. Saxe, James L. McClelland, and Surya Ganguli. A mathematical theory of semantic development in deep neural networks. *Proceedings of the National Academy of Sciences*, 116(23):11537–11546, 2019. doi: 10.1073/pnas.1820226116. URL https://www.pnas.org/doi/abs/10.1073/pnas.1820226116.

[23] Taylor W. Webb, Steven M. Frankland, Awni Altabaa, Simon Segert, Kamesh Krishnamurthy, Declan Campbell, Jacob Russin, Tyler Giallanza, Zack Dulberg, Randall O'Reilly, John Lafferty, and Jonathan D. Cohen. The relational bottleneck as an inductive bias for efficient abstraction, 2024.

[24] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540): 529–533, February 2015. ISSN 00280836. URL http://dx.doi.org/10.1038/nature14236.

[25] Daniel L. K. Yamins, Ha Hong, Charles F. Cadieu, Ethan A. Solomon, Darren Seibert, and James J. DiCarlo. Performance-optimized hierarchical models predict neural responses in higher visual cortex. *Proceedings of the National Academy of Sciences*, 111(23):8619–8624, 2014. doi: 10.1073/pnas.1403112111. URL https://www.pnas.org/doi/abs/10.1073/pnas.1403112111.

[26] James C. R. Whittington, Joseph Warren, and Tim E.J. Behrens. Relating transformers to models and neural representations of the hippocampal formation. In *International Conference on Learning Representations*, 2022. URL https://openreview.net/forum?id=B8DVo9B1YE0.

[27] A.L. Hodgkin and A.F. Huxley. A quantitative description of membrane current and its application to conduction and excitation in nerve. *Journal of Physiology*, 117: 500–544, 1952.

[28] Kenneth D. Miller, Joseph B. Keller, and Michael P. Stryker. Ocular dominance column development: Analysis and simulation. *Science*, 245(4918):605–615, 1989.

doi: 10.1126/science.2762813. URL https://www.science.org/doi/abs/10.1126/science.2762813.

[29] Sen Song, Kenneth D. Miller, and L. F. Abbott. Competitive hebbian learning through spike-timing-dependent synaptic plasticity. *Nature Neuroscience*, 3:919–926, 2000. URL https://api.semanticscholar.org/CorpusID:9530143.

[30] Stefano Fusi, Patrick J. Drew, and L.F. Abbott. Cascade models of synaptically stored memories. *Neuron*, 45(4):599–611, 2005. ISSN 0896-6273. doi: https://doi.org/10.1016/j.neuron.2005.02.001. URL https://www.sciencedirect.com/science/article/pii/S0896627305001170.

[31] Chethan Pandarinath, Daniel J. O'Shea, Jasmine Collins, Rafal Jozefowicz, Sergey D. Stavisky, Jonathan C. Kao, Eric M. Trautmann, Matthew T. Kaufman, Stephen I. Ryu, Leigh R. Hochberg, Jaimie M. Henderson, Krishna V. Shenoy, L. F. Abbott, and David Sussillo. Inferring single-trial neural population dynamics using sequential auto-encoders. *bioRxiv*, 2017. doi: 10.1101/152884. URL https://www.biorxiv.org/content/early/2017/06/20/152884.

[32] Jeffrey E Markowitz, Winthrop F Gillis, Celia C Beron, Shay Q Neufeld, Keiramarie Robertson, Neha D Bhagat, Ralph E Peterson, Emalee Peterson, Minsuk Hyun, Scott W Linderman, Bernardo L Sabatini, and Sandeep Robert Datta. The striatum organizes 3d behavior via moment-to-moment action selection. *Cell*, 174 (1):44—58.e17, June 2018. ISSN 0092-8674. doi: 10.1016/j.cell.2018.04.019. URL https://europepmc.org/articles/PMC6026065.

[33] Scott W. Linderman, Annika L. A. Nichols, David M. Blei, Manuel Zimmer, and Liam Paninski. Hierarchical recurrent state space models reveal discrete and continuous dynamics of neural activity in c. elegans. *bioRxiv*, 2019. URL https://api.semanticscholar.org/CorpusID:155402644.

[34] Saurabh Vyas, Matthew D. Golub, David Sussillo, and Krishna V. Shenoy. Computation through neural population dynamics. *Annual Review of Neuroscience*, 43 (Volume 43, 2020):249–275, 2020. ISSN 1545-4126. doi: https://doi.org/10.1146/annurev-neuro-092619-094115. URL https://www.annualreviews.org/content/journals/10.1146/annurev-neuro-092619-094115.

[35] H. Sompolinsky, A. Crisanti, and H. J. Sommers. Chaos in random neural networks. *Phys. Rev. Lett.*, 61:259–262, Jul 1988. doi: 10.1103/PhysRevLett.61.259. URL https://link.aps.org/doi/10.1103/PhysRevLett.61.259.

[36] David Sussillo and L. F. Abbott. Generating coherent patterns of activity from chaotic neural networks. *Neuron*, 63(4):544–557, aug 2009. ISSN 1097-4199. doi: 10.1016/j.neuron.2009.07.018. URL http://dx.doi.org/10.1016/j.neuron.2009.07.018.

[37] Kanaka Rajan, L. F. Abbott, and Haim Sompolinsky. Stimulus-dependent suppression of chaos in recurrent neural networks. *Phys. Rev. E*, 82:011903, Jul 2010. doi: 10.1103/PhysRevE.82.011903. URL https://link.aps.org/doi/10.1103/PhysRevE.82.011903.

[38] Jonathan Kadmon and Haim Sompolinsky. Transition to chaos in random neuronal networks. *Phys. Rev. X*, 5:041030, Nov 2015. doi: 10.1103/PhysRevX.5.041030. URL https://link.aps.org/doi/10.1103/PhysRevX.5.041030.

[39] David Sussillo. Neural circuits as computational dynamical systems. *Current Opinion in Neurobiology*, 25:156–163, 2014. ISSN 0959-4388. doi: https://doi.org/10.1016/j.conb.2014.01.008. URL https://www.sciencedirect.com/science/article/pii/S0959438814000166. Theoretical and computational neuroscience.

[40] Alex H. Williams, Albert W. Hamood, and Eve Marder. *Neuromodulation in Small Networks*, pages 1–15. Springer New York, New York, NY, 2013. ISBN 978-1-4614-7320-6. doi: 10.1007/978-1-4614-7320-6_26-2. URL https://doi.org/10.1007/978-1-4614-7320-6_26-2.

[41] Peter Dayan. Twenty-five lessons from computational neuromodulation. *Neuron*, 76(1):240–256, 2012. ISSN 0896-6273. doi: https://doi.org/10.1016/j.neuron.2012.09.027. URL https://www.sciencedirect.com/science/article/pii/S0896627312008628.

[42] Eve Marder. Neuromodulation of neuronal circuits: back to the future. *Neuron*, 76 (1):1–11, 2012.

[43] L F Abbott. Modulation of function and gated learning in a network memory. *Proceedings of the National Academy of Sciences*, 87(23):9241–9245, 1990. doi: 10.1073/pnas.87.23.9241. URL https://www.pnas.org/doi/abs/10.1073/pnas.87.23.9241.

[44] Ann Kennedy, Prabhat S Kunwar, Ling-yun Li, Stefanos Stagkourakis, Daniel A Wagenaar, and David J Anderson. Stimulus-specific hypothalamic encoding of a persistent defensive state. *Nature*, 586(7831):730–734, 2020.

[45] Ben Tsuda, Stefan C. Pate, Kay M. Tye, Hava T. Siegelmann, and Terrence J. Sejnowski. Neuromodulators generate multiple context-relevant behaviors in a recurrent neural network by shifting activity hypertubes. *bioRxiv*, 2022. doi: 10.1101/2021.05.31.446462. URL https://www.biorxiv.org/content/early/2022/01/03/2021.05.31.446462.

[46] Francesca Mastrogiuseppe and Srdjan Ostojic. Linking connectivity, dynamics, and computations in low-rank recurrent neural networks. *Neuron*, 99(3):609–623.e29, August 2018. ISSN 0896-6273. doi: 10.1016/j.neuron.2018.07.003. URL http://dx.doi.org/10.1016/j.neuron.2018.07.003.

[47] Scott W. Linderman*, Matthew J. Johnson*, Andrew C. Miller, Ryan P. Adams, David M. Blei, and Liam Paninski. Bayesian learning and inference in recurrent switching linear dynamical systems. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2017.

[48] David Zoltowski, Jonathan Pillow, and Scott Linderman. A general recurrent state space framework for modeling neural dynamics during decision-making. In Hal Daumé III and Aarti Singh, editors, *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 11680–11691. PMLR, 13–18 Jul 2020. URL https://proceedings.mlr.press/v119/zoltowski20a.html.

[49] Mattia Rigotti, Omri Barak, Melissa R. Warden, Xiao-Jing Wang, Nathaniel D. Daw, Earl K. Miller, and Stefano Fusi. The importance of mixed selectivity in complex cognitive tasks. *Nature*, 497:585–590, 2013. URL https://api.semanticscholar.org/CorpusID:1146461.

[50] H.S. Seung. How the brain keeps the eyesstill. *Proceedings of the National Academy of Sciences*, 93(23):13339–13344, 1996. doi: 10.1073/pnas.93.23.13339. URL https://www.pnas.org/doi/abs/10.1073/pnas.93.23.13339.

[51] Richard J. Gardner, Erik Hermansen, Marius Pachitariu, Yoram Burak, Nils A. Baas, Benjamin A. Dunn, May-Britt Moser, and Edvard I. Moser. Toroidal topology of population activity in grid cells. *bioRxiv*, 2021. doi: 10.1101/2021.02.25.432776. URL https://www.biorxiv.org/content/early/2021/02/25/2021.02.25.432776.

[52] Manuel Beiran, Nicolas Meirhaeghe, Hansem Sohn, Mehrdad Jazayeri, and Srdjan Ostojic. Parametric control of flexible timing through low-dimensional neural manifolds. *bioRxiv*, 2021. doi: 10.1101/2021.11.08.467806. URL https://www.biorxiv.org/content/early/2021/11/10/2021.11.08.467806.

[53] Cooper D. Grossman and Jeremiah Y. Cohen. Neuromodulation and neurophysiology on the timescale of learning and decision-making. *Annual review of neuroscience*, 45:317–337, July 2022. ISSN 0147-006X. doi: 10.1146/annurev-neuro-092021-125059. Publisher Copyright: Copyright © 2022 by Annual Reviews.

[54] Mark W. Howe, Patrick L. Tierney, Stefan G. Sandberg, Paul E. M. Phillips, and Ann M. Graybiel. Prolonged dopamine signalling in striatum signals proximity and value of distant rewards. *Nature*, 500(7464):575–579, August 2013. doi: 10.1038/nature12475. URL https://ideas.repec.org/a/nat/nature/v500y2013i7464d10.1038_nature12475.html.

[55] Kamesh Krishnamurthy, Tankut Can, and David J. Schwab. Theory of gating in recurrent neural networks. *Phys. Rev. X*, 12:011011, Jan 2022. doi: 10.1103/PhysRevX.12.011011. URL https://link.aps.org/doi/10.1103/PhysRevX.12.011011.

[56] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Comput.*, 9(8):1735–1780, nov 1997. ISSN 0899-7667. doi: 10.1162/neco.1997.9.8.1735. URL https://doi.org/10.1162/neco.1997.9.8.1735.

[57] Yong Yu, Xiaosheng Si, Changhua Hu, and Jianxun Zhang. A Review of Recurrent Neural Networks: LSTM Cells and Network Architectures. *Neural Computation*, 31

(7):1235–1270, 07 2019. ISSN 0899-7667. doi: 10.1162/neco_a_01199. URL https://doi.org/10.1162/neco_a_01199.

[58] Manuel Beiran, Alexis Dubreuil, Adrian Valente, Francesca Mastrogiuseppe, and Srdjan Ostojic. Shaping Dynamics With Multiple Populations in Low-Rank Recurrent Networks. *Neural Computation*, 33(6):1572–1615, 05 2021. ISSN 0899-7667. doi: 10.1162/neco_a_01381. URL https://doi.org/10.1162/neco_a_01381.

[59] Alexis Dubreuil, Adrian Valente, Manuel Beiran, Francesca Mastrogiuseppe, and Srdjan Ostojic. The role of population structure in computations through neural dynamics. *bioRxiv*, 2021. doi: 10.1101/2020.07.03.185942. URL https://www.biorxiv.org/content/early/2021/06/23/2020.07.03.185942.

[60] David Barber. Expectation correction for smoothed inference in switching linear dynamical systems. *Journal of Machine Learning Research*, 7(89):2515–2540, 2006. URL http://jmlr.org/papers/v7/barber06a.html.

[61] Josue Nassar, Scott Linderman, Monica Bugallo, and Il Memming Park. Tree-structured recurrent switching linear dynamical systems for multi-scale modeling. In *International Conference on Learning Representations*, 2019. URL https://openreview.net/forum?id=HkzRQhR9YX.

[62] Zoubin Ghahramani and Geoffrey E. Hinton. Switching state-space models. 1996. URL https://api.semanticscholar.org/CorpusID:16365840.

[63] Kevin P Murphy. Switching kalman filters. 1998.

[64] Adrian Valente, Srdjan Ostojic, and Jonathan Pillow. Probing the relationship between latent linear dynamical systems and low-rank recurrent neural network models. *Neural Computation*, 34(9):1871–1892, 08 2022. ISSN 0899-7667. doi: 10.1162/neco_a_01522. URL https://doi.org/10.1162/neco_a_01522.